

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

JOÃO VITOR DO AMARAL SPOLAVORE

**CANM: Mecanismo de Otimização de Custo
por Migração de Nós entre Pools
Heterogêneos em Ambientes Orquestrados
pelo Kubernetes**

Monografia apresentada como requisito parcial
para a obtenção do grau de Bacharel em Ciência
da Computação

Orientador: Prof. Dr. Arthur Francisco Lorenzon

Porto Alegre
2026

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Reitora: Prof^a. Marcia Cristina Bernardes Barbosa

Vice-Reitor: Prof. Pedro de Almeida Costa

Pró-Reitor de Graduação: Prof^a. Nádyá Pesce da Silveira

Diretor do Instituto de Informática: Prof. Luciano Paschoal Gaspary

Coordenador do Curso de Ciência de Computação: Prof. Álvaro Freitas Moreira

Bibliotecário-chefe do Instituto de Informática: Alexsander Borges Ribeiro

RESUMO

A otimização de custos é um dos principais desafios na adoção de infraestrutura em nuvem orquestrada por contêineres, na qual recursos provisionados e subutilizados continuam a gerar custos sob o modelo *pay-as-you-go*. Nesse contexto, os mecanismos nativos de autoescalamento do *Kubernetes* mostram-se limitados quando é preciso manter uma quantidade mínima de nós em execução ou não se deseja alterar o número total de máquinas provisionadas. Este trabalho propõe o CANM, *Cost Aware Node Migrator*, um mecanismo que, em vez de expandir ou retrain a frota, migra as cargas de trabalho entre *pools* de diferentes capacidades e custos, reduzindo o custo em períodos de ociosidade e atendendo a aumentos de demanda pela migração da carga para máquinas de maior desempenho, e não pela adição de instâncias. A avaliação foi conduzida em uma infraestrutura real do Google Cloud, reproduzindo a carga de trabalho de sete microsserviços de produção da Econodata – predominantemente limitados por operações de E/S – por meio de dois perfis de tráfego (dia útil e fim de semana) calibrados a partir de volumes reais (cerca de 605 mil requisições diárias em dias úteis e 285 mil nos finais de semana). Frente a um cluster homogêneo composto exclusivamente pelas máquinas de maior desempenho (*high pool*), adotado como *baseline*, o CANM reduziu o custo de operação entre 4,8 % e 22,7 % no horizonte medido de 2 horas – economia que, projetada para 24 horas, cresce para a faixa de 18,1 % a 29,7 %, conforme o perfil de carga. As migrações sob alta carga impuseram impactos transitórios de latência e de erros, enquanto em períodos ociosos a degradação foi praticamente nula. Constatou-se, por fim, que o desempenho do mecanismo depende fortemente da arquitetura do software e das características do cluster.

Palavras-chave: Computação em nuvem. Kubernetes. Otimização de custos. Migração de nós. Orquestração de contêineres.

CANM: A Cost Optimization Mechanism through Node Migration across Heterogeneous Pools in Kubernetes-Orchestrated Environments

ABSTRACT

Cost optimization is one of the central challenges in adopting container-orchestrated cloud infrastructure, where provisioned and underutilized resources keep incurring costs under the *pay-as-you-go* model. In this context, native *Kubernetes* autoscaling mechanisms prove limited when a minimum number of nodes must remain running, or when the total number of provisioned machines should not be changed. This work proposes CANM, *Cost Aware Node Migrator*, a mechanism that, instead of expanding or shrinking the fleet, migrates workloads between pools of different capacities and costs, reducing cost during idle periods and meeting demand spikes by migrating the workload to higher-performance machines rather than by adding instances. The evaluation was carried out on a real Google Cloud infrastructure, reproducing the workload of seven production microservices from Econodata – predominantly I/O-bound – through two traffic profiles (weekday and weekend) calibrated from real volumes (about 605,000 daily requests on weekdays and 285,000 on weekends). Compared to a homogeneous cluster composed exclusively of the higher-performance machines (*high pool*), adopted as the baseline, CANM reduced the operating cost between 4.8% and 22.7% over the measured 2-hour horizon – savings that, projected to 24 hours, grow to a range of 18.1% to 29.7%, depending on the load profile. Migrations performed under high load imposed transient impacts on latency and error rate, whereas during idle periods the degradation was nearly negligible. Finally, the mechanism’s performance was found to depend strongly on the software architecture and on the cluster’s characteristics.

Keywords: Cloud computing. Kubernetes. Cost optimization. Node migration. Container orchestration.

LISTA DE FIGURAS

Figura 2.1	Representação dos tipos de hipervisores	18
Figura 2.2	Diferença de arquitetura entre VMs e ambientes containerizados	20
Figura 2.3	Arquitetura de um cluster <i>Kubernetes</i>	26
Figura 4.1	Etapas da migração de um nó do <i>low pool</i> para o <i>high pool</i>	46
Figura 5.1	Utilização média de vCPU por nó em dias úteis	55
Figura 5.2	Utilização média de vCPU por nó nos finais de semana	56
Figura 5.3	Utilização média de memória por nó em dias úteis	57
Figura 5.4	Utilização média de memória por nó nos finais de semana	57
Figura 5.5	Carga incremental de VUs	59
Figura 5.6	Taxas de erros ao longo dos testes de carga	59
Figura 5.7	Utilização vCPU média do cluster ao longo dos testes de carga	60
Figura 5.8	Vazão de requisições média do cluster ao longo dos testes de carga	60
Figura 5.9	VUs por perfil de carga	61
Figura 5.10	Estabilização da utilização de CPU de nós recém-criados pelo CANM	65
Figura 6.1	Vazão média de requisições do <i>profile</i> de dia útil no <i>low pool</i>	67
Figura 6.2	Utilização média de vCPU do cluster no <i>profile</i> de dia útil no <i>low pool</i>	68
Figura 6.3	Erros em requisições do <i>profile</i> de dia útil no <i>low pool</i>	68
Figura 6.4	Latência de requisições do <i>profile</i> de dia útil no <i>low pool</i>	69
Figura 6.5	Vazão média de requisições do <i>profile</i> de dia útil no <i>high pool</i>	69
Figura 6.6	Utilização média de vCPU do cluster no <i>profile</i> de dia útil no <i>high pool</i>	70
Figura 6.7	Erros em requisições do <i>profile</i> de dia útil no <i>high pool</i>	70
Figura 6.8	Latência de requisições do <i>profile</i> de dia útil no <i>high pool</i>	71
Figura 6.9	Vazão média de requisições do <i>profile</i> de fim de semana no <i>low pool</i>	71
Figura 6.10	Utilização média de vCPU do cluster no <i>profile</i> de fim de semana no <i>low pool</i>	72
Figura 6.11	Erros em requisições do <i>profile</i> de fim de semana no <i>low pool</i>	72
Figura 6.12	Latência de requisições do <i>profile</i> de fim de semana no <i>low pool</i>	72
Figura 6.13	Vazão média de requisições do <i>profile</i> de fim de semana no <i>high pool</i>	73
Figura 6.14	Utilização média de vCPU do cluster no <i>profile</i> de fim de semana no <i>high pool</i>	73
Figura 6.15	Erros em requisições do <i>profile</i> de fim de semana no <i>high pool</i>	73
Figura 6.16	Latência de requisições do <i>profile</i> de fim de semana no <i>high pool</i>	74
Figura 6.17	Vazão média de requisições do <i>profile</i> de dia útil com o CANM ativado ...	75
Figura 6.18	Composição dos nós do cluster ao longo do tempo no <i>profile</i> de dia útil com o CANM ativado	75
Figura 6.19	Latência de requisições do <i>profile</i> de dia útil com o CANM ativado	77
Figura 6.20	Erros em requisições do <i>profile</i> de dia útil com o CANM ativado	77
Figura 6.21	Vazão média de requisições do <i>profile</i> de fim de semana com o CANM ativado	79
Figura 6.22	Composição dos nós do cluster ao longo do tempo no <i>profile</i> de fim de semana com o CANM ativado	79
Figura 6.23	Erros em requisições do <i>profile</i> de fim de semana com o CANM ativado ..	80
Figura 6.24	Latência de requisições do <i>profile</i> de fim de semana com o CANM ativado ..	80
Figura 6.25	Composição projetada do cluster ao longo de 24 horas no <i>profile</i> de dia útil com o CANM ativado	83

Figura 6.26 Composição projetada do cluster ao longo de 24 horas no <i>profile</i> de fim de semana com o CANM ativado	84
--	----

LISTA DE TABELAS

Tabela 2.1	Exemplos de instâncias de máquinas virtuais na Google Cloud Platform	18
Tabela 4.1	Exemplificação de <i>pools</i> de nós com equivalência de recurso	39
Tabela 4.2	Principais parâmetros de configuração do CANM e seus efeitos.....	49
Tabela 5.1	Microserviços e rotas mais utilizadas da arquitetura alvo	54
Tabela 5.2	Média diária de requisições por rota — dia útil e fim de semana	55
Tabela 5.3	Relação entre a quantidade de usuários virtuais e o percentual do limiar configurado	61
Tabela 6.1	Métricas médias dos cenários de referência no <i>profile</i> de dia útil.....	71
Tabela 6.2	Métricas médias dos cenários de referência no <i>profile</i> de fim de semana	74
Tabela 6.3	Comparativo das métricas médias do <i>profile</i> de dia útil entre os cenários de referência e o CANM	78
Tabela 6.4	Comparativo das métricas médias do <i>profile</i> de fim de semana entre os cenários de referência e o CANM.....	81
Tabela 6.5	Comparativo de custo de computação com e sem a atuação do CANM	82

LISTA DE ABREVIATURAS E SIGLAS

GCP	Google Cloud Platform
IaaS	Infraestructure as a Service
PaaS	Platform as a Service
SaaS	Software as a Service
API	Application Programming Interface
HPA	Horizontal Pod Autoscaling
VPA	Vertical Pod Autoscaling
CA	Cluster Autoscaling
QoS	Quality of Service
CANM	Cost Aware Node Migrator

SUMÁRIO

1 INTRODUÇÃO	11
1.1 Objetivos	12
1.2 Estrutura do texto	13
2 REVISÃO BIBLIOGRÁFICA.....	14
2.1 Computação na Nuvem	14
2.1.1 Provedores de Serviço em Nuvem	15
2.1.2 Modelos de serviço de computação em nuvem	16
2.2 Máquinas virtuais	16
2.3 Ambientes Containerizados.....	19
2.4 Kubernetes.....	21
2.4.1 Objetos do Kubernetes	22
2.4.2 Agendamento de Nós e Anotações	23
2.4.3 Arquitetura do Cluster.....	24
2.4.4 Autoescalonamento.....	25
3 TRABALHOS RELACIONADOS	28
3.1 Otimização de Desempenho e Custo no Escalonamento de Clusters	28
3.2 Estratégias de Provisionamento de Recursos em Nível de Aplicação	32
3.3 Avaliação de Desempenho de Autoescalonadores	34
4 PROPOSTA DO CANM.....	36
4.1 Visão Geral e Objetivo.....	36
4.2 Ciclo de operação	37
4.2.1 Dimensionalidade	38
4.2.2 Pontuação dos Nós.....	39
4.3 Migração de Nós.....	41
4.3.1 Etapas da Migração.....	43
4.4 Arquitetura	46
4.4.1 Reconciliação.....	47
4.4.2 Parametrização e Adaptabilidade.....	48
4.5 Componentes	49
4.5.1 Migrator Orchestrator	50
4.5.2 Node Migrator.....	50
4.5.3 Metrics Adapter	50
4.5.4 Kubernetes Client.....	51
4.5.5 Audit Logger.....	51
5 METODOLOGIA	52
5.1 Ambiente de testes.....	52
5.2 Arquitetura alvo.....	53
5.3 Caracterização da carga de trabalho	54
5.4 Reprodução da carga de trabalho	57
5.4.1 Intensidade da carga.....	58
5.4.2 Cenários de experimentação	61
5.5 Configuração experimental do CANM	63
5.5.1 Parâmetros de decisão de migração	63
5.5.2 Parâmetros de temporização	64
5.5.3 Parâmetros de espera de nós recém-criados.....	64
6 ANÁLISE DOS RESULTADOS	66
6.1 Cenários de referência: um achado contraintuitivo	67
6.1.1 Profile dia útil.....	67

6.1.2 Profile fim de semana.....	71
6.2 O CANM em operação	74
6.2.1 Profile dia útil.....	74
6.2.2 Profile fim de semana.....	79
6.3 Economia de custo.....	81
7 CONCLUSÃO E TRABALHOS FUTUROS	85
REFERÊNCIAS.....	86

1 INTRODUÇÃO

A crescente demanda por recursos computacionais, impulsionada pela expansão de sistemas distribuídos e aplicações de larga escala, motivou a adoção da computação em nuvem como modelo predominante para a execução de cargas de trabalho. Esse paradigma caracteriza-se pela disponibilização sob demanda de recursos computacionais por meio da internet, permitindo que organizações executem suas aplicações em infraestruturas remotas mantidas por terceiros, sem a necessidade de expandir continuamente um parque computacional próprio (MELL; GRANCE, 2011). Dentre os benefícios proporcionados por esse modelo, destacam-se a elasticidade dos recursos, que possibilita ajustar a capacidade computacional conforme a variação da demanda, e a facilidade de escalar sistemas distribuídos, uma vez que o provisionamento de novas máquinas pode ser realizado de forma ágil e sob demanda. Aliado a isso, o modelo de cobrança *pay-as-you-go*, no qual o usuário paga apenas pelos recursos efetivamente utilizados, torna a nuvem uma alternativa economicamente atrativa frente à manutenção de uma infraestrutura *on-premises*.

Nesse contexto, à medida que arquiteturas baseadas em microserviços e em ambientes containerizados ganharam adesão, tornou-se necessário o emprego de ferramentas capazes de orquestrar e gerenciar contêineres em larga escala, a fim de escalar automaticamente a quantidade de instâncias, garantir redundância e evitar erros humanos decorrentes da operação manual. O *Kubernetes* (BURNS et al., 2016) apresenta-se como uma solução para essa finalidade, sendo um orquestrador de código aberto responsável por automatizar a implantação, o escalonamento e o gerenciamento de aplicações containerizadas em ambientes distribuídos. Ao operar sobre clusters compostos por múltiplos nós, o *Kubernetes* viabiliza a execução resiliente de sistemas, integrando-se naturalmente às infraestruturas de computação em nuvem e usufruindo da elasticidade por elas oferecida.

Para garantir que a capacidade do cluster acompanhe as variações da carga de trabalho, o ecossistema *Kubernetes* disponibiliza mecanismos de autoescalonamento, responsáveis por ajustar dinamicamente os recursos provisionados. Esses mecanismos atuam expandindo a capacidade computacional em períodos de maior demanda e retraindo-a em situações de ociosidade, respondendo automaticamente às flutuações na utilização dos recursos. Dessa forma, o autoescalonamento torna-se fundamental para a otimização dos custos de provisionamento de máquinas, uma vez que evita a manutenção de recursos ociosos e adequa a infraestrutura à necessidade real das aplicações.

Entretanto, esses mecanismos apresentam limitações em determinados cenários

operacionais. Muitas aplicações exigem uma quantidade mínima de nós em execução constante, seja para garantir alta disponibilidade, redundância ou o atendimento de requisitos arquiteturais específicos. Nesses casos, ainda que as máquinas operem sob baixa utilização, os autoescaladores são incapazes de reduzir a infraestrutura além do limite inferior pré-estabelecido (TAMIRU et al., 2020), mantendo recursos subutilizados que continuam a gerar custos operacionais (REISS et al., 2012; CORTEZ et al., 2017). Essa limitação evidencia uma lacuna na otimização de custos, especialmente em situações nas quais a quantidade mínima de máquinas necessárias é considerável.

Adicionalmente, observa-se um cenário recorrente no qual organizações, por razões econômicas ou arquiteturais, optam por não expandir a quantidade de máquinas provisionadas no cluster, mas ainda assim desejam reduzir seus custos com a infraestrutura em nuvem sem comprometer o desempenho computacional nos momentos de maior demanda. Nessas circunstâncias, faz-se necessária uma abordagem capaz de otimizar o custo das instâncias em períodos de ociosidade e, simultaneamente, responder a aumentos de carga sem alterar a quantidade total de máquinas em execução. É diante dessa problemática que o presente trabalho se insere, propondo um mecanismo que explora a heterogeneidade das instâncias ofertadas pelos provedores de nuvem como alternativa à expansão e retração tradicionais do parque computacional.

1.1 Objetivos

O presente trabalho tem por objetivo propor o CANM, *Cost Aware Node Migrator*, um mecanismo capaz de otimizar os custos de provisionamento de máquinas em clusters *Kubernetes* executados em infraestruturas de computação em nuvem. O mecanismo é responsável por identificar cenários de subutilização ou sobreutilização de recursos nos nós do cluster e realizar migrações para instâncias mais econômicas ou com maior capacidade computacional. Dessa forma, busca-se reduzir os custos operacionais em períodos de ociosidade e, simultaneamente, responder a aumentos na demanda computacional sem alterar a quantidade total de máquinas em execução no cluster.

Adicionalmente, este trabalho avalia o desempenho do mecanismo proposto em uma infraestrutura real sujeita a variações dinâmicas na utilização de recursos. Para isso, são analisados diferentes cenários de carga, permitindo investigar o comportamento do CANM tanto em situações de elevada demanda computacional quanto em períodos de ociosidade dos recursos.

1.2 Estrutura do texto

O presente trabalho está organizado da seguinte maneira: no Capítulo 2, são apresentados os conceitos teóricos fundamentais para a compreensão deste estudo, iniciando com uma discussão sobre computação em nuvem, seguida de uma introdução a máquinas virtuais e ambientes containerizados, e finalizando com uma descrição do *Kubernetes* e de seus mecanismos nativos. No Capítulo 3, são discutidos os trabalhos relacionados a esta pesquisa. No Capítulo 4, o mecanismo proposto é apresentado em profundidade, partindo de uma visão geral da solução e evoluindo para o detalhamento completo do funcionamento do CANM. No Capítulo 5, é descrita a metodologia empregada para avaliar o desempenho do mecanismo em uma infraestrutura real de computação em nuvem. No Capítulo 6, são apresentados e analisados os resultados obtidos durante a avaliação experimental. Por fim, no Capítulo 7, são expostas as conclusões deste trabalho, bem como as oportunidades de pesquisa e os trabalhos futuros decorrentes do estudo realizado.

2 REVISÃO BIBLIOGRÁFICA

Neste capítulo, apresentaremos os conceitos teóricos fundamentais para o entendimento do trabalho proposto. Nele, abordaremos noções básicas acerca da computação em nuvem, os benefícios que esse modelo proporciona para as aplicações que a adotam e como esses serviços são disponibilizados aos usuários. Além disso, apresentaremos os fundamentos sobre máquinas virtuais e discutiremos o papel do hipervisor na virtualização desses recursos. Adicionalmente, será descrito o que são ambientes containerizados, como eles auxiliam arquiteturas baseadas em microserviços, introduziremos ferramentas que possibilitam de forma simplificada a implementação da containerização em aplicações e, por fim, descreveremos o *Kubernetes*.

2.1 Computação na Nuvem

Em razão do aumento do desenvolvimento de sistemas distribuídos e aplicações de grande escala, a demanda por recursos computacionais cresceu significativamente. Com o objetivo de escalar essas aplicações e melhorar a qualidade do serviço ofertado, organizações tecnológicas passaram a demandar expansão de servidores físicos para agregar sua infraestrutura computacional local. Desse modo, a complexidade de gerenciar o crescimento da capacidade aumentava proporcionalmente com o parque computacional da instituição. Além disso, a natureza dinâmica das cargas de trabalho no ambiente digital, como a redução de acessos em um site de varejo durante a madrugada e o aumento em um período sazonal como o Natal, resultava em um cenário de subutilização de recursos em determinados momentos e alta utilização em outros, exigindo alta capacidade de provisionamento variável (ARMBRUST et al., 2010).

Diante desse cenário, a computação em nuvem surge para contornar tais desafios. Esse modelo de computação é caracterizado pela disponibilização e utilização sob demanda de recursos computacionais na forma de serviços através da internet, como armazenamento, processamento e outros recursos de infraestrutura¹. Diferentemente da computação *on-premises*, na qual a demanda computacional é executada em máquinas físicas pertencentes à própria entidade, a computação em nuvem possibilita que cargas de trabalho sejam executadas em infraestruturas remotas mantidas por terceiros. Ainda, esse modelo permite

¹Definição adaptada de SUSNJARA, S.; SMALLEY, I. *O que é computação em nuvem?* IBM Think. Disponível em: <<https://www.ibm.com/br-pt/think/topics/cloud-computing>>. Acesso em: 18 jun. 2026.

maior flexibilidade no gerenciamento do recurso computacional, visto que o usuário pode acessar serviços em nuvem a partir de qualquer localidade contanto que possua uma conexão com a internet. Por fim, a computação em nuvem viabiliza a escalabilidade da capacidade computacional da organização tecnológica, uma vez que dispensa a necessidade de expansão da infraestrutura física própria.

2.1.1 Provedores de Serviço em Nuvem

Nesse contexto de limitação de escalabilidade de servidores físicos locais e expansão da computação em nuvem, emergem os provedores de serviço em nuvem como alternativa para o provisionamento sob demanda de recursos computacionais. Esses provedores oferecem um conjunto de soluções tecnológicas disponibilizadas via internet, eliminando a necessidade de manutenção de uma infraestrutura *on-premises*. Dessa forma, ao invés da organização administrar um datacenter em uma infraestrutura própria, ela delega a responsabilidade de gestão do hardware físico para um provedor de nuvem e passa a utilizar remotamente recursos virtualmente alocados. Assim, torna-se possível otimizar o provisionamento da capacidade computacional com base na demanda da carga de trabalho, ao mesmo tempo que reduz a sobrecarga de gerenciamento dos recursos.

Ainda, provedores de nuvem pública, como a Google Cloud Platform (GCP), Amazon Web Services (AWS) e Microsoft Azure, oferecem um modelo de cobrança *pay-as-you-go*. Nesse modelo, o usuário paga apenas pelo uso efetivo dos recursos computacionais consumidos na nuvem. Esse fato contribui com a melhoria da flexibilidade de gestão dos recursos alocados, visto que possibilita reduzir a capacidade computacional sem a necessidade do pagamento contínuo de recursos previamente adquiridos. Por conta disso, a computação em nuvem contrapõe-se à maneira tradicional de computação em servidores locais ao possibilitar maior elasticidade dos componentes computacionais, uma vez que, em um cenário de infraestrutura *on-premises*, os recursos físicos permanecem gerando custos operacionais independentemente do nível de utilização.

Por fim, os provedores de serviço em nuvem viabilizam a alocação de recursos em diferentes regiões geográficas. Nesse âmbito, organizações que desejam ampliar a disponibilidade da sua infraestrutura, ao expandir a presença de seus serviços para áreas de interesse estratégico, beneficiam-se significativamente dos serviços oferecidos por esses provedores. Desse modo, a adoção de uma infraestrutura em nuvem proporciona diversos benefícios, como a elasticidade de recursos sob demanda, a melhoria na disponibilidade, a

redução de custos por meio do modelo *pay-as-you-go* e a diminuição da complexidade de gerenciamento do hardware físico.

2.1.2 Modelos de serviço de computação em nuvem

No contexto da computação em nuvem, os recursos computacionais podem ser disponibilizados por meio de diferentes modelos de serviço, como os modelos de *Infrastructure as a Service* (IaaS), *Platform as a Service* (PaaS) ou *Software as a Service* (SaaS) (MELL; GRANCE, 2011). Cada modelo tem características e casos de uso distintos, diferenciando-se com base no recurso computacional oferecido e nível de abstração. No modelo IaaS, a organização tecnológica possui um controle maior no gerenciamento dos recursos de interesse, apresentando participação ativa na escolha das propriedades desses componentes. Nesse cenário, o usuário possui mais flexibilidade, sendo possível especificar as características do hardware de interesse, como tipo do processador e disco. Além disso, nesse modelo o usuário tem autonomia para administrar aspectos de rede, como comunicação entre máquinas adquiridas e regras de firewall. O serviço *Compute Engine*, disponibilizado pelo provedor Google Cloud Platform, é um exemplo de IaaS, haja vista que permite criar e executar cargas de trabalho em máquinas virtuais.

Em contrapartida, no modelo PaaS, a flexibilidade do gerenciamento de recursos é reduzida, uma vez que parte significativa dessa responsabilidade fica a cargo do provedor de nuvem. Esse modelo foca-se em facilitar a atividade de desenvolvimento de software ao abstrair a sobrecarga administrativa da gestão da infraestrutura. Nesse contexto, o serviço *Heroku* (Heroku, 2026) é um exemplo, tendo em vista que possibilita a execução de uma aplicação sem a necessidade do usuário configurar um servidor. Por fim, o modelo de serviço SaaS concentra-se em oferecer soluções prontas para o usuário final, sem a necessidade do usuário manejar qualquer infraestrutura que sustenta a aplicação. Nesse modelo, podemos citar o *Google Drive*, como exemplo.

2.2 Máquinas virtuais

Uma máquina virtual (*Virtual Machine*) é um ambiente virtual isolado que emula um computador físico completo, tendo seus recursos de hardware concedidos por um software especializado implantado em uma máquina física (Microsoft, 2026). A utilização

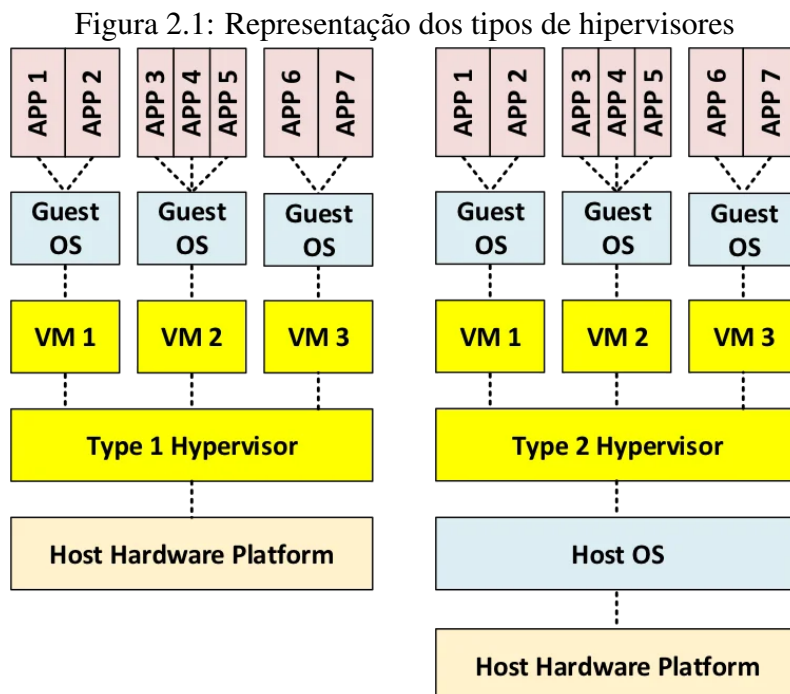
de VMs possibilita que diferentes sistemas operacionais convidados executem lado a lado em um único servidor físico hospedeiro, também denominado host, que pode ou não possuir um sistema operacional próprio. Ainda, VMs compartilhando recursos em um mesmo hospedeiro, como CPU, memória e armazenamento, executam de maneira independente graças a um software de gerenciamento de máquinas virtuais denominado *hipervisor*.

Esse software é o agente responsável por permitir que VMs executem de forma isolada em um mesmo servidor físico. Implementado diretamente no hardware ou no sistema operacional do hospedeiro, ele separa logicamente as VMs umas das outras, virtualiza os recursos de computação física e administra o fornecimento desses recursos para as máquinas convidadas. Essa virtualização beneficia significativamente o modelo de computação em nuvem, uma vez que otimiza a utilização do recurso ao permitir que diferentes usuários desfrutem de ambientes isolados na mesma plataforma física. Ademais, o hipervisor permite criar, implementar e desativar máquinas virtuais de forma rápida e sob demanda, possibilitando que as organizações respondam rapidamente às variações nas cargas de trabalho.

Os hipervisores podem ser classificados em dois tipos. Um hipervisor do tipo 1 executa diretamente no hardware do hospedeiro, sem a necessidade de um sistema operacional para intermediar o acesso aos recursos computacionais. Por conta disso, tendem a apresentar melhor eficiência e segurança, visto que não há uma camada intermediária entre eles e o recurso físico. Em contraponto, um hipervisor do tipo 2 não executa diretamente na plataforma física, mas sim como uma aplicação em um sistema operacional. Por conta de necessitar de um sistema operacional para intermediação com o componente físico, esse tipo de implementação pode introduzir maior latência em suas operações – impactando o desempenho das aplicações.

Em virtude dos hipervisores e das técnicas de virtualização, as máquinas virtuais têm impacto significativo na implementação do modelo de infraestrutura como serviço. Graças a essas tecnologias, provedores de nuvem são capazes de otimizar a utilização de seus recursos de hardware ao possibilitarem que múltiplos clientes sejam atendidos pela mesma infraestrutura física. Ainda, esses provedores conseguem realocar de forma eficiente as VMs entre computadores físicos devido à portabilidade oferecida pela virtualização. Nesse sentido, a adoção de máquinas virtuais permite que usuários obtenham maior capacidade computacional com base em sua demanda, contribuindo com a escalabilidade da infraestrutura organizacional. Por fim, é importante destacar que determinados provedores

de nuvem permitem a alocação integral de máquinas físicas, sendo esse cenário mais comum em aplicações de alto desempenho. A Figura 2.1 exemplifica em alto nível a arquitetura dos dois tipos de hipervisores de forma a representar as suas diferenças.



Fonte: FIRESMITH, Donald. Virtualization via Virtual Machines. 2017. Disponível em: <<https://www.sei.cmu.edu/blog/virtualization-via-virtual-machines/>>. Acesso em: 29 mar. 2026.

Tabela 2.1: Exemplos de instâncias de máquinas virtuais na Google Cloud Platform

Instância	Família	Plataforma	Tipo	vCPU	Memória (GB)
n2-standard-4	N2	Intel	Standard	4	16
n2-highcpu-8	N2	Intel	High-CPU	8	8
c4d-standard-4	C4D	AMD	Standard	4	16
c4d-highcpu-8	C4D	AMD	High-CPU	8	8
e2-standard-4	E2	Intel/AMD	Standard	4	16
e2-highcpu-8	E2	Intel/AMD	High-CPU	8	8

Fonte: Adaptado de Google Cloud (2026).

A Tabela 2.1 exemplifica um subconjunto de instâncias de máquinas virtuais com recursos pré-definidos disponíveis na GCP. As famílias de máquinas correspondem a conjuntos de configurações de processamento e memória projetadas para atender diferentes tipos de cargas de trabalho, como aplicações de uso geral, com aceleradores ou com maior demanda de memória (Google Cloud, 2026). Ainda, as famílias de máquinas são subdivididas por série com base na geração e tipo de processador. Instâncias pré-definidas não permitem a configuração individual de memória e CPU, porém, cabe destacar que é

possível criar uma instância de VM com capacidade computacional personalizada para as famílias de máquinas N e E. Devido às diferenças de plataforma entre as famílias, instâncias de VMs com a mesma quantidade de memória e CPU podem apresentar custo por uso distinto. A instância *e2-standard-4*, por exemplo, possui um custo por uso menor que a instância *c4d-standard-4*, visto que a família E é projetada para uso geral a um custo menor e a família C para desempenho consistentemente alto.

2.3 Ambientes Containerizados

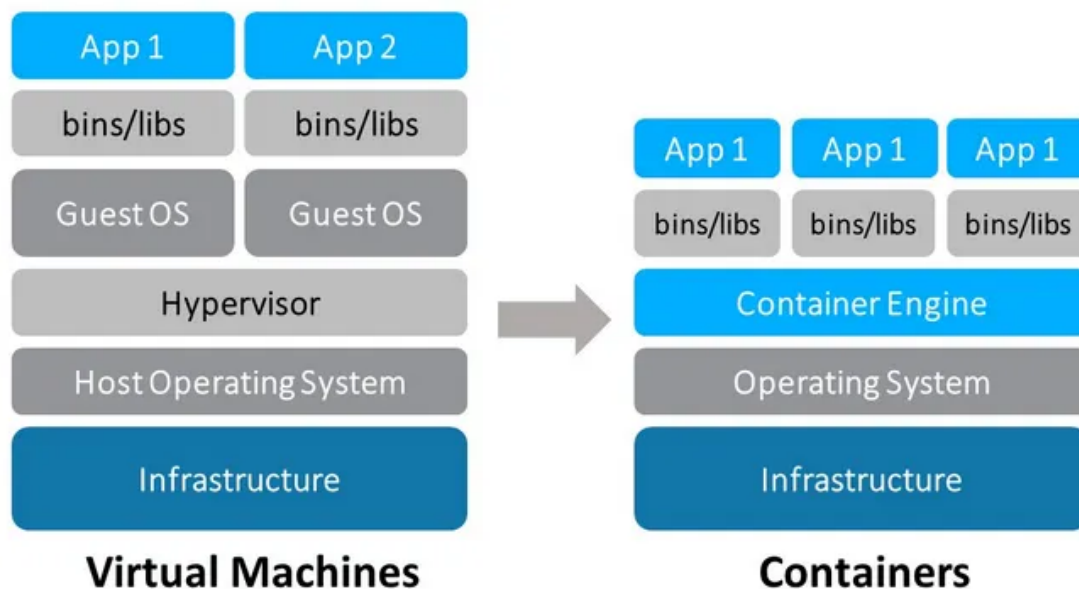
De forma semelhante às máquinas virtuais discutidas na Seção 2.2, ambientes containerizados oferecem isolamento lógico para aplicações, permitindo a abstração do ambiente em que estão efetivamente executando. Constituídos por contêineres, esses ambientes realizam o isolamento de processos a nível de sistema operacional, diferente das VMs que virtualizam recursos e isolam a nível de hardware. Os contêineres podem ser compreendidos como pacotes leves que encapsulam o código e as dependências de uma aplicação. Por compartilharem o kernel do sistema operacional hospedeiro, eles apresentam alta portabilidade em ambientes compatíveis, sendo capazes de iniciar mais rápido uma vez que não necessitam de um sistema operacional interno.

Em virtude dos benefícios supracitados, a adesão de ambientes containerizados expandiu-se de maneira significativa, especialmente no contexto de arquiteturas baseadas em microsserviços. Nessas arquiteturas, a aplicação é implementada como um conjunto de serviços pequenos, independentes e fracamente acoplados, os quais se comunicam através de um determinado protocolo, como o *Hypertext Transfer Protocol* (HTTP). Dessa forma, cada serviço é visto como uma entidade lógica separada com características e responsabilidades específicas, variando em fatores como linguagem de programação, dependências de código e necessidade computacional.

Por conta disso, a adoção de contêineres eleva a eficiência das arquiteturas baseadas em microsserviços ao possibilitar o isolamento das aplicações e suas dependências. Em um cenário em que essa arquitetura é empregada sem a aplicação da containerização, as dependências de código são instaladas no sistema de arquivos do hospedeiro de forma não isolada. Assim, é possível que ocorra uma interferência das dependências de um serviço em outro. Nesse sentido, a aplicação de ambientes containerizados contribui para a minimização dessa problemática, uma vez que permite o isolamento lógico no sistema de arquivos, evitando a utilização de dependências de software instaladas globalmente

no hospedeiro. A Figura 2.2 exemplifica em alto nível as diferenças de arquitetura entre máquinas virtuais e contêineres.

Figura 2.2: Diferença de arquitetura entre VMs e ambientes containerizados



Fonte: DUARTE, Igor. Arquitetura Docker. Medium, 2020. Disponível em: <<https://medium.com/@IgorDuarte17/arquitetura-docker-c76cb14ffac6>>. Acesso em: 04 abr. 2026.

Para viabilizar a utilização prática desses ambientes, surgiram ferramentas especializadas na containerização de aplicações, sendo o Docker uma das mais difundidas. O Docker é uma ferramenta de código aberto responsável por criar e gerenciar o processo de containerização em um determinado hospedeiro, empregando funcionalidades do sistema operacional para criar o isolamento entre processos de maneira fácil e rápida (Docker, 2026). Essa tecnologia utiliza um método de implantação baseado em imagens imutáveis, em que se definem as bibliotecas, ferramentas e dependências que a aplicação precisa para ser executada como contêiner. Assim, os contêineres do Docker são caracterizados como instâncias ativas e em execução dessas imagens previamente definidas.

Em virtude da imutabilidade das imagens, o Docker contribui diretamente com a reprodutibilidade e padronização de ambientes de execução, uma vez que garante que a aplicação seja executada sob as mesmas condições, independentemente do ambiente hospedeiro. Dessa forma, reduzem-se significativamente problemas relacionados à inconsistência de dependências entre diferentes ambientes, como desenvolvimento, teste e produção.

Entretanto, é importante ressaltar que a tecnologia de containerização não é restrita ao Docker, mas teve sua adoção amplamente impulsionada por esse software. A ferramenta Podman, por exemplo, também facilita a containerização de aplicações, com maior ênfase

na segurança do ambiente hospedeiro. Ainda, apesar dos benefícios supracitados, o Docker, por si só, não provê mecanismos nativos para orquestração e escalonamento de contêineres. Nesse sentido, em um sistema distribuído com diversas aplicações containerizadas executando, torna-se responsabilidade do usuário a gestão das máquinas utilizadas, a distribuição da carga de trabalho e a expansão da quantidade de instâncias ativas. Diante desse cenário, torna-se necessária a utilização de ferramentas capazes de automatizar o gerenciamento e a orquestração de contêineres em larga escala.

2.4 Kubernetes

O emprego de ambientes containerizados oferece diversos benefícios para a melhoria de um sistema. Como citado na Seção 2.3, o Docker simplifica o processo de criação e execução de contêineres, permitindo que aplicações usufruam desses ambientes de maneira fácil – sem necessitar que o usuário realize configurações complexas de infraestrutura. Porém, em sistemas distribuídos de larga escala – nos quais diversos contêineres estão em execução em múltiplas máquinas – a gestão desses ambientes torna-se um desafio, uma vez que ferramentas como o Docker não disponibilizam mecanismos nativos de orquestração e gerenciamento distribuído de contêineres. Nesse sentido, em uma situação em que uma máquina tem sua execução comprometida, o processo de realocação dos contêineres deve ser realizado manualmente.

Diante desse cenário de limitação, o *Kubernetes* (K8s) apresenta-se como uma ferramenta capaz de abordar essas problemáticas ao realizar o gerenciamento automático de contêineres em ambientes distribuídos. Desenvolvido por engenheiros do Google, o K8s é uma plataforma de código aberto responsável por orquestrar cargas de trabalho e serviços containerizados. Constituído por um cluster com um único ou múltiplos nós, o K8s permite que sistemas distribuídos sejam executados de forma resiliente, oferecendo funcionalidades como descoberta de serviços, balanceamento de carga, suporte a implantação de novas versões e reversão para versões anteriores, otimização de uso das máquinas do cluster, controle do ciclo de vida de contêineres, entre outras.

2.4.1 Objetos do Kubernetes

No contexto do *Kubernetes*, os objetos são entidades persistentes que representam o estado desejado do cluster. Esses objetos descrevem aspectos como as aplicações containerizadas em execução, os recursos disponíveis e as configurações necessárias, permitindo que o sistema atue para manter o estado real conforme o especificado. Com base na necessidade do usuário, o *Kubernetes* permite a criação de diversos objetos com propriedades e responsabilidades distintas, tais como *Pod*, *Deployment*, *Service*, *DaemonSet*, *Jobs*, *ReplicaSet*, entre outros. Devido ao escopo do trabalho proposto, abrangearemos apenas as definições e características dos *Pods*, *ReplicaSets*, *Deployments* e *Services*.

Nesse sentido, o *Pod* é o menor objeto de computação implantável no *Kubernetes*, representando um ou mais contêineres executando no cluster. Esses contêineres compartilham recursos de armazenamento e rede, sendo sempre co-allocados e co-escalados no mesmo contexto compartilhado de execução. Um *Pod* não é um processo, mas um ambiente para a execução de contêineres. Ainda, é importante destacar que, por mais que seja possível executar múltiplos contêineres no mesmo *Pod*, o uso mais comum no K8s é a execução de um único contêiner por *Pod* (Kubernetes, 2026). Por fim, os *Pods* possuem um ciclo de vida gerenciado pelo *Kubernetes*, podendo assumir diferentes estados ao longo de sua execução, como *Pending*, *Running*, *Succeeded* e *Failed*.

De maneira complementar, o *ReplicaSet* é o objeto do K8s responsável por manter um conjunto estável de *Pods* executando constantemente – sendo utilizado para garantir a disponibilidade de uma quantidade específica de *Pods* idênticos. Por exemplo, um *ReplicaSet* detecta quando a quantidade de *Pods* em um grupo observável difere da quantidade desejada, atuando de forma a reconciliar o estado atual com o estado desejado. Ainda, por mais que o *Kubernetes* permita que o usuário crie e administre *ReplicaSets*, a prática mais comum e recomendada é utilizar *Deployments* para o gerenciamento desse objeto.

O *Deployment* atua no gerenciamento de um conjunto de *Pods* utilizando um *ReplicaSet* para garantir a quantidade de *Pods* sendo executados. Por conta disso, os *Deployments* são abstrações de alto nível dos *ReplicaSets*, controlando e expandindo o seu funcionamento ao oferecer soluções para desafios que os *ReplicaSets*, por si só, não resolvem – como atualizações progressivas (*rolling updates*), controle de versões e *rollback*. Dessa forma, simplifica-se o gerenciamento declarativo das aplicações e aumenta-se a

confiabilidade durante o ciclo de vida dos contêineres.

Por fim, um *Service* no *Kubernetes* é um objeto que define uma abstração de rede lógica sobre um conjunto de *Pods*, permitindo sua descoberta e acesso por meio de um endereço estável e de mecanismos de balanceamento de carga, independentemente da natureza efêmera dos *Pods*.

2.4.2 Agendamento de Nós e Anotações

Além dos objetos discutidos anteriormente, é necessário introduzir o conceito de Nó, que representa uma máquina de trabalho do cluster — física ou virtual — na qual os *Pods* são efetivamente executados. Cada nó é gerenciado pelo plano de controle do *Kubernetes*, que decide, por meio do agendador (*scheduler*), em qual nó cada *Pod* deve ser alocado. O *Kubernetes* oferece mecanismos para influenciar diretamente esse agendamento, bem como para associar metadados arbitrários aos seus objetos, recursos relevantes para o presente trabalho.

Cordon e Uncordon

A operação de *cordon* marca um nó como não-agendável (*unschedulable*). A partir desse momento, o agendador deixa de alocar novos *Pods* naquele nó. Entretanto, os *Pods* que já estavam em execução não são afetados, permanecendo ativos até que sejam removidos por outro meio. Portanto, o processo de *cordon* é uma forma de isolar um nó sem interromper imediatamente as cargas de trabalho que ele hospeda.

Já a operação de *uncordon* restaura a condição de agendável do nó, permitindo novamente que o agendador aloque *Pods* nele. Esse par de operações é comumente empregado em cenários de manutenção, nos quais se deseja primeiro impedir a chegada de novas cargas a um nó (*cordon*), para então esvaziá-lo de forma controlada, e, em caso de cancelamento da operação, reverter o isolamento (*uncordon*).

Drain

A operação de *drain* consiste em esvaziar um nó de seus *Pods* de maneira controlada, sendo tipicamente empregada como preparação para a manutenção ou a remoção daquele nó. Essa operação combina dois passos: primeiro, o nó é marcado como não-agendável (*cordon*), de modo que nenhum novo *Pod* seja nele alocado. Em seguida, os *Pods* em

execução são removidos por meio de despejo (*eviction*).

Diferentemente de uma exclusão direta, o despejo realizado pelo *drain* é gracioso: ele é intermediado pela *Eviction API* do *Kubernetes*, que respeita os *PodDisruptionBudgets* (PDBs) definidos para as aplicações. Um PDB especifica a disponibilidade mínima que um conjunto de *Pods* deve manter, impedindo que um número excessivo de réplicas seja removido simultaneamente e, assim, preservando a continuidade do serviço durante a operação.

Annotations

As *annotations* (anotações) são pares de chave-valor que podem ser associados a qualquer objeto do *Kubernetes* como metadata. Diferentemente das *labels* — que se destinam à identificação e seleção de objetos por meio de seletores —, as *annotations* armazenam metadados não identificadores, não sendo utilizadas para selecionar ou agrupar recursos.

Essa característica torna as anotações adequadas para registrar informações arbitrárias e, eventualmente, estruturadas, frequentemente consumidas por ferramentas, bibliotecas e controladores externos. Por estarem persistidas no próprio objeto, elas permitem que tais ferramentas armazenem e recuperem estado associado a um recurso de maneira durável, acompanhando o ciclo de vida do objeto ao qual pertencem.

2.4.3 Arquitetura do Cluster

A arquitetura de um cluster no *Kubernetes* é dividida em dois grupos principais, sendo eles o plano de controle e o plano de dados. O plano de controle é responsável por gerenciar o estado geral do cluster, sendo constituído pelos componentes *kube-apiserver*, *etcd*, *kube-scheduler*, *kube-controller-manager* e, opcionalmente, *cloud-controller-manager*. Cada um desses componentes desempenha um papel específico na manutenção e operação do cluster. Nesse contexto, o *kube-apiserver* atua como ponto central de comunicação, sendo o componente principal do plano de controle. Ele disponibiliza uma API para o gerenciamento do cluster – sendo o ponto único de comunicação entre o cluster e o usuário. Já o *etcd* é um armazenamento do tipo chave-valor utilizado como suporte para todos os dados do cluster – servindo como fonte única de verdade para a tomada de decisões. Por outro lado, o *kube-scheduler* é o componente responsável por observar os *Pods*

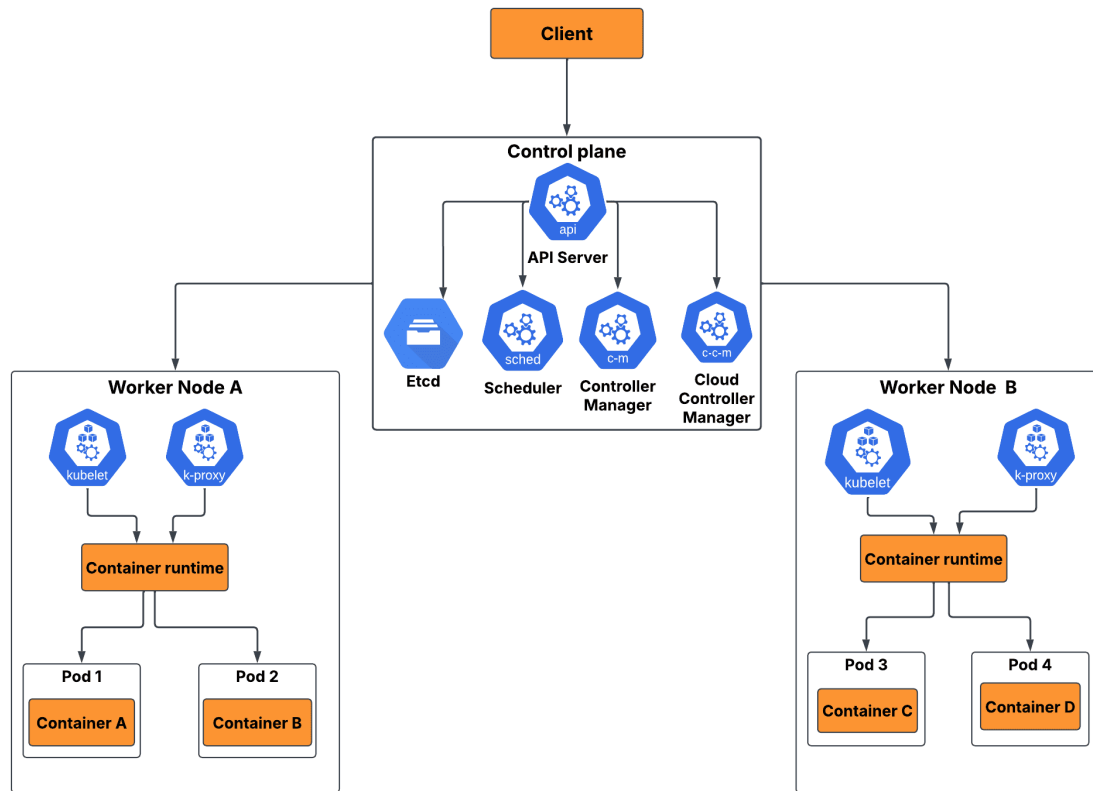
recém-criados que ainda não possuem um nó atribuído e selecionar um nó adequado para a execução desses *Pods*. Adicionalmente, o *kube-controller-manager* monitora o estado atual do cluster e reconcilia com o estado desejado, assim, caso algum objeto do cluster esteja em um estado inadequado, esse componente atua na sua autocorreção. Por fim, o *cloud-controller-manager* pode ser opcionalmente utilizado para gerenciar a integração do cluster do K8s com a infraestrutura de um provedor de nuvem.

Por outro lado, no plano de dados, os componentes *kubelet*, runtime de contêineres e, opcionalmente, o *kube-proxy* são responsáveis pela execução e gerenciamento dos contêineres – estando presentes em todos os nós do cluster. O *kubelet* é o agente responsável por garantir que os contêineres definidos em um *Pod* estejam em execução, de forma a assegurar a integridade da aplicação. Além disso, esse componente informa constantemente para o *kube-apiserver* o estado do nó e dos objetos em execução. Já o runtime de contêiner é o mecanismo fundamental para que o *Kubernetes* consiga executar contêineres eficientemente, sendo incumbido de gerenciar o ciclo de vida desses elementos. Finalmente, o *kube-proxy* gerencia regras de rede e encaminhamento de tráfego entre os serviços e *Pods*. A Figura 2.3 exemplifica visualmente a arquitetura de um cluster do *Kubernetes* com a separação do plano de controle e do plano de dados.

2.4.4 Autoescalonamento

O autoescalonamento é um mecanismo que possibilita o ajuste automático da capacidade de provisionamento, a fim de garantir maior elasticidade de recursos de acordo com a demanda da carga de trabalho. Esse mecanismo é capaz de adequar os recursos computacionais da infraestrutura que o emprega, aumentando a quantidade de máquinas em períodos de maior demanda e reduzindo esses componentes em situações nas quais a utilização desses recursos diminui.

Nesse sentido, os autoescalonadores podem ser divididos em dois grupos: reativos e preditivos. Os autoescalonadores reativos utilizam métricas do sistema para a tomada de decisão, como, por exemplo, a utilização de CPU ou a latência da aplicação. Dessa forma, caso o consumo de processamento ultrapasse um limite pré-definido, esse método executará uma ação, podendo resultar no aumento ou na redução de recursos. Em contrapartida, autoescalonadores preditivos fazem uso de dados históricos e atuais para prever a necessidade de modificação no provisionamento de recursos, antecipando-se a variações futuras na carga de trabalho. Nesse contexto, diversas técnicas de Inteligência

Figura 2.3: Arquitetura de um cluster *Kubernetes*

Fonte: MMOPELWA, Boemo Wame. *Kubernetes Architecture*. Cherry Servers, 2025. Disponível em: <<https://www.cherryservers.com/blog/kubernetes-architecture>>. Acesso em: 30 mar. 2026.

Artificial, como o aprendizado por reforço, podem ser empregadas para a implementação dessa categoria de autoescalador. Dessa forma, os autoescaladores contribuem para a melhoria da qualidade do serviço ofertado e para a redução de custos, ao garantir a elasticidade dos recursos computacionais com base na demanda.

No contexto de orquestração de contêineres, esses mecanismos tornam-se fundamentais para garantir a eficiência operacional e a escalabilidade das aplicações. O *Kubernetes* disponibiliza um conjunto de autoescaladores reativos capazes de ajustar os recursos do cluster em resposta a variações na demanda. Nativamente, são disponibilizados dois tipos de autoescaladores que diferem em relação à direção do escalonamento: o escalonamento horizontal (*Horizontal Pod Autoscaler - HPA*), responsável por ajustar a quantidade de instâncias (*Pods*) em execução, e o escalonamento vertical (*Vertical Pod Autoscaler - VPA*), que atua na modificação dos recursos computacionais atribuídos a cada instância, como CPU e memória.

Além disso, o *Cluster Autoscaler* é um componente do ecossistema *Kubernetes*, amplamente suportado pelos principais provedores de nuvem, responsável por ajustar dinamicamente a quantidade de nós no cluster. Ele adiciona novos nós quando existem

Pods em estado Pending por insuficiência de recursos, e remove nós ociosos para reduzir custos.

3 TRABALHOS RELACIONADOS

Como citado na Subseção 2.4.4, os autoescalonadores de clusters são utilizados para ajustar dinamicamente a quantidade de nós em um cluster com o objetivo de evitar o subprovisionamento ou superprovisionamento dos recursos computacionais. Esse mecanismo permite que a infraestrutura se adeque à carga de trabalho, garantindo desempenho estável das aplicações em cenários de aumento na demanda e redução de custos em situações de diminuição da utilização dos recursos. Por esses motivos, diversos trabalhos concentram-se no aperfeiçoamento de técnicas de autoescalonamento de clusters, visando otimizar o funcionamento desse mecanismo.

Neste capítulo, apresentaremos uma revisão geral dos trabalhos que propõem otimizações de performance e custo no gerenciamento de aplicações contêinerizadas executando tanto em clusters totalmente virtuais quanto em clusters híbridos. Além disso, abrangearemos técnicas do estado da arte utilizadas para o provisionamento de recursos em infraestruturas na nuvem baseadas em *Kubernetes*. Somado a isso, discutiremos acerca de um estudo que se propõe a formalizar uma metodologia científica para a avaliação de desempenho e custo de autoescalonadores. Por fim, é importante destacar que, embora o presente trabalho não implemente um autoescalonador, as principais contribuições na área de otimização de custos em clusters *Kubernetes* estão relacionadas a esses mecanismos, os quais possuem relação direta com os objetivos que nossa pesquisa busca endereçar.

3.1 Otimização de Desempenho e Custo no Escalonamento de Clusters

Em (WANG; ZHANG; WU, 2020), os autores apresentam um autoescalonador de cluster (CA) que emprega uma arquitetura baseada na classificação dos nós em três categorias: Nós Normais, Nós de Reserva e Nós Virtuais. Os Nós Normais compõem a infraestrutura padrão do cluster e permanecem continuamente em execução, participando ativamente do processamento da carga de trabalho. Em contrapartida, os Nós de Reserva funcionam como um repositório de recursos, não estando constantemente ativos no cluster principal, mas disponíveis em um *pool* compartilhado, sendo adicionados automaticamente sob demanda quando há necessidade de aumento de capacidade.

Por fim, os Nós Virtuais são utilizados para lidar com picos de demanda inesperados. Esses nós são provisionados por meio de chamadas a APIs de provedores de nuvem, permitindo a transferência da carga de trabalho para a nuvem quando os recursos locais e

de reserva não são suficientes. Além disso, o estudo emprega uma abordagem híbrida que combina mecanismos preditivos e reativos para a tomada de decisões no escalonamento. A avaliação experimental demonstrou que, ao utilizar essa arquitetura hierárquica e preditiva, o sistema apresentou maior velocidade de escalonamento em comparação ao autoescalador padrão do *Kubernetes*, além de reduzir a instabilidade do cluster em cenários de carga periódica, mitigando oscilações excessivas entre expansão e contração de recursos.

Entretanto, como o foco principal do trabalho está na melhoria do desempenho e da estabilidade do escalonamento, a discussão sobre otimização de custos mostra-se limitada. Esse aspecto torna-se ainda mais relevante considerando que os Nós Virtuais, por serem provisionados em provedores de nuvem, podem impactar negativamente os custos monetários, especialmente quando não há consideração sobre as características da instância e seus respectivos custos associados.

Por fim, destaca-se que a arquitetura proposta não é executada integralmente em ambiente de nuvem, operando de forma híbrida entre infraestrutura local e provedores externos. Como consequência, sua reprodutibilidade depende da disponibilidade de recursos físicos locais, o que pode limitar sua adoção por organizações ou usuários que não dispõem dessa infraestrutura. Essas limitações evidenciam a necessidade de abordagens que considerem, de forma integrada, aspectos de desempenho, custo e viabilidade de implantação.

Ainda, levando em consideração a otimização de custos, o estudo (ZHONG; BUYA, 2020) propõe uma estratégia de orquestração de contêineres com foco no custo-benefício em infraestruturas de computação em nuvem baseadas em *Kubernetes*. Com a motivação de otimizar o processo de gerenciamento de recursos com o menor custo possível, os autores implementaram sobre a plataforma padrão do *Kubernetes* um componente de autoescalamento customizado denominado *HTAS (Heterogeneous Task Allocation Strategy)*. Esse mecanismo classifica as cargas de trabalho em dois grupos – longa duração, como servidores e bancos de dados, e curta duração, como tarefas de processamento em lote. Tal classificação é utilizada durante o processo de escalonamento de forma a agrupar tarefas de curta duração que possuem tempo de execução similar no mesmo nó e, para serviços de longa duração, escalonar minimizando a quantidade de recursos ociosos no nó.

Além disso, o *HTAS* emprega um sistema de autoescalamento heterogêneo capaz de selecionar uma combinação de instâncias de máquinas virtuais com melhor custo-eficiência para suprir a demanda atual. Dessa forma, a estratégia proposta diferencia-se de mecanismos de autoescalamento que adicionam apenas uma instância homogênea

por ciclo, limitando a elasticidade diante de variações bruscas de carga. Por fim, os pesquisadores implementam um processo de reescalonamento de *Pods* através da migração da carga de trabalho para outros nós, a fim de reduzir a quantidade de máquinas no cluster e, por conseguinte, reduzir o custo operacional.

Contudo, o estudo limitou a heterogeneidade das VMs apenas na diferenciação da quantidade de recursos – como CPU e memória –, sem considerar os tipos de máquinas na otimização do custo, como instâncias otimizadas para computação ou para memória. Ainda, o artigo utilizou exclusivamente carga de trabalho sintética nos experimentos, sendo que as tarefas de processamento em lote foram implementadas como simples comandos *sleep*, uma simplificação acentuada em relação a cargas reais. Esse fato implica que nuances complexas de carga de trabalho real – como padrões de consumo de recursos ao longo do tempo – foram excluídas da experimentação.

Ademais, ainda no contexto de otimização de custo na orquestração de contêineres, o trabalho (CHUNG; PARK; GANGER, 2018) propõe um escalonador de clusters especializado na orquestração de tarefas em lote em clusters virtuais na nuvem pública denominado *STRATUS*. Com o objetivo de redução do custo monetário na execução das cargas de trabalho, a ferramenta proposta utiliza mecanismos de empacotamento consciente de tempo de execução, seleção conjunta de empacotamento e tipo de instância e limpeza de instâncias subutilizadas. De forma semelhante ao (ZHONG; BUYYA, 2020), o *STRATUS* agrupa tarefas com tempo de execução similares no mesmo nó, de modo que todas finalizem aproximadamente no mesmo tempo. Assim, sendo possível desligar o nó após a finalização de todas as tarefas – minimizando a ociosidade do recurso computacional da máquina. Além disso, a fim de otimizar o custo ao reduzir a quantidade de nós utilizados no cluster, o *STRATUS* também apresenta um mecanismo de migração de tarefas entre máquinas.

No quesito de escalonamento do cluster, o *STRATUS* considera instâncias de diferentes tamanhos na decisão de aquisição junto ao provedor de nuvem, levando em conta a variação dinâmica de preços no momento da decisão. Porém, analogamente nas limitações apontadas no *HTAS*, o processo de escalonamento do *STRATUS* não considera os diferentes tipos de máquinas, utilizando apenas o caráter de dimensionalidade da máquina virtual dentro de um mesmo conjunto pré-determinado.

Outra limitação desse mecanismo é que ele foi projetado para gerenciar exclusivamente tarefas em lote, ignorando completamente serviços de longa duração como servidores. Ainda, o *STRATUS* carece de implementação em um ambiente real de orques-

tração como o *Kubernetes*, uma vez que teve sua experimentação realizada integralmente via simulação – limitando a captura de todas as nuances de um cenário real. Por fim, o mecanismo apresenta baixa portabilidade para outros provedores de nuvem, dado que o *STRATUS* baseou-se fortemente no mercado de VMs Spot (Amazon Web Services, 2026) – o que dificulta a reprodutibilidade em provedores com características distintas.

De forma semelhante, considerando características específicas de serviços ofertados por provedores de nuvem, destaca-se o autoprovisionamento de nós (*Node Auto Provisioning* - *NAP*) presente na *Google Cloud Platform* - *GCP*. Esse mecanismo possibilita que o *Google Kubernetes Engine* (*GKE*) – um serviço de gerenciamento de clusters *Kubernetes* – crie e administre automaticamente node pools com base em requisitos da carga de trabalho. Com o objetivo de reduzir custos com a gestão da infraestrutura, o *NAP* opera como uma expansão do autoescalonamento de clusters, provisionando novos *pools* de nós que cumpram os requisitos de hardware e os pedidos de recursos dos *Pods* pendentes. Além disso, esse serviço possibilita que o usuário defina manualmente o tamanho mínimo das instâncias – como quantidade mínima de *cores* e memória – e famílias específicas de máquinas dos nós presentes nos *pools* criados automaticamente.

Por conta dessas características, os autores em (TAMIRU et al., 2020) avaliaram experimentalmente o *NAP* em comparação com o autoescalonador de clusters padrão do *Kubernetes*. Utilizando o *GKE* como ambiente de experimentação, os pesquisadores empregaram duas aplicações representativas com cargas de trabalho distintas: uma composta por tarefas de curta e longa duração baseada em *traces* reais de um cluster do Google e outra baseada em uma aplicação de microsserviços submetida a uma carga sintética.

Os resultados demonstraram que, de forma geral, o *CA-NAP* apresentou melhor desempenho de autoescalonamento que o *CA*, uma vez que o provisionamento de nós de tamanhos heterogêneos permitiu uma adequação mais precisa à demanda da carga de trabalho. Ainda, os autores constataram que o desempenho do *CA-NAP* é influenciado principalmente pela composição da carga, destacando-se em cenários compostos por diversas tarefas de curta e longa duração com requisições de recursos diversificadas. Entretanto, contrariando as expectativas iniciais, observou-se que o *CA-NAP* não proporcionou uma redução de custos significativa em comparação ao *CA*.

Por fim, cabe destacar que o *NAP* não permite definir um número mínimo de nós que cada *pool* deve possuir, de modo que, para aplicações que necessitam de uma quantidade mínima de nós, é necessário criar um conjunto manualmente. Ademais, o *NAP* atua sobre os *requests* de recursos declarados pelos *Pods*, e não sobre o consumo real

medido em execução. Como consequência, ainda que o autoescalador possa consolidar nós subutilizados — removendo-os e reagendando seus *Pods* —, ele não reavalia o *tier* da máquina: uma vez que um *pool* é criado, a carga de trabalho não é migrada para um conjunto mais econômico enquanto permanecer agendável. De maneira inversa, caso os *Pods* em execução passem a consumir uma quantidade elevada de recursos sem que isso se reflita em seus *requests*, o NAP não aplica nenhuma política de migração para um *pool* de maior desempenho. Isso ocorre porque o método retratado depende que novos *Pods* sejam criados — manualmente ou por meio do *HPA* — e fiquem pendentes de agendamento para que uma nova ação de provisionamento de nós seja disparada.

Com isso, evidencia-se a necessidade de abordagens que considerem os diferentes tipos de máquinas na otimização de custo, bem como a habilidade de migrar a carga de trabalho entre *pools* de diferentes capacidades – aspectos que o presente trabalho busca endereçar. Ademais, tanto em (CHUNG; PARK; GANGER, 2018) como em (ZHONG; BUYYA, 2020), a implementação da solução depende das estimativas do tempo de execução das tarefas para a tomada de decisão – informação que nem sempre é trivial de obter-se em ambientes reais. Em contraste, o presente trabalho propõe uma abordagem que não exige qualquer conhecimento prévio sobre o tempo de execução das tarefas para a tomada de decisões, abrangendo principalmente aplicações de longa duração.

3.2 Estratégias de Provisionamento de Recursos em Nível de Aplicação

Segundo (LAKSHAN; HUSSAIN, 2025), organizações que executam aplicações em infraestruturas de nuvem devem priorizar a otimização de custos, uma vez que estratégias ineficientes de alocação de recursos podem elevar significativamente as despesas operacionais. Nesse estudo, os autores destacam que as políticas atuais de alocação de recursos em clusters do *Kubernetes* carecem da capacidade de adaptar-se dinamicamente às necessidades do negócio, o que pode resultar em desperdício de recursos durante flutuações de demanda. Diante desse cenário, o estudo apresenta diversas abordagens do estado da arte em autoescalamento inteligente para endereçar esses desafios, como escalonamento preditivo, otimização multiobjetivo e técnicas baseadas em inteligência artificial, incluindo aprendizado por reforço e aprendizado profundo.

Dentre essas abordagens, destaca-se o escalonamento preditivo como uma estratégia eficaz para mitigar problemas de subprovisionamento e superprovisionamento em clusters do *Kubernetes*. Essa técnica utiliza modelos de previsão de séries temporais e

aprendizado de máquina para antecipar a demanda por recursos e otimizar sua alocação. Séries temporais consistem em dados organizados ao longo do tempo, nos quais a ordem cronológica é relevante para a análise. Nesse contexto, os autores observaram que o uso do escalonamento preditivo contribui para a manutenção da qualidade de serviço, ao mesmo tempo em que reduz os custos operacionais por meio da antecipação da carga de trabalho futura. Esse comportamento também é evidenciado em (KURANAGE et al., 2023), no qual a aplicação de uma abordagem baseada em aprendizado profundo apresentou melhor equilíbrio entre custo e desempenho quando comparada ao escalonamento horizontal reativo padrão do *Kubernetes*.

Em contrapartida, diversos trabalhos utilizam técnicas de autoescalonamento reativas, as quais empregam fatores atuais do sistema para determinar quando e como os recursos serão escalonados. Esse mecanismo monitora determinada métrica especificada pelo usuário – como utilização de CPU, latência de rede ou quantidade de usuários ativos no sistema – e adequa o provisionamento de recursos caso o valor atual da métrica em análise coincida com limites ou valores-alvo pré-definidos. Por exemplo, em (BAARZI; KESIDIS, 2021) os autores propõem o SHOWAR, um mecanismo de escalonamento reativo customizado para microsserviços que, ao invés de monitorar a utilização de CPU como o HPA padrão do *Kubernetes*, monitora a latência de fila de execução do kernel Linux (runq latency). Quando esse valor ultrapassa um limiar definido pelo operador, um controlador PID ajusta reativamente o número de réplicas do microsserviço para corrigir o desvio observado.

Ademais, é importante destacar que a aplicação de técnicas de escalonamento reativo apresenta, em geral, resultados inferiores no desempenho e custo monetário em comparação aos mecanismos de escalonamento preditivo (TRAN; VU; KIM, 2022). Entretanto, o mecanismo reativo destaca-se pela maior facilidade de implementação, implicando menor carga operacional — sendo, inclusive, o mecanismo padrão de autoescalonamento do *Kubernetes*. Ainda, diversos trabalhos combinam a fácil implementação de métodos reativos com a eficiência do escalonamento preditivo. Por exemplo, em (AHMAD et al., 2025) os autores propõem uma abordagem híbrida composta por dois mecanismos complementares: o *Smart HPA*, um escalonador reativo que monitora a demanda atual dos microsserviços e otimiza o compartilhamento de recursos entre eles; e o *ProSmart HPA*, um escalonador proativo baseado em aprendizado de máquina que antecipa variações de carga e realiza o provisionamento de recursos antes que os limiares sejam violados.

Diante do exposto, observa-se que, embora abordagens preditivas e híbridas apre-

sentem vantagens em desempenho, sua complexidade de implementação pode ser um fator limitante em determinados contextos. Nesse sentido, o presente trabalho adota uma abordagem que prioriza a viabilidade prática e a facilidade de implantação, cujos detalhes são discutidos no Capítulo 4.

3.3 Avaliação de Desempenho de Autoescaladores

De maneira distinta dos trabalhos supracitados, os quais propõem novos mecanismos de autoescalamento, em (STRAESSER et al., 2023) os autores apresentam uma metodologia prática de avaliação e configuração de autoescaladores. Segundo os autores, não existe na literatura uma metodologia de avaliação amplamente utilizada, por conta disso, muitos trabalhos usam métricas próprias ou avaliam apenas por simulação – o que dificulta a transferência dos resultados para a prática. Desse modo, o artigo propõe um processo dividido em cinco etapas para garantir uma avaliação padronizada, reproduzível e de implementação viável.

Os autores caracterizam a primeira etapa como a definição do comportamento de uso do sistema em análise. Em um cenário real, diferentes aplicações – ou até mesmo tarefas da mesma aplicação – podem induzir cargas muito diferentes, de modo que é necessário encontrar uma distribuição realista de requisições, tal como parâmetros e conjuntos de dados que sejam condizentes com a carga real do sistema. Diretamente relacionado com isso, a segunda etapa foca-se em selecionar um padrão de tráfego factível, visto que usar um ciclo real de 24 horas por experimento seria custoso demais. Por conta desse fator, os autores sugerem selecionar padrões de interesse da carga de trabalho analisada como, por exemplo, trechos nos quais a carga aumenta ou diminui com maior frequência. Ainda, o estudo abrange nessa etapa a estimativa da carga máxima suportada pelo ambiente de testes, uma vez que pode ser necessário reescalar o padrão para essa capacidade obtida.

Na etapa três o objetivo destacado pelos autores é garantir que o estado inicial do sistema seja sempre o mesmo no início do experimento. Os autores destacam que essa é a única etapa que não pôde ser generalizada no guia, pois depende fortemente do ambiente de nuvem alvo. Ademais, na etapa quatro é realizada uma análise qualitativa do comportamento do autoescalador, na qual os autores propõem a observação de sete antipadrões que indicam possíveis configurações inadequadas. Esses antipadrões são avaliados a partir de quatro cenários simples de carga de trabalho — como carga crescente,

decrecente e constante — e abrangem comportamentos como a ausência de reação a variações de demanda, o provisionamento incorreto em estado estável, o escalonamento oscilatório (jitter), além de situações de escalonamento e desescalonamento rápidos ou lentos demais. Para cada antipadrão, os autores apontam as possíveis causas associadas e sugerem reconfigurações apropriadas, facilitando a interpretação dos resultados pelo profissional responsável pela configuração.

Por fim, na etapa cinco é conduzida uma análise quantitativa, na qual o autoescalador sob avaliação é comparado com outra configuração ou com uma linha de base (baseline) por meio de métricas bem definidas. Nesse contexto, os autores consideram a razão de violações de SLO, o custo de execução — quantificado pela integral da quantidade de instâncias provisionadas ao longo do tempo — e uma métrica combinada que pondera ambos os objetivos de acordo com a prioridade do caso de uso.

Nesse âmbito, embora os autores foquem seus esforços na avaliação e configuração de autoescaladores — utilizando principalmente o *HPA* como objeto de estudo —, a experimentação e a configuração do artefato proposto no presente trabalho basearam-se fortemente nas etapas supracitadas. Em especial, as três primeiras etapas foram utilizadas diretamente na realização da experimentação discutida no Capítulo 5. Com isso, busca-se atingir uma avaliação coerente do trabalho proposto por meio da adoção das etapas da metodologia que são diretamente aplicáveis ao contexto deste trabalho, haja vista que essas abordam aspectos relevantes tanto para a avaliação de autoescaladores quanto para a experimentação realizada neste trabalho.

4 PROPOSTA DO CANM

Neste capítulo, apresentaremos a definição do CANM, a problemática que ele visa endereçar e suas semelhanças e diferenças com os trabalhos relacionados supracitados. Ademais, discutiremos como esse mecanismo foi efetivamente implementado e os componentes que o integram, abordando as decisões de arquitetura que o CANM emprega, métricas consumidas para a tomada de decisões, ações de compensação e reconciliação do estado do cluster em eventuais falhas críticas. Por fim, detalharemos os hiperparâmetros da aplicação, os quais possibilitam que o mecanismo seja adaptado a diferentes cenários de carga de trabalho.

4.1 Visão Geral e Objetivo

O CANM é um sistema autônomo de otimização de custos em clusters do *Kubernetes* que executam em provedores de nuvem. Esse artefato observa continuamente o uso de recursos dos nós e, quando detecta subutilização ou sobrecarga, executa migrações automáticas entre *pools* de nós de capacidades e custos distintos. Diferentemente do auto-escalador de cluster nativo do *Kubernetes*, o CANM não expande ou retrai a quantidade de recursos totais, mas visa otimizar a utilização desses recursos considerando máquinas com a mesma capacidade de recursos – diferenciando-se por características da instância ofertada pelo provedor de nuvem, como família, série e tipo.

Por conta disso, o CANM é capaz de reduzir custos em cenários nos quais os autoescaladores citados na Seção 3.1 não conseguem, como, por exemplo, quando a quantidade mínima de instâncias é atingida. Isso ocorre porque esses mecanismos reduzem custos essencialmente removendo máquinas ociosas; assim, uma vez atingido o número mínimo de instâncias pré-determinado, tornam-se incapazes de obter economia adicional, ainda que essas instâncias operem em baixa carga. O CANM, por sua vez, não reduz a frota abaixo desse mínimo, e sim diminui o custo migrando essas mesmas instâncias para um *pool* mais econômico. Esse cenário é potencializado em situações em que o usuário necessita de uma quantidade mínima considerável de máquinas para garantir alta disponibilidade e redundância. Assim, em um cenário em que é exigido um mínimo de dez máquinas executando constantemente, os autoescaladores tradicionais são ineficazes em diminuir o custo com as instâncias provisionadas, ao passo que o CANM pode migrá-las para máquinas mais econômicas, mantendo a quantidade mínima especificada.

Ademais, o CANM atua em contextos em que é preferível ou exigido responder ao aumento da demanda sem expandir a quantidade de máquinas executando no cluster. Nessas situações, o CANM é capaz de realizar a migração da carga de trabalho para famílias de máquinas com maior desempenho por núcleo, assim, a demanda é atendida pela mesma quantidade de instâncias – respeitando os requisitos pré-definidos. Um exemplo dessa situação, considerando as instâncias do *Google Compute Engine* e uma carga de trabalho executando inicialmente em uma instância *e2-standard-4* (US\$ 0,13402284/h¹), os autoescaladores citados anteriormente podem decidir adicionar uma nova instância diante do aumento do uso de CPU – dobrando o custo total (US\$ 0,26804568/h). Sob outra ótica, o CANM migra a carga de trabalho para uma máquina otimizada para computação *c2-standard-4* (US\$ 0,208808/h²) para suportar essa demanda – a um custo 22% menor que a solução anterior.

Logo, o CANM tem como objetivo minimizar o custo com provisionamento de máquinas e responder ao aumento dinâmico da demanda sem expandir a frota, migrando a carga de trabalho para máquinas de maior desempenho em vez de adicionar novas instâncias. Cabe a ele detectar nós com baixa utilização e migrar sua carga de trabalho para máquinas mais econômicas. Adicionalmente, ele é capaz de identificar nós com alta demanda de processamento e migrar para instâncias com melhor desempenho. Essas ações são realizadas automaticamente graças aos mecanismos reativos que o artefato implementa – os quais garantem que as decisões de operação levem o cluster de um estado adequado para outro. Tais ações executam durante o ciclo de operação do CANM, o qual discutiremos detalhadamente a seguir.

4.2 Ciclo de operação

Todas as operações implementadas pelo CANM executam durante um ciclo de operação – também denominado de *tick*. Cada tick é processado em intervalos de tempo definidos pelo usuário e visa otimizar a utilização dos recursos do cluster através da análise individual dos nós. Nesses ciclos, é possível que haja uma migração de instâncias, uma reconciliação do estado do cluster ou que nenhuma ação seja efetuada. Esta seção detalha

¹Preço sob demanda da instância *e2-standard-4* (família *general-purpose*) na região *us-central1* (Iowa). Disponível em: <<https://cloud.google.com/products/compute/pricing/general-purpose>>. Acesso em: 1 jun. 2026.

²Preço sob demanda da instância *c2-standard-4* (família *compute-optimized*) na região *us-central1* (Iowa). Disponível em: <<https://cloud.google.com/products/compute/pricing/compute-optimized>>. Acesso em: 1 jun. 2026.

o funcionamento do mecanismo de migração, desde a identificação de nós candidatos até as ações realizadas no cluster para atingir o objetivo esperado. Além disso, vale destacar que, embora a reconciliação seja a primeira etapa do ciclo de operação, seu funcionamento será discutido na Seção 4.4, pois está diretamente relacionado à arquitetura do CANM.

4.2.1 Dimensionalidade

Antes de iniciarmos a discussão sobre os mecanismos supracitados, é fundamental explicitar que o CANM opera restritivamente sobre exatos dois *pools* de nós previamente definidos. Em clusters *Kubernetes* hospedados em provedores de nuvem, é comum a existência de múltiplos *pools* com características e finalidades distintas. Nesse contexto, a organização de um cluster hipotético pode determinar que microsserviços sejam executados em um determinado *pool* de nós, enquanto o banco de dados permanece alocado em outro. Na versão atual, o mecanismo proposto requer que sejam informados previamente quais *pools* de nós serão considerados durante seu ciclo de operação. Além disso, ele não identifica automaticamente qual *pool* contém os nós de maior desempenho e qual contém os nós de menor custo computacional, nem se existe diferença de performance entre eles. Dessa forma, cabe ao usuário criar os *pools* e informar ao artefato a semântica associada a cada um deles. Finalmente, para fins de padronização, o agrupamento de maior capacidade computacional será denominado *high pool*, enquanto a estrutura de computação mais econômica será chamada de *low pool*.

Essa abordagem visa facilitar a implementação do mecanismo e possibilitar que o CANM seja aplicado a diferentes cenários e demandas, permitindo que o usuário defina os *pools* de nós mais adequados às suas necessidades. Ademais, a proposta considera que esses *pools* possuam recursos computacionais equivalentes, tais como CPU, memória e armazenamento. Entretanto, conforme mencionado anteriormente, essa não constitui uma restrição para a operação do CANM, uma vez que cabe ao usuário criar, configurar e informar tais definições ao sistema. Todavia, a utilização de *pools* com paridade de recursos computacionais minimiza a ocorrência de situações em que *Pods* provenientes de nós em processo de migração não possam ser escalonados nos nós de destino devido à insuficiência de recursos disponíveis.

Por exemplo, a Tabela 4.1 apresenta uma configuração que minimiza a situação descrita anteriormente. Nela, são apresentados dois *pools* de nós com equivalência de recursos computacionais, diferenciando-se apenas pelo desempenho de seus processadores.

Tabela 4.1: Exemplificação de *pools* de nós com equivalência de recurso

Pool	Máquina	vCPUs	RAM	Processador	Custo/h (USD) ¹
A	c2-standard-4	4	16 GiB	Intel Xeon Cascade Lake	0.209
B	e2-standard-4	4	16 GiB	Intel Xeon ou AMD EPYC ²	0.134

¹ Instâncias do Google Compute Engine (GCE). Valores de referência sob demanda (on-demand), sem considerar Committed Use Discounts (CUDs) ou outras modalidades de desconto.

² Nas instâncias E2, a plataforma de CPU é definida pelo Google Cloud e pode variar entre processadores Intel Xeon e AMD EPYC conforme a região e a disponibilidade da infraestrutura. Os preços podem variar de acordo com a região selecionada.

4.2.2 Pontuação dos Nós

Conforme discutido no início deste capítulo, o CANM detecta nós subutilizados ou sobreutilizados nos *pools* analisados e executa migrações com o objetivo de adequar a carga de trabalho para uma instância mais econômica ou com melhor desempenho computacional. Para conseguir realizar essa identificação, o CANM aplica uma pontuação para cada nó presente nesses *pools*. Essa pontuação relaciona a utilização agregada de CPU e memória do nó a pesos normalizados, os quais determinam a contribuição de cada recurso para o valor final calculado. Dessa forma, para cada nó presente nos *pools* aplica-se a fórmula abaixo:

$$node_score = (C_w \cdot C_u + M_w \cdot M_u) \div 100 \quad (4.1)$$

onde:

- C_w e M_w são os **pesos** atribuídos, respectivamente, à CPU e à memória — definidos pelo usuário e normalizados internamente pelo mecanismo antes do cálculo —, que determinam a contribuição relativa de cada recurso para a pontuação;
- C_u e M_u são a **utilização** agregada, respectivamente, de CPU e de memória do nó, expressa em percentual no intervalo $[0, 100]$.

As variáveis C_w e M_w são parametrizadas pelo usuário, uma vez que a relevância de cada recurso para a pontuação final depende diretamente das características da aplicação avaliada. Nesse sentido, em cenários nos quais a aplicação seja limitada por CPU, é possível atribuir um peso maior à variável C_w em relação à M_w . De forma análoga, aplicações limitadas por memória podem priorizar a variável M_w . Ademais, qualquer valor

maior ou igual a zero pode ser atribuído a essas variáveis, visto que o CANM realiza a normalização dos pesos antes de aplicá-los ao cálculo da pontuação.

Já as variáveis C_u e M_u representam a utilização agregada de CPU e memória do nó, respectivamente. A fim de obter essas informações, o mecanismo realiza uma requisição HTTP para o servidor de métricas Prometheus ³. Nesse contexto, vale destacar que o CANM não implementa um servidor de métricas próprio, o mecanismo apenas executa *queries* no formato PromQL (*Prometheus Query Language*) para um servidor externo informado pelo usuário através da variável de ambiente `PROMETHEUS_API_URL`.

Para que tais consultas sejam atendidas, o servidor Prometheus informado pelo usuário deve satisfazer um conjunto de pré-requisitos. É necessário que ele colete (*scrape*) as métricas expostas pelo *cAdvisor* embutido no *kubelet* de cada nó, sob um *job* cujo nome corresponda ao valor configurado no parâmetro `PROMETHEUS_JOB`. Além disso, as séries temporais coletadas devem carregar um *label* `node` que identifique o nó de origem de cada métrica, comumente obtido por meio de uma regra de *relabeling* que mapeia o nome do nó do *Kubernetes* para esse *label*. Esse requisito é fundamental, pois é o *label* `node` que permite tanto a agregação por nó (`by (node)`) quanto o casamento entre as séries de consumo e de capacidade durante a divisão.

A partir dessas séries, o mecanismo consome quatro métricas distintas. As métricas de tempo de CPU consumido e o *working set* de memória dos contêineres, sendo restritas ao *cgroup* raiz (`id="/"`) para refletir o consumo agregado do nó como um todo. Já as métricas `machine_cpu_cores` e `machine_memory_bytes` expõem, respectivamente, a quantidade de núcleos e a memória total da máquina subjacente. Dessa forma, as duas primeiras métricas compõem o numerador (consumo) e as duas últimas, o denominador (capacidade) das consultas, de modo que sua razão, multiplicada por cem, resulta no percentual de utilização de CPU e de memória de cada nó.

A fim de obter as métricas de utilização agregada de CPU e memória dos nós do cluster, o CANM executa as consultas apresentadas no Código 4.1 junto ao servidor de métricas Prometheus. Como as consultas retornam os valores agregados por nó, é necessário executá-las apenas uma vez a cada ciclo de operação do mecanismo.

```
sum(rate(container_cpu_usage_seconds_total{id="/" }[$time_window]))
  by (node)
  / sum(machine_cpu_cores) by (node) * 100
```

³O Prometheus é um sistema de monitoramento e servidor de métricas *open source* responsável por coletar, armazenar e disponibilizar séries temporais de métricas provenientes de aplicações e infraestruturas computacionais. Referência: <https://prometheus.io/docs/introduction/overview/>


```
( sum(container_memory_working_set_bytes{id="/"}) by (node)
  / sum(machine_memory_bytes) by (node) ) * 100
```

Código 4.1: Consultas *PromQL* para o cálculo da utilização agregada de CPU e memória por nó.

Nessas consultas, a variável `time_window` é definida pelo usuário e corresponde ao intervalo temporal considerado para o cálculo da utilização média de CPU.

Por último, como o resultado da Equação 4.1 é dividido pelo literal 100, a pontuação do nó tende a se situar no intervalo $[0,1]$, em que 0 indica um nó completamente ocioso e 1 indica um nó com utilização integral de seus recursos de CPU ou memória. Cabe ressaltar, contudo, que esse intervalo representa o comportamento esperado em regime normal de operação: como o mecanismo não aplica limitação explícita (*clamping*) ao valor final, situações como a ausência de métricas para um determinado nó ou artefatos de medição — por exemplo, a extrapolação do `rate` do Prometheus, capaz de reportar utilização de CPU superior a 100% — podem, pontualmente, produzir pontuações fora desse intervalo. Entretanto, como veremos a seguir, a ocorrência de pontuações superiores a 1 não compromete a decisão de migração, visto que a política não utiliza o valor absoluto da pontuação, mas apenas o compara aos limiares de decisão.

Por fim, cabe delimitar o escopo do modelo de pontuação: na versão atual, apenas os recursos de CPU e memória são considerados, por serem os que caracterizam a capacidade das famílias de instâncias avaliadas neste trabalho. Recursos especializados, como GPUs, não são contemplados pela fórmula. Sua incorporação exigiria a adição de novas métricas e de pesos correspondentes, bem como a garantia de que os *pools* de origem e de destino ofereçam tais recursos de forma equivalente, de modo a não inviabilizar o reagendamento dos *Pods* durante a migração. Essa extensão é deixada como trabalho futuro.

4.3 Migração de Nós

Uma vez calculada a pontuação de todos os nós presente nos *pools* analisados, o CANM é capaz de identificar nós subutilizados ou sobreutilizados a fim de decidir se o nó deve ser migrado para uma instância no outro *pool*. Nesse contexto, para que o CANM decida realizar uma migração é necessário que o nó observado respeite as seguintes condições:

- **Não estar em período de *cooldown*.** Após sua criação, todo nó permanece imune à avaliação por um intervalo configurável — definido por `HIGH_NODE_COOLDOWN` para o *high pool* e por `LOW_NODE_COOLDOWN` para o *low pool*. Esse mecanismo evita que um nó recém-migrado seja imediatamente reavaliado, prevenindo migrações sucessivas em sentidos opostos. Caso não seja informado um valor para as variáveis supracitadas, o CANM utilizará 5 minutos como padrão para ambos os *pools*.
- **Possuir pontuação que ultrapasse o limiar associado ao seu *pool*.** Um nó do *high pool* é considerado subutilizado, e portanto candidato a migrar para o *low pool*, quando sua pontuação é menor ou igual ao limiar `LOW_SCORE_THRESHOLD` (padrão de 0,35). De forma análoga, um nó do *low pool* é considerado sobreutilizado, e candidato a migrar para o *high pool*, quando sua pontuação é maior ou igual ao limiar `HIGH_SCORE_THRESHOLD` (padrão de 0,60).

Pontuações situadas entre os dois limiares definem uma *zona segura*: nós nessa faixa apresentam utilização adequada ao seu *pool* e, portanto, não disparam nenhuma migração. Quando mais de um nó de um mesmo *pool* satisfaz essas condições em um mesmo ciclo de operação, o CANM seleciona aquele cuja pontuação é mais extrema — isto é, o nó de *menor* pontuação no *high pool* (mais ocioso) ou o de *maior* pontuação no *low pool* (mais sobrecarregado). Apenas uma migração é iniciada por ciclo, de modo que o impacto de cada ação sobre o cluster possa ser observado antes de uma eventual nova migração.

A ordem em que os *pools* são avaliados é determinada pela política configurada em `MIGRATION_POLICY`. Na política `prioritizeCost`, o CANM avalia primeiro o *high pool* em busca de nós subutilizados — priorizando a redução de custos — e, somente se nenhum candidato for encontrado, avalia o *low pool* em busca de nós sobrecarregados. A política `prioritizePerformance` inverte essa ordem, priorizando a resposta à sobrecarga antes da economia. Essa lógica de avaliação é exemplificada no Algoritmo 1.

Ademais, recomenda-se que o usuário defina valores específicos para os limiares com base nas especificidades das suas aplicações e infraestrutura, evitando a adoção automática dos parâmetros padrões supracitados. Essa customização é necessária porque os indícios de subutilização ou sobreutilização dependem diretamente do perfil de carga e do comportamento dinâmico de cada aplicação. Desse modo, um nó que mantém constantemente um score de 0,60 pode, em um cenário de alta criticidade, ser considerado estável, enquanto para um sistema de menor demanda esse mesmo valor pode indicar

ociosidade de recursos, justificando um ajuste fino nas métricas de monitoramento.

Um cenário de pior caso a se considerar é o de oscilação (*flapping*), no qual uma mesma carga migraria repetidamente entre os *pools* em sentidos opostos — por exemplo, um nó migrado para o *high pool* que, ao ver sua utilização relativa cair, torna-se imediatamente candidato a retornar ao *low pool*, e vice-versa. Esse comportamento é indesejável, pois cada migração impõe um custo transitório de desempenho e um sobre-custo associado à criação e à remoção de máquinas. O projeto do CANM contém três mecanismos que, em conjunto, contêm essa oscilação. Primeiro, o período de *cooldown* torna todo nó recém-migrado imune à reavaliação por um intervalo configurável, impedindo reversões imediatas. Segundo, a *zona segura* entre `LOW_SCORE_THRESHOLD` e `HIGH_SCORE_THRESHOLD` introduz uma histerese: para migrar em um sentido e depois no oposto, a pontuação do nó precisa atravessar toda a faixa entre os dois limiares, e não apenas flutuar em torno de um único ponto de decisão. Terceiro, as janelas de avaliação (`LOW_POOL_TIME_WINDOW_EVAL` e `HIGH_POOL_TIME_WINDOW_EVAL`, apresentadas na Tabela 4.2) exigem que a condição de subutilização ou sobreutilização se sustente ao longo de um intervalo de observação, filtrando flutuações transitórias de curta duração. Soma-se a isso o fato de que apenas uma migração é iniciada por ciclo, o que faz a composição do cluster evoluir de forma gradual e observável. Cabe ressaltar, contudo, que a eficácia dessa contenção depende da calibração adequada dos parâmetros: janelas de avaliação ou *cooldowns* demasiadamente curtos podem reintroduzir o risco de oscilação, reforçando a importância do ajuste discutido na Seção 5.5.

4.3.1 Etapas da Migração

Após garantir que o nó avaliado atende às condições de migração supracitadas, o CANM inicia o processo de migração por meio de etapas sequenciais denominadas sagas. Uma saga consiste em um conjunto de transações locais que, em caso de falha, executam ações compensatórias capazes de reverter as operações previamente realizadas. A motivação para a adoção dessa decisão arquitetural será discutida em profundidade na Seção 4.4. Ademais, o CANM realiza anotações nos nós de origem e destino ao longo do processo de migração. Essas anotações armazenam um conjunto de informações que permitem ao artefato rastrear ações previamente executadas em cada nó. Além disso, em situações de falhas críticas, tais informações possibilitam a identificação de nós que se encontram em estados inconsistentes, viabilizando a execução de procedimentos de

Algorithm 1: Decisão de migração em um ciclo de operação

```

1  $N_{high} \leftarrow$  nós do high pool ordenados por score crescente;
2  $N_{low} \leftarrow$  nós do low pool ordenados por score decrescente;
3 if política = prioritizeCost then
4   if avaliarPool( $N_{high}$ , LOW_SCORE_THRESHOLD,  $\leq$ , high→low) then
5     return;
6   avaliarPool( $N_{low}$ , HIGH_SCORE_THRESHOLD,  $\geq$ , low→high);
7 else
8   if avaliarPool( $N_{low}$ , HIGH_SCORE_THRESHOLD,  $\geq$ , low→high) then
9     return;
10  avaliarPool( $N_{high}$ , LOW_SCORE_THRESHOLD,  $\leq$ , high→low);
11
12 Função avaliarPool(nós, limiar, op, direção) :
13 foreach nó  $\in$  nós do
14   if nó está em cooldown then continua;
15   if score(nó) op limiar then
16     migrarNó(nó, direção);
17     return verdadeiro; // apenas uma migração por ciclo
18 return falso;

```

reconciliação do cluster.

O processo de migração é composto por três sagas, em que cada etapa somente é iniciada após a conclusão bem-sucedida da etapa anterior. O objetivo de cada saga é detalhado a seguir, bem como as respectivas ações compensatórias empregadas em situações de falha.

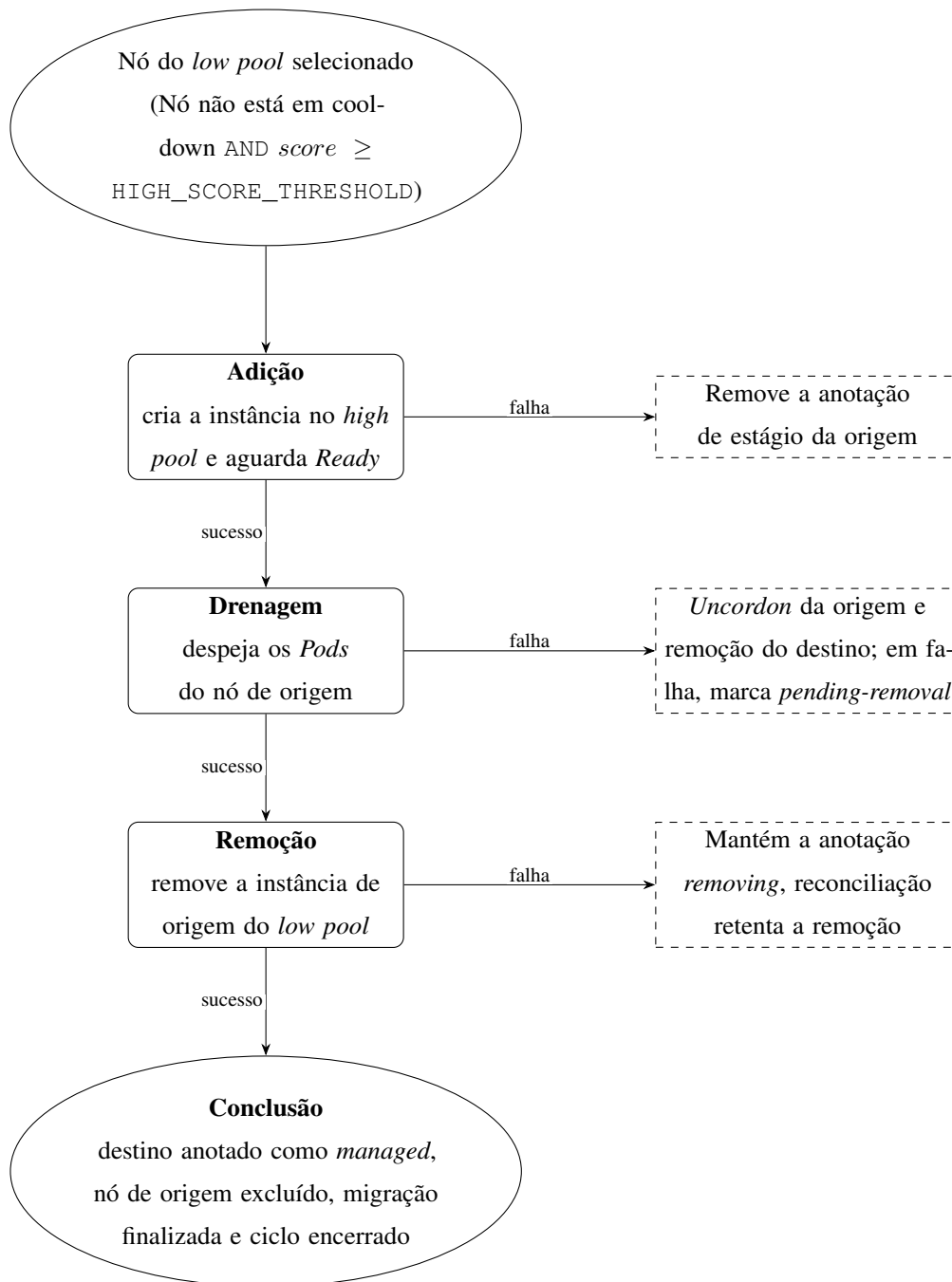
- **Adição.** Tem por objetivo criar a instância de destino no *pool* alvo. O nó de origem é anotado com o estágio atual (*canm.io/migration-stage:addition*), a instância é criada junto ao provedor e o mecanismo aguarda que ela seja registrada e fique pronta (*Ready*) no cluster, sendo então anotada como criada (*canm.io/state:created*). Adicionalmente, com o objetivo de facilitar a rastreabilidade das operações realizadas pelo CANM, toda nova instância criada pelo mecanismo utiliza o prefixo *canm* em sua nomenclatura. Essa convenção permite identificar de forma inequívoca os nós provisionados pelo artefato, característica fundamental para o correto funcionamento do mecanismo de reconciliação. **Compensação:** como nenhuma carga de trabalho foi movida e, em caso de falha, nenhuma instância chegou a ser provisionada, basta remover a anotação de estágio do nó de origem, restaurando o cluster ao estado inicial.
- **Drenagem.** Tem por objetivo esvaziar o nó de origem, despejando seus *Pods* para que sejam reagendados no nó de destino recém-criado (ou em outros nós disponíveis).

Compensação: o nó de origem é reabilitado (*uncordon*) e a instância de destino criada na etapa anterior é removida. Caso essa remoção falhe, o nó de destino é marcado como pendente de remoção, delegando a limpeza ao *loop* de reconciliação (discutido na Subseção 4.4.1) .

- **Remoção.** Tem por objetivo remover a instância de origem, já drenada, junto ao provedor de nuvem. **Compensação:** não há ação compensatória imediata, pois o nó de origem já foi drenado e o nó de destino já assumiu a carga; a anotação de estágio é mantida, sinalizando ao *loop* de reconciliação que a remoção deve ser retentada em ciclos subsequentes.

Concluída a etapa de remoção, o mecanismo anota a instância de destino como gerenciada (*canm.io/state:managed*), registrando a migração como bem-sucedida, encerrando o processo de migração e, por conseguinte, o ciclo de operação. Para exemplificar o que foi descrito nesta seção, o diagrama abaixo ilustra o processo completo de migração de um nó inicialmente pertencente ao *low pool*:

Figura 4.1: Etapas da migração de um nó do *low pool* para o *high pool*.



4.4 Arquitetura

Conforme discutido no início da Subseção 4.3.1, o processo de migração do CANM é estruturado por meio de sagas, compostas por transações locais e respectivas ações compensatórias executadas em situações de falha. Essa abordagem decorre da necessidade de garantir que as operações realizadas pelo mecanismo não conduzam o cluster a estados inconsistentes ou indesejados. Nesse contexto, o CANM assume que

chamadas de API ao provedor de nuvem estão sujeitas a falhas e, consequentemente, que operações previamente executadas podem necessitar de reversão. Por exemplo, conforme ilustrado na Figura 4.1, caso ocorra uma falha durante a drenagem do nó de origem, o nó de destino previamente criado é removido por meio de uma ação compensatória, e o processo de migração é interrompido.

Além disso, o CANM considera a possibilidade de falhas críticas no processo responsável por sua execução, as quais podem interromper o fluxo de migração em etapas intermediárias. Por exemplo, caso ocorra uma falha entre as sagas de adição e drenagem, o cluster poderá permanecer com um nó adicional não previsto pelo usuário. Para lidar com esse cenário, o CANM implementa um mecanismo de reconciliação responsável por identificar e corrigir estados inconsistentes no início dos ciclos de operação.

4.4.1 Reconciliação

Utilizando o prefixo do nome das instâncias do cluster e as anotações dos nós definidas em 4.3.1, o mecanismo de reconciliação identifica todos os nós que foram criados ou alterados pelo CANM. Desse modo, no início de cada ciclo de operação — e antes de qualquer nova avaliação do cluster — esse mecanismo inspeciona tais nós em busca de estados inconsistentes, decorrentes de migrações anteriores que não foram concluídas, seja por uma falha crítica em alguma das etapas, seja pelo encerramento abrupto do processo entre a execução de uma operação e o registro de sua respectiva anotação. Caso algum estado pendente seja encontrado, o CANM executa a ação corretiva apropriada ao caso identificado e posterga a avaliação do cluster para o ciclo seguinte. Com isso, garante-se que nenhuma nova migração seja iniciada enquanto houver estados incompletos a resolver, evitando que decisões sejam tomadas sobre um cluster que ainda não se encontra em um estado consistente.

Dentre as correções aplicadas pela reconciliação, destacam-se:

- **Exclusão de nós órfãos.** Caso exista um nó criado pelo CANM (identificado pelo prefixo) que não possua nenhuma anotação `canm.io/*`, conclui-se que sua criação foi interrompida antes que o mecanismo registrasse seu estado. Como esse nó nunca chegou a assumir carga de trabalho e seu propósito é indeterminado, o CANM o remove do cluster, evitando o custo de uma instância órfã.
- **Nó em estado não terminal.** Refere-se a um nó criado pelo CANM cuja ano-

tação `canm.io/state` é diferente de *managed* — isto é, *created* ou *pending-removal*. Esse cenário indica uma migração interrompida antes de sua conclusão ou uma compensação que não pôde remover o nó de destino. Para um nó em estado *created*, o mecanismo consulta o nó de origem referenciado na anotação `canm.io/source-node`: se a origem já não existe, ou se ela ainda está na etapa de remoção, conclui-se que o destino já assumiu a carga de trabalho e ele é promovido a *managed*; caso contrário, o nó de destino é removido. Para um nó em estado *pending-removal*, a remoção é simplesmente retentada.

- **Nó de origem preso em uma etapa da migração.** Refere-se a um nó de origem que ainda carrega a anotação `canm.io/migration-stage`, sinalizando que uma das etapas da migração não foi concluída. A ação corretiva depende da etapa em que o nó ficou retido: na etapa de **adição**, basta remover a anotação, pois nenhuma instância de destino chegou a ser criada; na etapa de **drenagem**, o nó de origem é reabilitado (*uncordon*), caso esteja isolado, e a anotação é removida; e na etapa de **remoção**, a exclusão da instância de origem — já drenada — é retentada.

Por fim, é aplicada a anotação `canm.io/last-reconciliation` a todos os nós para os quais foi necessária alguma ação de reconciliação, com exceção daqueles que foram removidos do cluster. Essa anotação registra o instante da última tentativa de reconciliação realizada pelo mecanismo.

Antes de iniciar a reconciliação de um nó em estado inconsistente, o CANM consulta o valor dessa anotação. Caso o intervalo de tempo decorrido desde a última tentativa seja inferior a cinco minutos, a reconciliação do nó é temporariamente adiada. Essa estratégia evita que um mesmo nó seja continuamente reavaliado em cenários nos quais a própria operação de reconciliação esteja sujeita a falhas. Consequentemente, o mecanismo garante que outros nós em estados inconsistentes também possam ser processados, promovendo uma distribuição mais equilibrada dos esforços de reconciliação ao longo do cluster.

4.4.2 Parametrização e Adaptabilidade

Adicionalmente, é importante destacar que a arquitetura do CANM foi projetada de forma a possibilitar sua adaptação a diferentes cenários de utilização. Para isso, o mecanismo externaliza parâmetros operacionais estratégicos por meio de variáveis de ambiente, permitindo que o usuário o configure de acordo com as características e requisitos

específicos de sua aplicação. A Tabela 4.2 apresenta os parâmetros que podem ser definidos para personalizar o comportamento do mecanismo.

Tabela 4.2: Principais parâmetros de configuração do CANM e seus efeitos.

Parâmetro	O que modifica
<i>Prometheus</i>	
PROMETHEUS_API_URL	Endereço do servidor Prometheus consultado para obter as métricas dos nós.
PROMETHEUS_JOB	Nome do <i>job</i> do Prometheus que expõe as métricas dos nós a serem consultadas.
<i>Node Pools</i>	
HIGH_NODE_POOL	Nome do <i>node pool</i> de maior capacidade e custo (<i>high pool</i>).
LOW_NODE_POOL	Nome do <i>node pool</i> de menor capacidade e custo (<i>low pool</i>).
<i>Decisão de migração</i>	
LOW_SCORE_THRESHOLD	Limiar de pontuação abaixo do qual um nó do <i>high pool</i> é considerado subutilizado e candidato à migração para o <i>low pool</i> .
HIGH_SCORE_THRESHOLD	Limiar de pontuação acima do qual um nó do <i>low pool</i> é considerado sobrecarregado e candidato à migração para o <i>high pool</i> .
MIGRATION_POLICY	Ordem de avaliação dos <i>pools</i> : prioriza a redução de custo (<i>prioritizeCost</i>) ou o desempenho (<i>prioritizePerformance</i>).
CPU_WEIGHT	Peso da utilização de CPU no cálculo da pontuação do nó.
MEMORY_WEIGHT	Peso da utilização de memória no cálculo da pontuação do nó.
<i>Cooldowns</i>	
HIGH_NODE_COOL_DOWN	Período em que um nó recém-criado no <i>high pool</i> fica imune à reavaliação, evitando migrações sucessivas.
LOW_NODE_COOL_DOWN	Período em que um nó recém-criado no <i>low pool</i> fica imune à reavaliação, evitando migrações sucessivas.
<i>Temporização</i>	
CHECK_INTERVAL	Intervalo entre os ciclos de operação (reconciliação seguida de avaliação).
LOW_POOL_TIME_WINDOW_EVAL	Janela temporal considerada no cálculo da utilização dos nós do <i>low pool</i> .
HIGH_POOL_TIME_WINDOW_EVAL	Janela temporal considerada no cálculo da utilização dos nós do <i>high pool</i> .

4.5 Componentes

O CANM é constituído por cinco componentes principais, sendo cada componente responsável por uma série de ações específicas que conjuntamente possibilitam o pleno funcionamento da aplicação. Esses componentes foram implementados como classes do *Typescript* em um ambiente de execução *Node.js*. Optamos por utilizar tais ferramentas

uma vez que elas facilitam a utilização de tipagem de dados, realização de chamadas HTTPS para APIs externas, execução *bash scripts* e portabilidade para diversos sistemas. Os componentes que constituem o CANM estão descritos abaixo.

4.5.1 Migrator Orchestrator

É o componente central e coordenador do CANM. Sua responsabilidade é conduzir o ciclo de operação: a intervalos regulares, ele solicita os *scores* de utilização dos nós, decide — com base na política configurada e nos limiares de decisão — quais nós devem ser migrados e em qual direção, e orquestra a execução da migração. Cabe a ele também a reconciliação de migrações incompletas. É o único componente que toma decisões; as ações concretas são delegadas aos demais.

4.5.2 Node Migrator

É responsável pelas operações sobre os nós específicas do provedor de nuvem. Esse componente traduz as decisões do *Migrator Orchestrator* em ações sobre a infraestrutura, como a criação e a remoção de instâncias e a coordenação da drenagem e do isolamento dos nós envolvidos na migração. Desse modo, ele abstrai as particularidades do provedor, permitindo que a lógica de decisão permaneça independente do provedor de nuvem utilizado.

4.5.3 Metrics Adapter

É responsável por fornecer os dados que orientam as decisões de migração. Consulta o *Prometheus* para obter as métricas de utilização (CPU e memória) de cada nó e as consolida em um *score* composto e ponderado. É a partir desse *score* que o *Migrator Orchestrator* avalia os limiares e determina se uma migração deve ocorrer.

4.5.4 Kubernetes Client

É a camada de comunicação com o *cluster Kubernetes*. Concentra as operações sobre a API do *Kubernetes* e o *kubectl*, como listar nós e *Pods*, drenar e isolar nós, anotar nós com o estado da migração, entre outras funcionalidades. Semelhante ao Node Migrator, esse componente abstrai a implementação da comunicação com o *Kubernetes*, oferecendo aos demais componentes uma interface única para interagir com o cluster.

4.5.5 Audit Logger

É responsável pela observabilidade e pela rastreabilidade das ações do mecanismo. Persiste o registro de cada migração, compensação e reconciliação executada. Dessa forma, torna possível auditar o comportamento do CANM e analisar, a posteriori, as decisões e ações tomadas pelo sistema.

5 METODOLOGIA

Com o objetivo de avaliar o comportamento do CANM em uma infraestrutura real, sujeita a variações dinâmicas na utilização de recursos, foi necessário conduzir um conjunto de experimentos capazes de verificar a efetividade da otimização de custos proposta pelo mecanismo. Nesse contexto, nossa parceira na indústria, a Econodata¹, disponibilizou um cluster de testes com configuração equivalente à do ambiente de produção, além de informações sobre os padrões de utilização de recursos observados no cluster principal em diferentes cenários operacionais.

Neste capítulo, inicialmente descrevemos a configuração do ambiente experimental e as características da arquitetura alvo, identificando os recursos computacionais que limitam seu desempenho. Em seguida, apresentamos os padrões de utilização de recursos observados no ambiente de produção e a forma como esses comportamentos foram reproduzidos no ambiente de experimentação por meio da ferramenta de testes de carga *k6*. Adicionalmente, detalhamos as métricas de avaliação e os modelos de referência utilizados para comparar o CANM, cujos resultados serão apresentados no Capítulo 6. Por fim, com base na Tabela 4.2, explicitamos os parâmetros adotados durante a experimentação e discutimos as motivações que fundamentaram cada configuração, considerando as características da infraestrutura e da arquitetura analisadas.

5.1 Ambiente de testes

Todos os experimentos foram executados em um cluster *Kubernetes* gerenciado pelo serviço *Google Kubernetes Engine (GKE)*, na versão *1.33.11-gke.1074000*, hospedado na zona *us-central1-a*. A fim de adequar o ambiente ao *modus operandi* do CANM, o cluster foi configurado com dois *pools* de nós: *pool-beta* e *pool-beta-high*, representando, respectivamente, o *low pool* e o *high pool*.

O *pool-beta-high* é composto por instâncias do tipo *n2-custom-2-16384*, equipadas com 2 vCPUs, 16 GiB de memória RAM e 100 GiB de armazenamento SSD, ao custo aproximado de US\$ 0,131014 por hora. Essa configuração é idêntica à utilizada no ambiente de produção da Econodata. Dessa forma, o *pool-beta-high* foi definido como o

¹A Econodata é uma plataforma de inteligência de mercado e prospecção B2B (vendas entre empresas). Utilizando *big data* e inteligência artificial, a plataforma reúne, cruza e atualiza dados públicos de milhões de empresas brasileiras para auxiliar equipes comerciais na identificação de potenciais clientes.

high pool, uma vez que representa a infraestrutura atualmente empregada em produção e cujo custo se deseja otimizar. Por sua vez, o *pool-beta* é composto por instâncias do tipo *e2-custom-2-16384*, mantendo a mesma quantidade de recursos computacionais (2 vCPUs, 16 GiB de memória RAM e 100 GiB de armazenamento SSD), porém ao custo de US\$ 0,09039966 por hora, aproximadamente 31% inferior ao das instâncias presentes no *high pool*. Nesse contexto, o *pool-beta* foi definido como o *low pool*, visto que as máquinas da série E2 apresentam menor custo operacional quando comparadas às da série N2.

Por fim, cabe destacar que os nós dos *pools* descritos anteriormente utilizaram como sistema operacional o Ubuntu versão 24.04.4 LTS (Noble Numbat). Ainda, todos os experimentos foram realizados em um cluster composto por quatro nós, os quais poderiam estar distribuídos entre os diferentes *pools* descritos anteriormente. Essa configuração foi adotada por representar a quantidade mínima de nós necessária para garantir a execução adequada da arquitetura alvo, a qual detalharemos a seguir.

5.2 Arquitetura alvo

No que se refere à arquitetura das aplicações executadas no cluster de experimentação, foi selecionado um subconjunto dos serviços presentes no ambiente de produção da Econodata, com o objetivo de concentrar a avaliação nos microsserviços que recebem o maior volume de requisições. Nesse contexto, foram selecionados sete microsserviços, todos gerenciados por *Deployments* configurados com quatro réplicas de *Pods* – configuração padrão do ambiente de produção.

Essas aplicações implementam servidores HTTP responsáveis por operações com diferentes perfis de utilização de recursos computacionais. Entre as principais atividades realizadas por esse conjunto de microsserviços destacam-se a inserção de dados em bancos de dados, o processamento e transformação de informações, a consulta a bancos de dados relacionais e não relacionais, bem como a renderização e disponibilização de páginas HTML aos usuários finais.

A Tabela 5.1 relaciona os microsserviços considerados na experimentação e as respectivas rotas HTTP empregadas na execução dos testes. Além disso, a Tabela 5.2 apresenta os volumes médios diários de requisições processados por essas rotas no ambiente de produção, diferenciando-se os períodos de dias úteis (segunda a sexta-feira) e finais de semana (sábado e domingo). Os dados apresentados nessas tabelas foram coletados ao longo de um período de 24 dias, compreendido entre 1º de maio de 2026 e 24 de maio de

2026.

Tabela 5.1: Microserviços e rotas mais utilizadas da arquitetura alvo

Microserviço	Rota
ecdt-historico-v2	/obter-nota-qualificacao
ecdt-admin-v2	/tarja-notificacao
ecdt-billing	/api/billing-v3/saldo
ecdt-busca	/searchFiliais
ecdt-crowdsourcing	/api/feedback
ecdt-busca	/checkSessao
ecdt-tags	/tags
ecdt-busca	/searchDadosDetalheEmpresa
ecdt-busca	/obterGrupoEconomico
ecdt-busca	/searchEventoCliente
ecdt-busca	/searchEmailsBuscados
ecdt-busca	/searchFiltroDecisores
ecdt-busca	/quickSearch
ecdt-busca	/searchDadosMapping
ecdt-busca	/searchCompanies
ecdt-historico-v2	/pega-prompts-pesquisas
ecdt-billing	/api/billing-v3/desbloquearEmpresas
ecdt-busca	/obterOrganograma
ecdt-busca	/pesquisarCamposPorCnpj
ecdt-busca	/searchCnpjsBloqueados
ecdt-busca	/searchBairros
ecdt-historico-v2	/salvar-pesquisa-recente

5.3 Caracterização da carga de trabalho

Os dados de carga e de utilização de recursos analisados nesta seção foram fornecidos pela Econodata e correspondem ao seu ambiente de produção — e não ao cluster de testes empregado na avaliação experimental. Com base nas Tabelas 5.1 e 5.2, é perceptível que a carga de trabalho realizada pelas aplicações presentes no cluster de produção varia consideravelmente ao longo da semana. Nesse contexto, ao se comparar os dias úteis com os finais de semana, observa-se que essas aplicações operam sob uma carga diária ~112% maior em dias úteis. Assim, a utilização de recursos do cluster também é afetada por tal dinamicidade. Esse fato se comprova por meio dos dados apresentados nas Figuras 5.1 e 5.2, em que a utilização média diária de vCPU por nó em dias úteis é superior à observada nos finais de semana.

Tabela 5.2: Média diária de requisições por rota — dia útil e fim de semana

Rota	Dia útil	Fim de semana
/obter-nota-qualificacao	271.000	120.792
/tarja-notificacao	45.394	23.624
/api/billing-v3/saldo	40.732	18.917
/searchFiliais	38.279	19.097
/api/feedback	23.222	11.199
/checkSessao	20.547	10.458
/tags	20.138	9.397
/searchDadosDetalheEmpresa	19.612	9.568
/obterGrupoEconomico	19.113	9.238
/searchEventoCliente	18.198	8.909
/searchEmailsBuscados	18.157	8.810
/searchFiltroDecisores	16.526	6.300
/quickSearch	13.107	6.884
/searchDadosMapping	10.937	4.960
/searchCompanies	6.649	2.970
/pega-prompts-pesquisas	4.546	1.920
/api/billing-v3/desbloquearEmpresas	3.476	2.155
/obterOrganograma	3.394	1.438
/pesquisarCamposPorCnpj	3.286	1.505
/searchCnpjsBloqueados	2.940	1.415
/searchBairros	2.915	4.306
/salvar-pesquisa-recente	2.719	1.368
Total	604.887	285.230

Figura 5.1: Utilização média de vCPU por nó em dias úteis

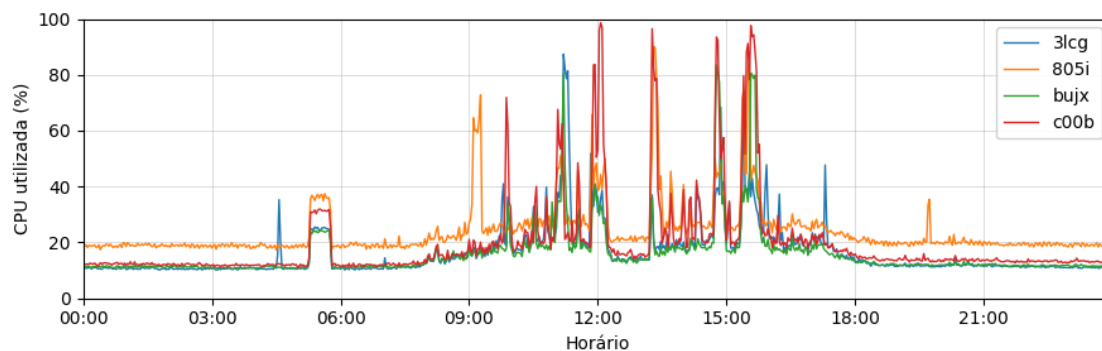
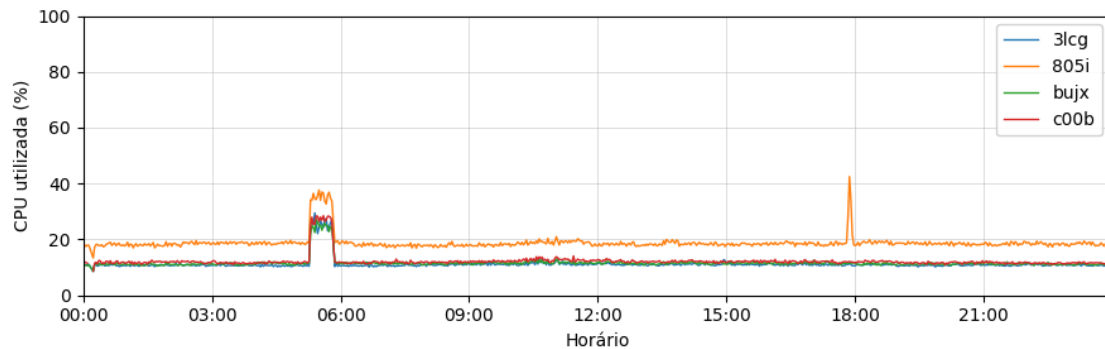


Figura 5.2: Utilização média de vCPU por nó nos finais de semana



Além de apresentar o consumo de vCPU em dias úteis, a Figura 5.1 evidencia um padrão bem definido de utilização desse recurso ao longo do dia. Entre meia-noite e seis horas da manhã, o cluster permanece em um estado de elevada ociosidade, uma vez que o consumo médio de vCPU por nó não ultrapassa 20%. Entretanto, observa-se um leve aumento na utilização por volta das seis horas da manhã, possivelmente associado à execução de processos periódicos da infraestrutura, visto que comportamento semelhante também pode ser observado na Figura 5.2.

A partir das nove horas da manhã, verifica-se um crescimento gradual na demanda por processamento. Esse comportamento pode ser explicado pelo início do horário comercial dos clientes que utilizam a plataforma da Econodata. Nesse contexto, a utilização de vCPU atinge seu primeiro pico por volta do meio-dia. Em seguida, possivelmente ocasionada pelo horário de almoço, observa-se uma redução temporária no consumo do recurso, seguida de um novo aumento próximo das 13 horas. Durante o período da tarde, a utilização mantém-se relativamente estável em níveis superiores aos observados durante a madrugada.

Por fim, a partir das 18 horas, horário geralmente associado ao encerramento do expediente comercial, o consumo de vCPU passa a apresentar uma tendência de queda gradual. Consequentemente, ao longo da noite, o cluster retorna a um estado de elevada ociosidade, semelhante ao observado durante a madrugada.

Em contrapartida, ao analisar a utilização média de memória por nó, não conseguimos extrair nenhuma informação sobre os padrões de carga de trabalho realizados no cluster. As Figuras 5.3 e 5.4 demonstram que, tanto em dias úteis quanto em finais de semana, o consumo de memória apresentou baixa variabilidade ao longo do período analisado, especialmente quando comparado aos padrões observados para a utilização de vCPU.

Figura 5.3: Utilização média de memória por nó em dias úteis

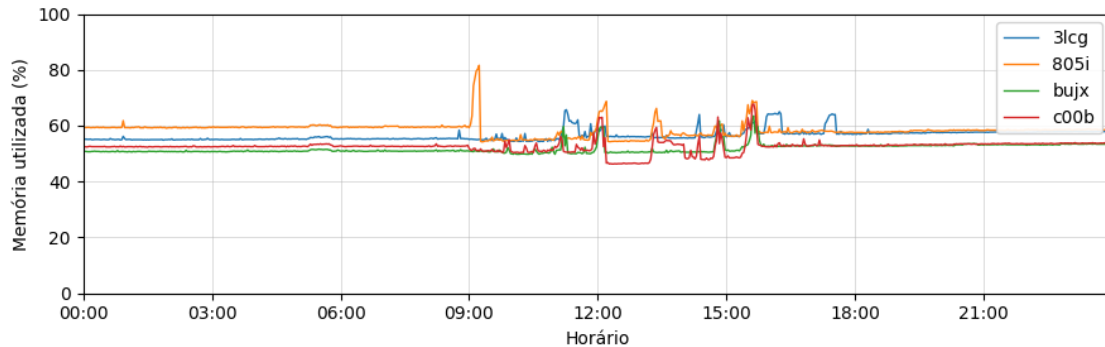
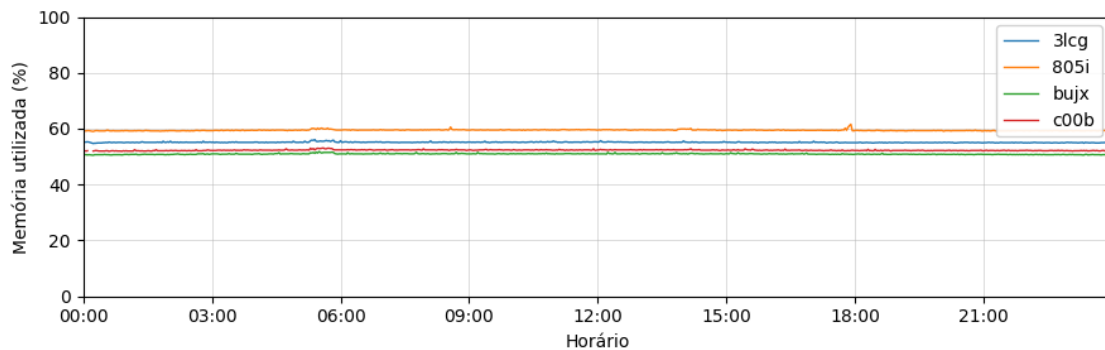


Figura 5.4: Utilização média de memória por nó nos finais de semana



5.4 Reprodução da carga de trabalho

Para representar o padrão de carga detalhado na seção anterior, utilizou-se a ferramenta *k6*, uma solução de código aberto voltada a testes de carga. No *k6*, o comportamento dos usuários é descrito por meio de *scripts*, e a carga é gerada por *Virtual Users* (VUs)². Essa abordagem permite controlar com precisão a concorrência, a distribuição de requisições entre as rotas dos microsserviços e a evolução da carga ao longo do tempo.

Adicionalmente, a carga foi dividida em dois *profiles* distintos, sendo eles: dia útil e fim de semana. Essa separação decorre do fato de que, além de o volume total de requisições variar entre esses períodos, a própria distribuição entre as rotas se altera. Nesse contexto, o profile de dia útil reproduz o padrão diário observado, com pico matinal, vale ao meio-dia, utilização constante no decorrer da tarde e diminuição gradual ao anoitecer. Em contrapartida, o profile de fim de semana reproduz um tráfego constante e sem sobrecarga de consumo. Em ambos os casos, o ciclo diário de 24 horas foi comprimido temporalmente,

²VUs são unidades de execução concorrentes que disparam requisições de forma independente, emulando usuários simultâneos da aplicação

permitindo observar o comportamento de um dia inteiro em um intervalo de execução reduzido, preservando-se a forma relativa da curva de carga. Por conta desse fator, os testes foram realizados ao longo de duas horas, considerando uma escala temporal em que um minuto de execução da simulação representa 12 minutos reais. Optou-se por utilizar essa escala temporal, uma vez que um período contínuo de duas horas é suficiente para reproduzir consideravelmente os padrões de utilização apresentados na Seção 5.3.

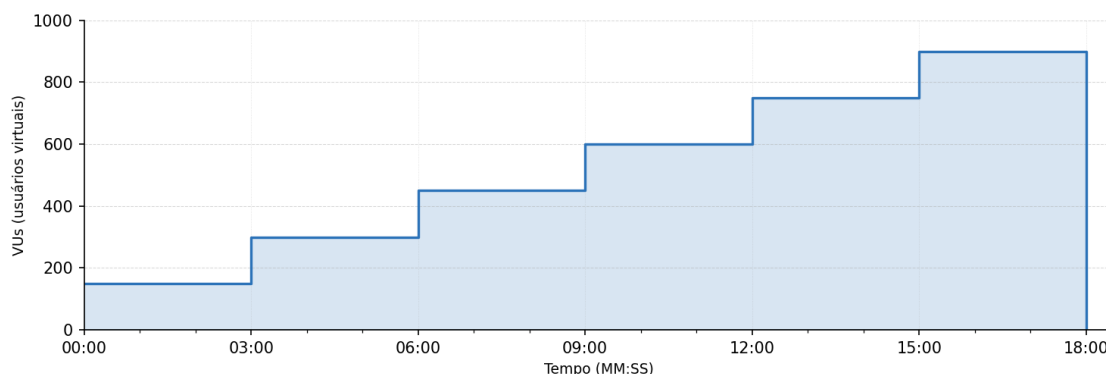
Por fim, com o objetivo de garantir a reprodutibilidade entre execuções, todas as requisições utilizam cargas úteis (*payloads*) fixas. Cada execução foi avaliada quanto a critérios de qualidade de serviço previamente estabelecidos, tais como: latência no percentil 95 (p_{95}), 99 (p_{99}) e a taxa de erros. Nesse contexto, o percentil de latência indica o tempo de resposta abaixo do qual se situa a fração correspondente das requisições: a latência p_{95} , por exemplo, é o valor abaixo do qual 95% das requisições foram atendidas, de forma que apenas 5% apresentaram tempo de resposta superior. Trata-se, portanto, de uma medida da cauda da distribuição — e não do valor máximo observado —, útil para caracterizar o comportamento do sistema sob os piores tempos de resposta.

5.4.1 Intensidade da carga

A intensidade de carga de cada *profile* foi calibrada a partir de um limiar de degradação identificado empiricamente por meio de testes de carga incremental no cluster de experimentação. Esses testes foram executados com todas as máquinas presentes no *high pool* ao longo de um período de 18 minutos, com uma carga inicial de 150 usuários virtuais (VUs), incrementada gradualmente em 150 VUs a cada três minutos – atingindo um máximo de 900 VUs simultâneos. Além disso, por se tratar de experimentos de curta duração e baixo custo operacional, a experimentação foi executada três vezes a fim de reduzir a influência de valores atípicos nos resultados obtidos. Nesse contexto, a Figura 5.5 apresenta o padrão de carga adotado durante a avaliação experimental.

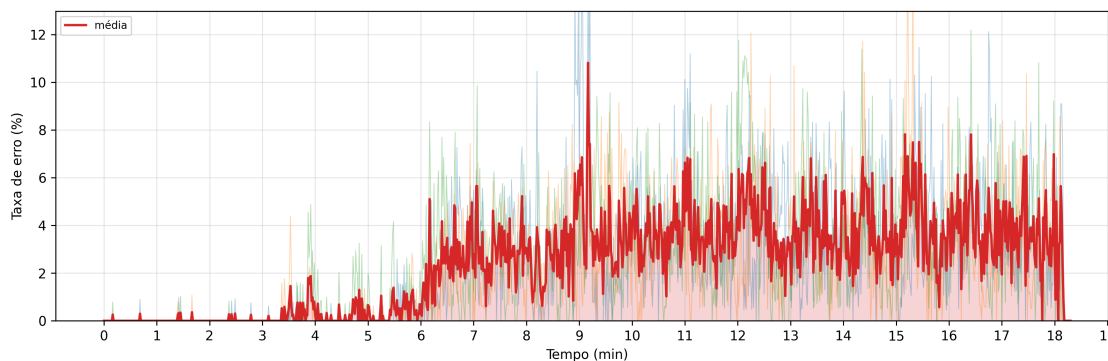
Nos testes de carga, considera-se um *erro* toda requisição que não é atendida com sucesso pela aplicação — tipicamente respostas com código de status HTTP de erro (por exemplo, da classe 5xx) ou falhas de conexão e *timeouts*. Neste contexto, tais erros decorrem da incapacidade da infraestrutura de processar o volume de requisições simultâneas dentro dos limites de tempo esperados, e não de defeitos funcionais da aplicação. Desse modo, com base na Figura 5.6, observa-se que o cluster operou praticamente sem erros até 150 VUs e, no degrau de 300 VUs, apresentou apenas picos intermitentes de erro (entre

Figura 5.5: Carga incremental de VUs



1% e 2%), com retorno a zero entre eles — indicando que o sistema ainda era capaz de drenar a fila de requisições. Contudo, a partir de 450 usuários virtuais (minuto 6), a taxa de erro torna-se sustentada e cresce consideravelmente, sem retornar a zero, demonstrando uma limitação da infraestrutura e da arquitetura para atender esse volume de requisições simultâneas. Portanto, adotou-se 300 VUs como teto operacional (100% do limiar), por corresponder ao último degrau em que a degradação ainda é transitória, antes do ponto de saturação.

Figura 5.6: Taxas de erros ao longo dos testes de carga



Adicionalmente, a Figura 5.7 evidencia que o aumento do número de usuários virtuais para além de 300 não se traduz em maior utilização média de vCPU do cluster. Esse comportamento revela uma limitação de escalabilidade das aplicações executadas nessa infraestrutura, uma vez que a partir desse ponto, elas deixam de ser capazes de atender ao volume de requisições demandado.

Vale notar que a Figura 5.7 apresenta a utilização média do cluster, ao passo que a Figura 5.1 evidencia que nós individuais atingem 100% nos picos. O fato de a média permanecer em torno de 50% mesmo com a vazão estagnada a partir de 300 VUs (Figura 5.8) indica que o gargalo não decorre da capacidade agregada de vCPU, e sim da distribuição desigual da carga entre os nós.

Figura 5.7: Utilização vCPU média do cluster ao longo dos testes de carga

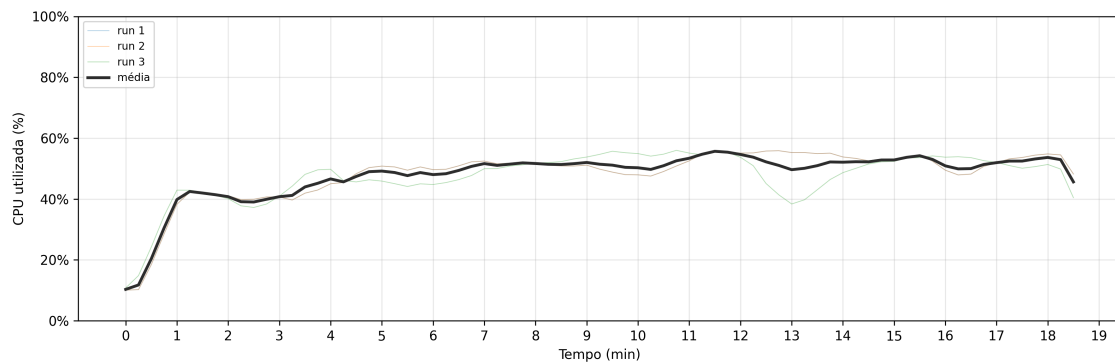
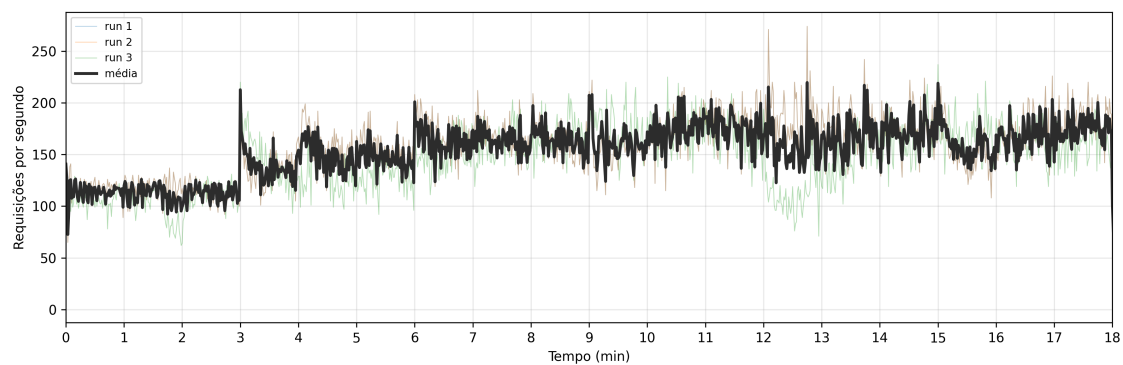


Figura 5.8: Vazão de requisições média do cluster ao longo dos testes de carga



Logo, a partir desse limiar, os dois *profiles* foram dimensionados de forma distinta. O *profile* de dia útil parte de um patamar noturno de 15 VUs, atingindo o teto de 300 VUs nos picos matinal e vespertino, instantes que reproduzem os momentos reais em que múltiplos nós individuais alcançam cerca de 100% de utilização de vCPU. Já o *profile* de fim de semana é mantido deliberadamente muito abaixo do limiar, operando em um patamar constante de 15 VUs, de modo a reproduzir um tráfego plano e sem saturação. Ambos os perfis de carga apresentam um aumento breve e abrupto no início da execução, reproduzindo o comportamento observado nas Figuras 5.1 e 5.2 – o qual simula o comportamento associado à execução de tarefas periódicas de curta duração.

Essa diferença entre *profiles* reflete a própria assimetria de volume observada em produção, em que, em dia útil, concentram-se, em média, 604.887 requisições por dia (~7 req/s), contra 285.230 requisições por dia (~3,3 req/s) no fim de semana. Ainda, cabe destacar que o número de VUs não corresponde linearmente ao volume de requisições de produção. Em vez disso, cada estágio foi calibrado como uma fração do limiar de degradação estabelecido ($300 \text{ VUs} \equiv 100\%$), posicionando a carga em relação ao ponto de saturação identificado e preservando o padrão temporal observado em produção. Esse mapeamento empírico e não-linear entre o número de VUs alvo e o percentual do limiar é

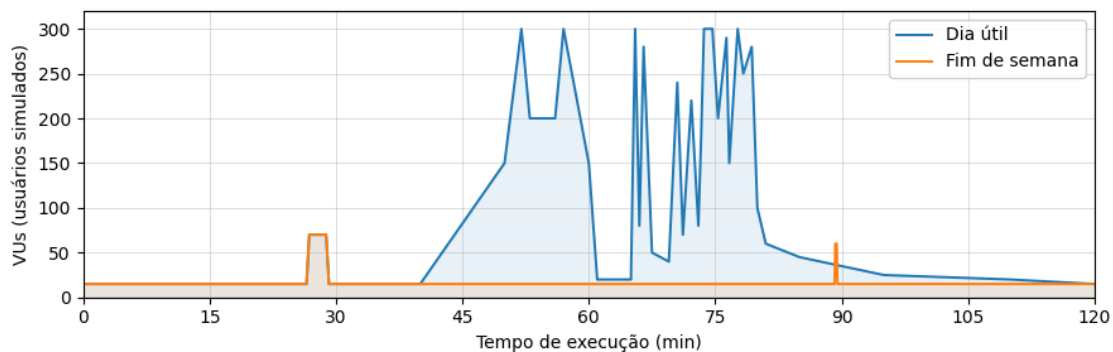
explicitado pela Tabela 5.3.

Tabela 5.3: Relação entre a quantidade de usuários virtuais e o percentual do limiar configurado

VU alvo no teste	% do limiar
15	5%
25–40	8–13%
50–70	17–23%
100–180	33–60%
200–280	67–93%
300	100%

Nesse contexto, a quantidade de usuários virtuais (VUs) utilizados em cada instante de tempo para cada perfil de carga é apresentada na figura seguinte.

Figura 5.9: VUs por perfil de carga



5.4.2 Cenários de experimentação

Os experimentos foram organizados segundo uma lógica de comparação entre cenários de referência, sem o CANM, e o cenário com o mecanismo ativo. Os dois cenários de referência estabelecem os limites entre os quais o resultado do CANM deve se situar. Idealmente, o mecanismo deve aproximar o custo do cluster ao do cenário mais econômico, sem degradar excessivamente o desempenho relativo ao cenário de maior capacidade. Abaixo, descrevemos concretamente as situações cobertas pela experimentação proposta.

- **Cenários de referência (*baseline*), com o CANM desativado:**
 - **Cluster homogêneo no *high pool*.** Todos os nós pertencem ao *pool* de maior capacidade e custo. Estabelece o teto de desempenho e o custo máximo de

operação.

- **Cluster homogêneo no *low pool*.** Todos os nós pertencem ao *pool* de menor capacidade e custo. Estabelece o piso de custo e a capacidade mínima disponível para a aplicação.
- **Cenário sob avaliação, com o CANM ativado:**
 - **Cluster iniciado no *high pool* com o CANM ativo.** A partir de uma configuração homogênea no *high pool*, o mecanismo passa a atuar livremente. Avalia-se se o CANM reduz o custo, aproximando-se do *baseline* do *low pool*, mas mantendo o desempenho dentro de limites aceitáveis em relação ao *baseline* do *high pool*.

Cabe esclarecer que, enquanto os cenários de referência mantêm o cluster *homogêneo* durante toda a execução — com todos os nós pertencentes a um único *pool* —, a atuação do CANM torna o cluster *heterogêneo*, ou seja, composto simultaneamente por nós de ambos os *pools* ao longo das migrações. É justamente essa heterogeneidade transitória que permite ao mecanismo ajustar a composição do cluster à demanda sem alterar o número total de máquinas.

Cabe justificar, ainda, a escolha desses cenários de referência como base de comparação, em vez de um confronto empírico direto com os trabalhos relacionados discutidos no Capítulo 3. As abordagens propostas em (CHUNG; PARK; GANGER, 2018) e (ZHONG; BUYYA, 2020) pressupõem a estimativa do tempo de execução das tarefas para a tomada de decisão e não dispõem de implementações abertas e reproduzíveis sobre a mesma infraestrutura, o que inviabiliza uma comparação experimental justa no escopo deste trabalho. Além disso, esses mecanismos — assim como o autoescalador de cluster nativo do *Kubernetes* — reduzem custos essencialmente pela adição e pela remoção de máquinas, estratégia estruturalmente distinta da migração entre *pools* realizada pelo CANM. Os cenários de referência homogêneos (*low* e *high pool*) capturam justamente os limites dessa estratégia: o custo de operar sempre na configuração mais econômica e o desempenho de operar sempre na de maior capacidade. Situar o resultado do CANM entre esses dois extremos permite, assim, avaliar diretamente o compromisso entre custo e desempenho que constitui o objeto deste trabalho.

5.5 Configuração experimental do CANM

Nesta seção, são apresentados os principais parâmetros do CANM (Tabela 4.2) utilizados durante a execução dos cenários experimentais descritos anteriormente. Além disso, são discutidas as motivações que fundamentaram a escolha desses parâmetros, com base nos dados obtidos a partir dos testes de carga apresentados na Subseção 5.4.1.

Cabe enfatizar, de antemão, que os valores adotados nesta seção não são universais: foram derivados dos dados de utilização do ambiente de produção da aplicação avaliada e das características do cluster de testes. Dessa forma, tais parâmetros dependem diretamente do perfil de carga da aplicação e da estrutura do cluster — como o número e a capacidade dos nós, as famílias de instâncias e o comportamento de reagendamento dos *Pods* —, de modo que sua transposição para outros contextos exige recalibração. Reforça-se, portanto, que a parametrização do CANM é sensível à aplicação e ao ambiente em que opera.

5.5.1 Parâmetros de decisão de migração

Para os limiares de decisão, utilizamos `LOW_SCORE_THRESHOLD` igual a 0,25 e `HIGH_SCORE_THRESHOLD` igual a 0,50. Essa decisão baseou-se nos dados de utilização média de vCPU extraídos do ambiente de produção (Figura 5.1), em que se observa que, fora do horário comercial, a utilização dos nós mantém-se em um patamar baixo — tipicamente abaixo de 25% —, caracterizando um regime de ociosidade. Nesses períodos, um nó é considerado subutilizado e candidato à migração para o *pool* de menor custo, o que motivou a definição de `LOW_SCORE_THRESHOLD` em 0,25.

Por outro lado, os períodos de maior demanda concentram-se no horário comercial, nos quais a utilização ultrapassa recorrentemente os 50% e tende a escalar em direção a picos próximos de 100%. Dessa forma, a ultrapassagem do patamar de 50% tende a marcar o início desse regime de demanda elevada, o que motivou a definição de `HIGH_SCORE_THRESHOLD` em 0,50. Além disso, esse valor permite reagir ao aumento de carga antes que o nó atinja a saturação.

Por fim, vale destacar que, como o cálculo da pontuação considerou exclusivamente a utilização de CPU (`MEMORY_WEIGHT` igual a 0), os limiares de 0,25 e 0,50 correspondem diretamente a 25% e 50% de utilização de CPU observados na figura.

Quanto aos pesos das métricas, foram definidos os valores `CPU_WEIGHT` igual a 1 e `MEMORY_WEIGHT` igual a 0, de modo que a pontuação dos nós considerasse exclusi-

vamente a utilização de CPU. Essa decisão foi fundamentada nos dados observados nas Figuras 5.1 e 5.3, em que, em períodos de alta demanda, o consumo de vCPU no cluster cresceu enquanto o de memória ficou estável.

Por fim, os experimentos foram conduzidos exclusivamente com a política de migração `prioritizeCost`, visto que este trabalho tem como foco a otimização dos custos operacionais. Os resultados obtidos são apresentados e discutidos no Capítulo 6.

5.5.2 Parâmetros de temporização

Para o intervalo entre ciclos de operação, utilizamos `CHECK_INTERVAL` igual a 1 minuto, valor padrão do mecanismo. Para as janelas de avaliação, utilizamos `LOW_POOL_TIME_WINDOW_EVAL` igual a 3 minutos e `HIGH_POOL_TIME_WINDOW_EVAL` igual a 10 minutos. O objetivo dessa escolha é garantir assimetria de avaliação entre os *pools*, de modo que a sobrecarga no *low pool* seja detectada rapidamente — por meio de uma janela curta, que reage prontamente a aumentos de demanda e prioriza a manutenção do desempenho —, enquanto a subutilização no *high pool* é avaliada sobre uma janela mais longa, tornando a decisão de redução de custos mais conservadora e evitando que flutuações transitórias de utilização provoquem a migração desnecessária de nós de maior custo.

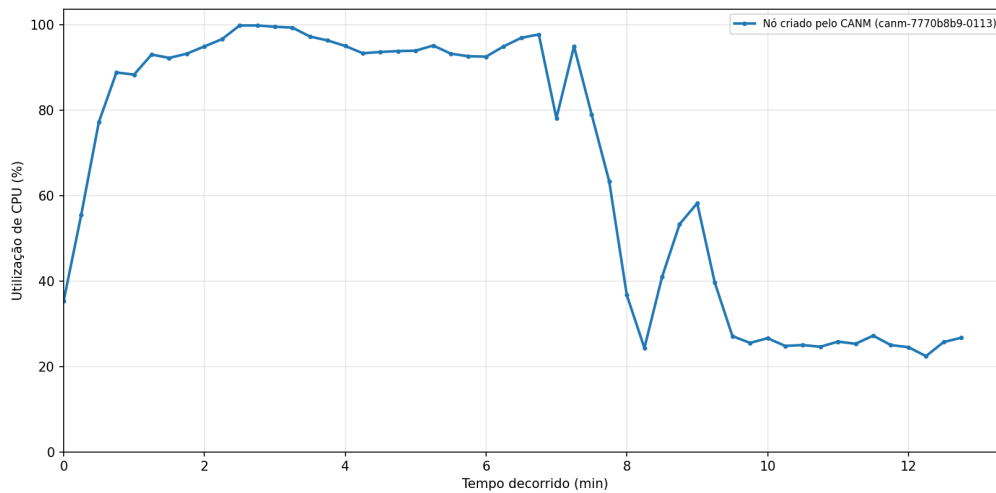
Ademais, vale destacar que os valores foram definidos como um ponto de partida razoável para materializar a assimetria de avaliação. Uma análise sistemática de sensibilidade desses parâmetros foge ao escopo deste trabalho e é deixada como trabalho futuro.

5.5.3 Parâmetros de espera de nós recém-criados

Para os períodos de imunidade de nós recém-criados, utilizamos `HIGH_NODE_COOL_DOWN` igual a 22 minutos e `LOW_NODE_COOL_DOWN` igual a 15 minutos. Esses valores foram derivados a partir de uma migração realizada manualmente, na qual observamos que o tempo necessário para que um nó recém-criado estabilize sua utilização de recursos é de aproximadamente 10 minutos no pior caso, conforme os dados de estabilização apresentados na Figura 5.10. Com base nessa observação, definimos o *cooldown* de cada *pool* como a soma de três componentes: o tempo de estabilização do nó (aproximada-

mente 10 minutos), a janela de avaliação do respectivo *pool* (10 minutos para o *high* e 3 minutos para o *low*) e uma margem conservadora adicional, destinada a absorver eventuais *outliers*. Cabe destacar, entretanto, que o tempo de estabilização depende diretamente da carga de trabalho específica de cada cluster, e que a margem adicional foi definida de forma heurística, não decorrendo de uma derivação formal.

Figura 5.10: Estabilização da utilização de CPU de nós recém-criados pelo CANM



6 ANÁLISE DOS RESULTADOS

Neste capítulo, são apresentados os resultados da avaliação experimental dos cenários descritos na Subseção 5.4.2. Inicialmente, são detalhados os dados observados nos cenários de referência, nos quais o CANM não realiza qualquer intervenção durante a execução dos experimentos. Esses cenários estabelecem os limites entre os quais o resultado do mecanismo proposto deve se situar.

Em seguida, são discutidos os resultados observados nos cenários em que o CANM opera livremente, permitindo avaliar seu impacto sobre as métricas consideradas neste trabalho.

Cada cenário é analisado em profundidade em sua respectiva seção, na qual são apresentados gráficos contendo os valores médios das execuções realizadas. Ao todo, são utilizados cinco tipos de gráficos, cuja semântica e propósito são descritos a seguir:

- **Vazão:** mostra a quantidade de requisições processadas por segundo (req/s) ao longo do tempo (em minutos). Cada execução individual é traçada em segundo plano e a curva em destaque representa a média entre elas, permitindo visualizar a capacidade de processamento do sistema em cada fase da carga.
- **Utilização de vCPU do cluster:** apresenta a porcentagem média de utilização de vCPU do cluster ao longo do tempo. Sua exibição restringe-se aos cenários em que o CANM está desativado, uma vez que, durante o processo de migração, a criação e a remoção contínuas de nós alteram o conjunto de séries ao longo do tempo, comprometendo a legibilidade e a análise do gráfico.
- **Latência:** apresenta o tempo de resposta das requisições (em microssegundos) ao longo do tempo, decomposto nos percentis $p50$ (mediana), $p95$ e $p99$. Cada curva corresponde à média das execuções para o respectivo percentil, evidenciando tanto o comportamento típico quanto o da cauda sob estresse.
- **Taxa de erro:** exibe a quantidade total de falhas ao longo do tempo. A curva em destaque representa a média das execuções, identificando os instantes em que o sistema deixou de atender requisições com sucesso e a intensidade dessas falhas.
- **Migrações:** mostra a composição do cluster ao longo do tempo, indicando a quantidade de nós em cada *pool* e demarcando o início e o término de cada migração, bem como o *score* que a motivou. Sua exibição restringe-se aos resultados obtidos com o CANM ativado, uma vez que somente nesses cenários ocorrem migrações entre os

pools.

Por fim, consolida-se em uma seção própria a análise de custo de provisionamento das máquinas utilizadas (Seção 5.1), comparando o custo obtido com o CANM ao dos cenários de referência — resultado central deste trabalho.

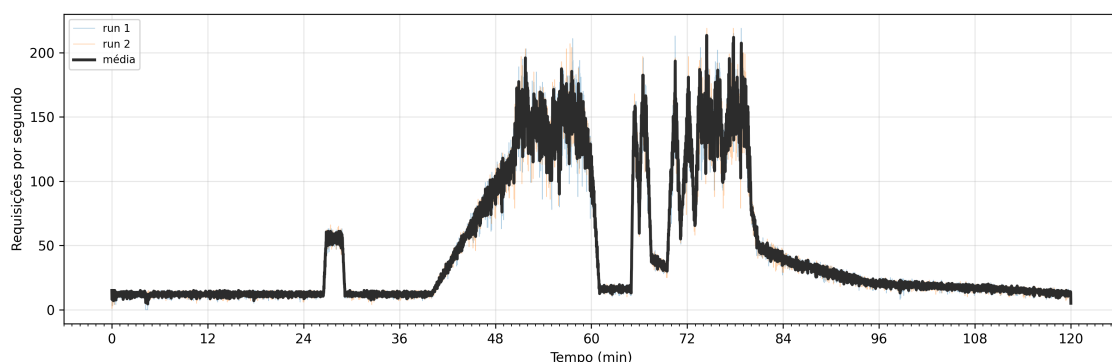
6.1 Cenários de referência: um achado contraintuitivo

Os cenários de referência correspondem à execução da carga sobre clusters homogêneos, mantendo as quatro máquinas fixas no *low pool* ou no *high pool* durante todo o experimento, sem qualquer atuação do CANM. Eles estabelecem os limites entre os quais o resultado do mecanismo deve se situar: o *low pool* representa o piso de custo, enquanto o *high pool* representa a configuração de maior capacidade computacional. A análise a seguir revela, contudo, um resultado contraintuitivo: sob a demanda de dia útil, o *pool* de maior desempenho por núcleo apresentou desempenho *inferior* ao do *pool* mais econômico. As causas desse comportamento são discutidas ao longo desta seção.

6.1.1 Profile dia útil

Inicia-se pelo *low pool*. A distribuição das requisições ao longo do tempo respeitou o perfil de carga definido (Figura 6.1), totalizando 328.616 requisições processadas nesse *profile*.

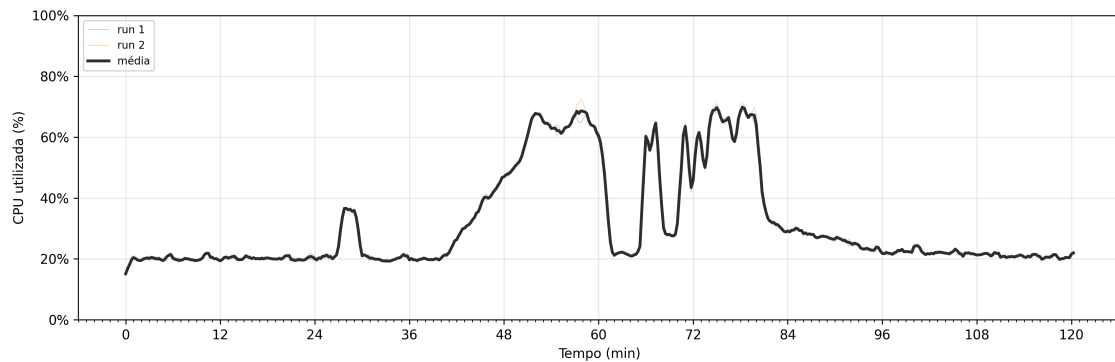
Figura 6.1: Vazão média de requisições do *profile* de dia útil no *low pool*



A utilização média de vCPU do cluster (Figura 6.2) acompanhou de forma clara o comportamento da vazão (Figura 6.1). Os períodos de maior volume de requisições coincidem com os de maior utilização de CPU, enquanto nos intervalos de ociosidade a

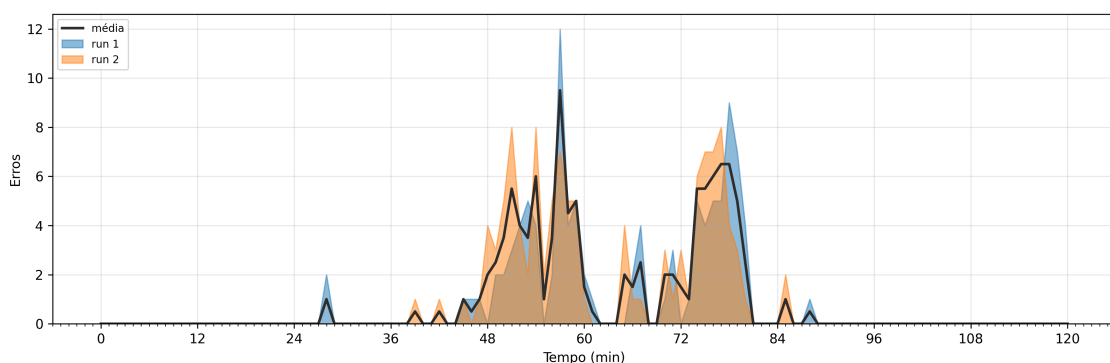
utilização permanece baixa. Contudo, cabe ressaltar que a utilização média do cluster não atingiu 100% em nenhum momento. Isso decorre da metodologia adotada nos testes, na qual a intensidade da carga é definida com base no limiar de degradação do serviço, e não na saturação da CPU, uma vez que, conforme discutido na Seção 5.4, a CPU não constitui o fator limitante do sistema.

Figura 6.2: Utilização média de vCPU do cluster no *profile* de dia útil no *low pool*



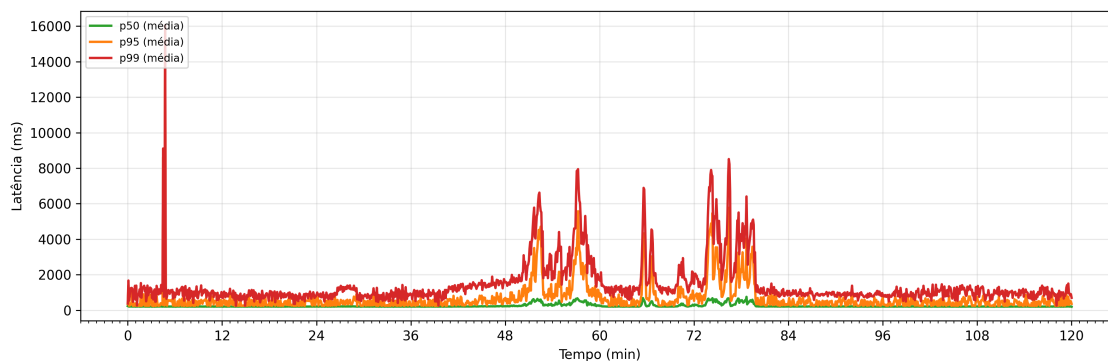
Ainda, observa-se uma relação direta entre os momentos de aumento da demanda e a degradação do sistema. Tanto a latência (Figura 6.4) quanto a taxa de erros (Figura 6.3) elevam-se justamente nos picos de carga, retornando a patamares baixos assim que a demanda diminui. Cabe ressaltar o pico isolado observado no início da execução, o qual é interpretado como um valor fora da curva, não associado a um aumento de demanda. Acredita-se que, com a realização de um número maior de execuções, esse efeito pontual tenderia a ser diluído na média.

Figura 6.3: Erros em requisições do *profile* de dia útil no *low pool*



Por fim, é perceptível que ainda que haja capacidade de CPU disponível, o serviço passa a se degradar antes que esse recurso seja plenamente utilizado. Levanta-se, como hipótese, que esse comportamento esteja relacionado à própria arquitetura da aplicação e à natureza das requisições atendidas, cujos *endpoints* dependem de operações de entrada e saída (*I/O*) com outros sistemas. Dessa forma, embora a CPU seja utilizada para processar

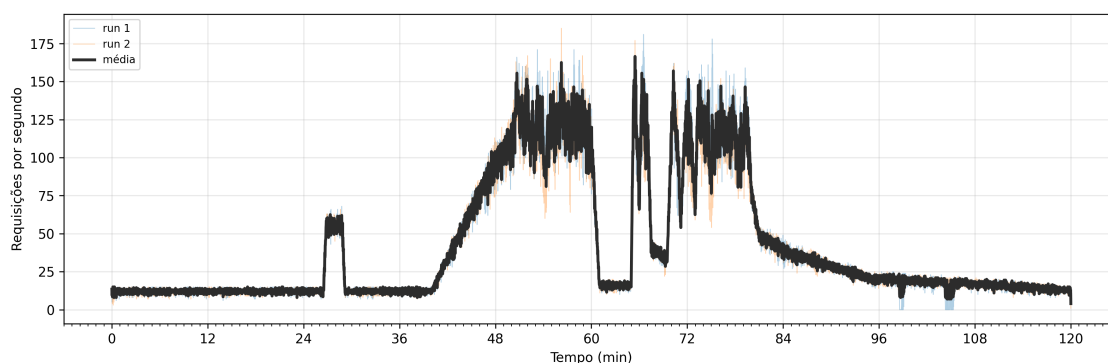
Figura 6.4: Latência de requisições do *profile* de dia útil no *low pool*



a fila de requisições, o fator que efetivamente limita o desempenho não é o processamento local, mas sim a espera pelas operações dependentes de sistemas externos.

Executou-se, em seguida, o mesmo *profile* sobre o *high pool*. A Figura 6.5 apresenta a vazão média de requisições nesse cenário, cuja distribuição ao longo do tempo segue o perfil de carga definido na Subseção 5.4.1. A quantidade total de requisições processadas foi de 298.460, valor aproximadamente 9,2% inferior ao registrado no *low pool* (328.616).

Figura 6.5: Vazão média de requisições do *profile* de dia útil no *high pool*



Esse resultado é, à primeira vista, contraintuitivo, uma vez que as máquinas do *high pool* (n2) possuem maior desempenho por núcleo que as do *low pool* (e2). A utilização média de vCPU do cluster (Figura 6.6) auxilia na interpretação desse comportamento, pois, mesmo durante os picos de carga, a utilização média manteve-se abaixo de 50%, com pico de apenas 45,6%. Isso indica que as máquinas de maior desempenho realizaram um trabalho semelhante exigindo menos esforço computacional.

Entretanto, mesmo exigindo menos esforço computacional, o *high pool* apresentou uma quantidade de erros (Figura 6.7) e uma latência (Figura 6.8) superiores às registradas no *low pool*. Esse contraste motiva a hipótese discutida a seguir.

Levanta-se, como hipótese, que justamente esse maior desempenho por núcleo tenha contribuído para a degradação observada. Como o fator limitante do sistema não é a

Figura 6.6: Utilização média de vCPU do cluster no *profile* de dia útil no *high pool*

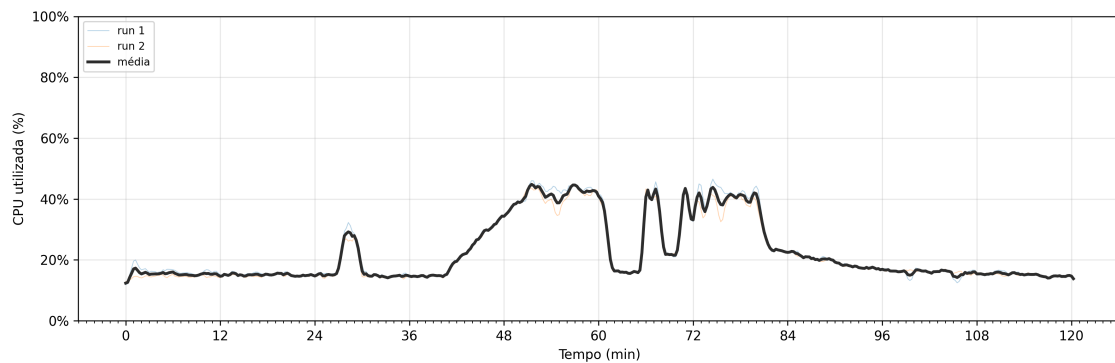
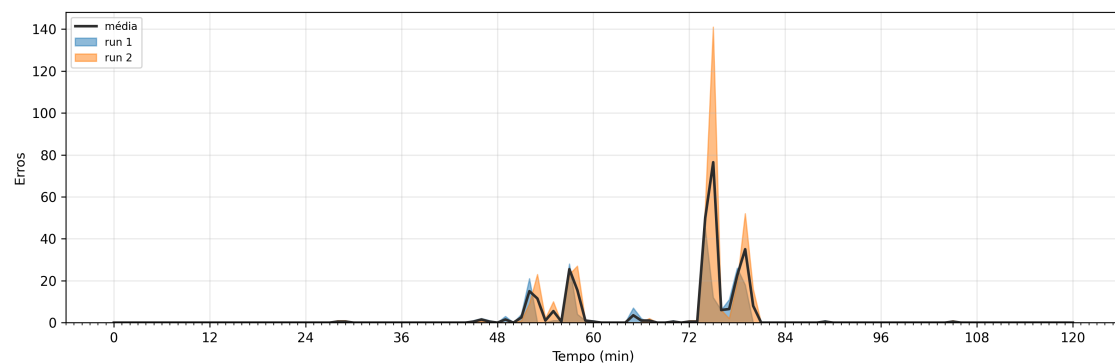


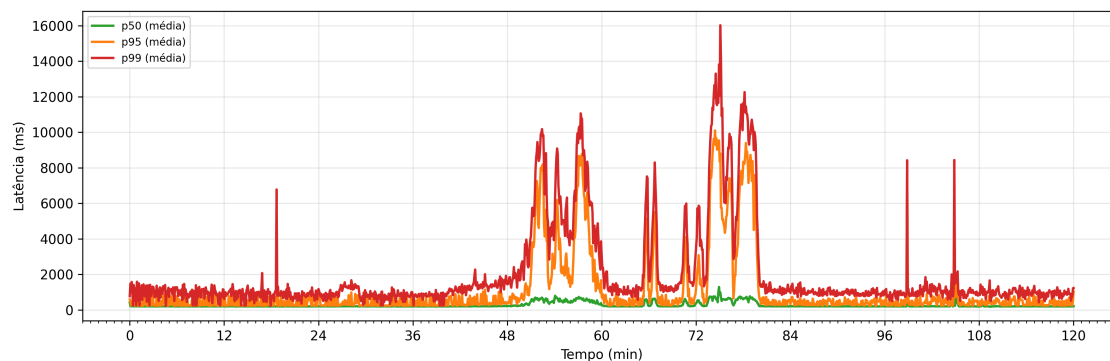
Figura 6.7: Erros em requisições do *profile* de dia útil no *high pool*



CPU, mas sim as operações de entrada e saída (*I/O*) com sistemas externos, uma CPU mais rápida tende a retirar e despachar requisições da fila em maior ritmo. Além disso, a maioria dos microserviços é implementada em *Node.js*, cujo modelo de execução baseia-se em um *loop* de eventos de *thread* única e E/S não-bloqueante. Nesse modelo, ao processar uma requisição que realiza uma chamada assíncrona, a aplicação não aguarda sua conclusão e passa imediatamente a processar outras requisições. Dessa forma, ao despachar requisições mais rapidamente, a máquina de maior desempenho eleva o número de operações de E/S simultaneamente em andamento, intensificando a pressão sobre os sistemas externos, o que se reflete em maior latência e maior taxa de erros. Em outras palavras, a maior capacidade de processamento não se traduz em ganho de desempenho, pois apenas acentua a limitação imposta pela dependência de E/S das aplicações.

A Tabela 6.1 consolida as métricas médias dos dois cenários de referência no *profile* de dia útil, evidenciando o contraste discutido.

Vale observar que, na mediana (p50), o *high pool* apresentou latência inferior à do *low pool* (218,7 ms contra 241,0 ms). Esse comportamento reforça a interpretação anterior, pois, descontado o efeito da limitação de E/S – que se manifesta principalmente na cauda da distribuição (p95 e p99) – as máquinas de maior desempenho por núcleo de

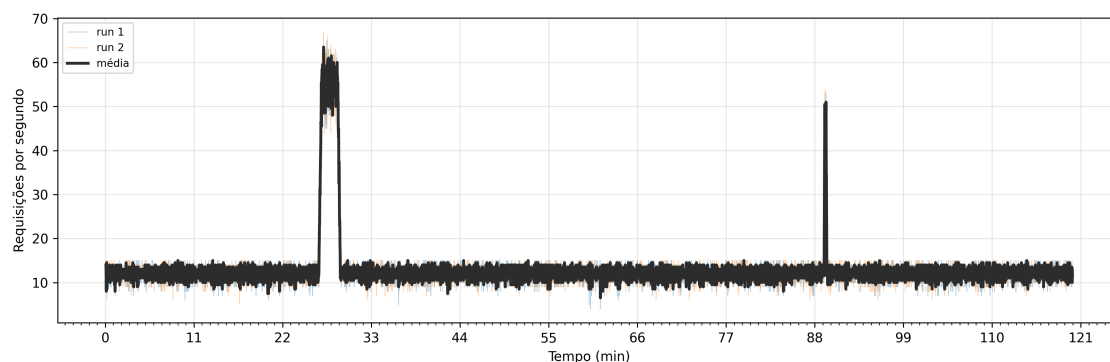
Figura 6.8: Latência de requisições do *profile* de dia útil no *high pool*Tabela 6.1: Métricas médias dos cenários de referência no *profile* de dia útil

Métrica	<i>Low pool</i>	<i>High pool</i>
Requisições	328.616	298.460
Erros (média total)	108 (0,0 %)	296 (0,1 %)
Vazão	45,6 req/s	41,4 req/s
Latência média	438,3 ms	583,4 ms
Latência p50	241,0 ms	218,7 ms
Latência p95	1287,6 ms	1792,5 ms
Latência p99	3404,8 ms	7409,1 ms

fato entregaram um tempo de resposta menor para a maior parte das requisições.

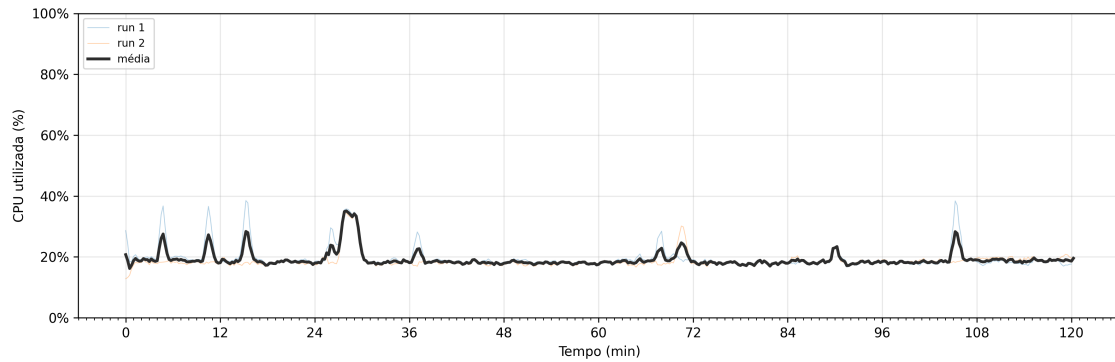
6.1.2 Profile fim de semana

No *profile* de fim de semana, ambos os *pools* foram submetidos ao padrão de carga definido na Subseção 5.4.1, caracterizado por uma utilização baixa e constante, interrompida apenas por picos esporádicos. Inicia-se novamente pelo *low pool* (Figura 6.9).

Figura 6.9: Vazão média de requisições do *profile* de fim de semana no *low pool*

A baixa demanda evidencia-se na utilização média de vCPU do cluster (Figura 6.10), que permanece reduzida durante a maior parte do tempo, mantendo o cluster predomi-

Figura 6.10: Utilização média de vCPU do cluster no *profile* de fim de semana no *low pool*



nantemente em estado ocioso. A quantidade média de erros (Figura 6.11) foi desprezível, com apenas 6 falhas no total. A latência (Figura 6.12), por sua vez, manteve-se estável ao longo de toda a execução, à exceção de um único *outlier* pontual identificado no gráfico. Contudo, o padrão de carga dos instantes vizinhos indica tratar-se de um caso isolado, sem reflexo no comportamento geral do sistema.

Figura 6.11: Erros em requisições do *profile* de fim de semana no *low pool*

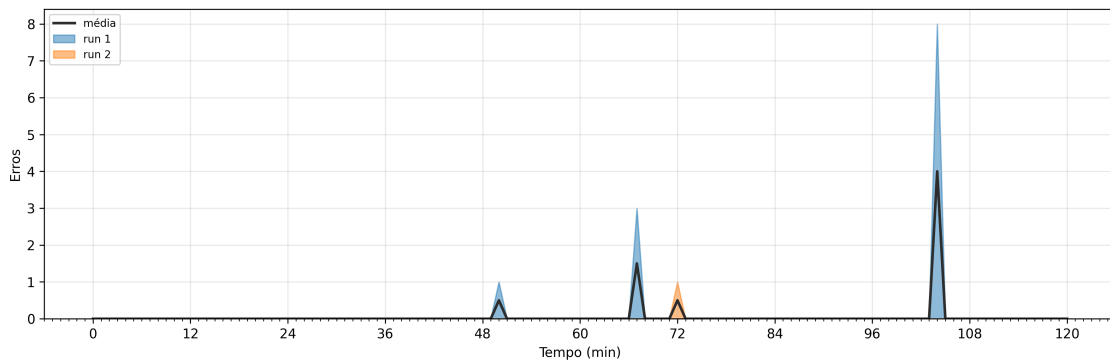
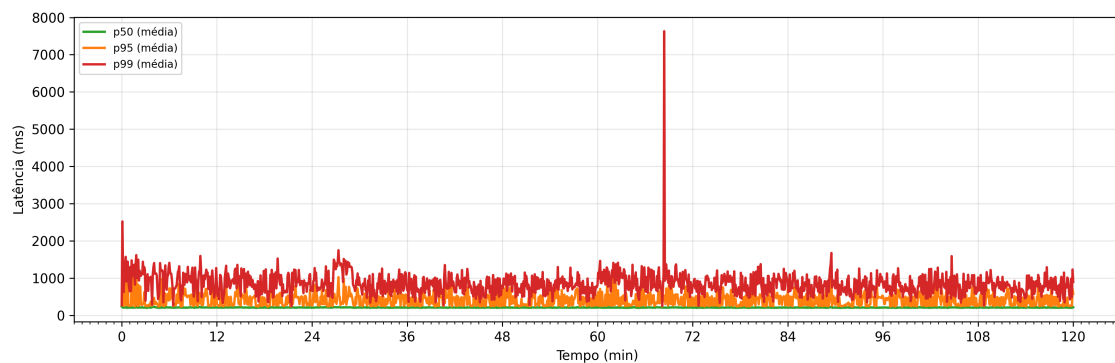


Figura 6.12: Latência de requisições do *profile* de fim de semana no *low pool*



O mesmo *profile* foi então executado sobre o *high pool* (Figura 6.13). É perceptível que os nós permanecem em estado de ociosidade durante quase toda a experimentação. Esse comportamento configura um cenário ideal para a atuação do CANM na otimização

de custos, uma vez que possibilita a migração dos nós do *high pool* para instâncias mais baratas do *low pool*.

Figura 6.13: Vazão média de requisições do *profile* de fim de semana no *high pool*

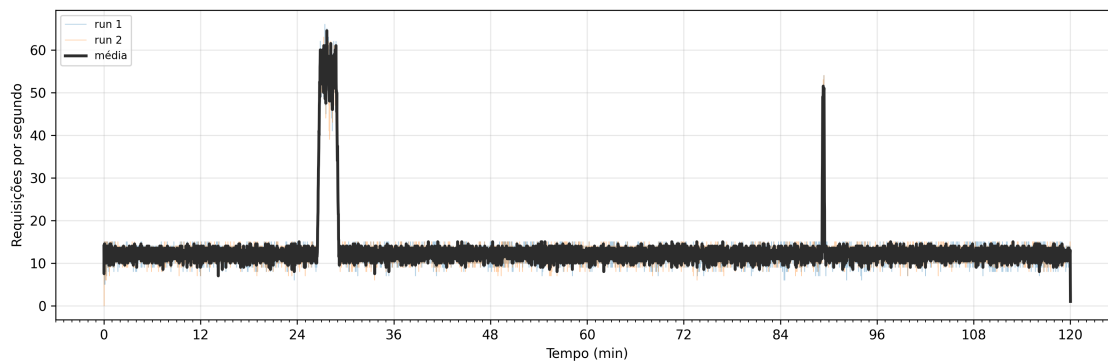


Figura 6.14: Utilização média de vCPU do cluster no *profile* de fim de semana no *high pool*

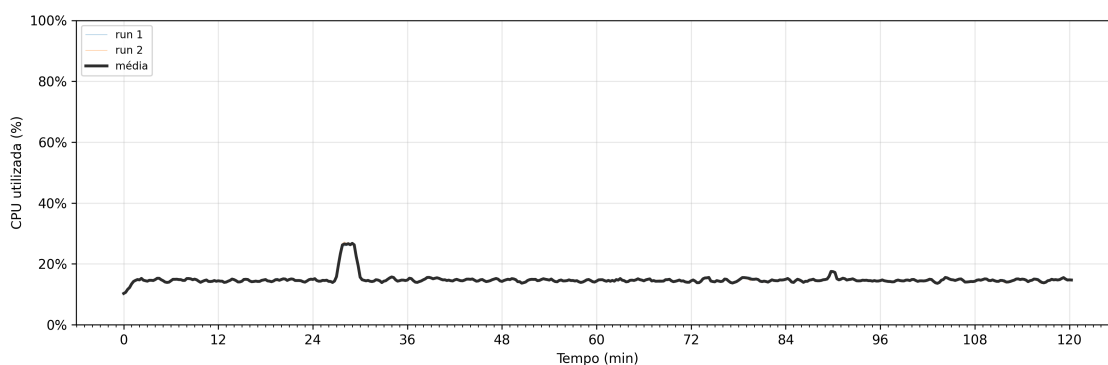
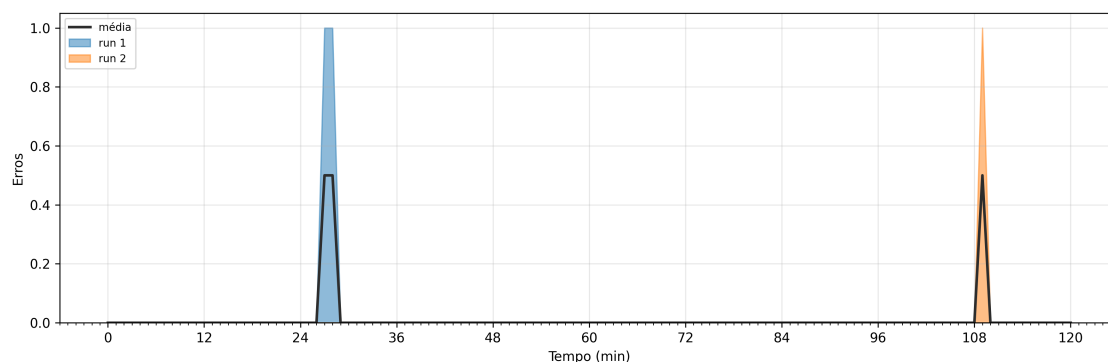
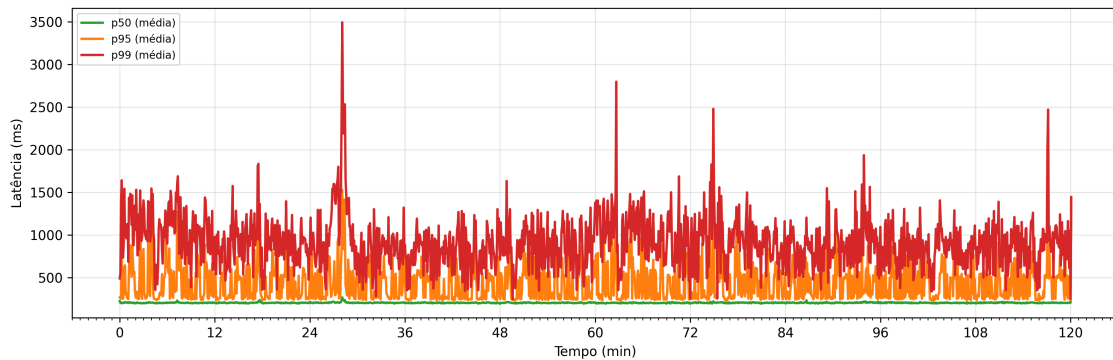


Figura 6.15: Erros em requisições do *profile* de fim de semana no *high pool*



Diferentemente do *profile* de dia útil, o baixo volume de tráfego do fim de semana não expôs a limitação de E/S identificada anteriormente. Em consequência, as métricas coletadas no *high pool* foram muito próximas das observadas no *low pool* no mesmo *profile* (Tabela 6.2), tanto em quantidade de erros quanto em latência. Isso indica que, em períodos de baixa utilização, a máquina de maior desempenho apresenta resultado semelhante ao da

Figura 6.16: Latência de requisições do *profile* de fim de semana no *high pool*

máquina mais econômica, constituindo um forte indício de que a migração realizada pelo CANM nesses cenários é adequada, reduzindo o custo sem prejuízo de desempenho.

Tabela 6.2: Métricas médias dos cenários de referência no *profile* de fim de semana

Métrica	<i>Low pool</i>	<i>High pool</i>
Requisições	93.572	93.510
Erros (média total)	6 (0,0 %)	2 (0,0 %)
Vazão	13,0 req/s	13,0 req/s
Latência média	243,6 ms	244,6 ms
Latência p50	208,6 ms	204,6 ms
Latência p95	312,5 ms	325,3 ms
Latência p99	1119,0 ms	1282,0 ms

6.2 O CANM em operação

6.2.1 Profile dia útil

Assim como nas execuções do *profile* de dia útil realizadas no *low pool* e no *high pool*, a distribuição das requisições ao longo do tempo nos testes com o CANM ativo respeitou o padrão definido na Subseção 5.4.1. Entretanto, nesse cenário, o total de requisições processadas foi de 296.332, o que representa uma redução de aproximadamente 10% na vazão em relação ao mesmo *profile* executado no *low pool*, valor praticamente equivalente ao registrado no *high pool*.

Essa redução na vazão foi ocasionada pelo aumento da latência durante os períodos em que as ações de migração estavam sendo realizadas. Esse fato se comprova pelas Figuras 6.18 e 6.19, as quais representam, respectivamente, as migrações realizadas e a latência das requisições ao longo do tempo.

Figura 6.17: Vazão média de requisições do *profile* de dia útil com o CANM ativado

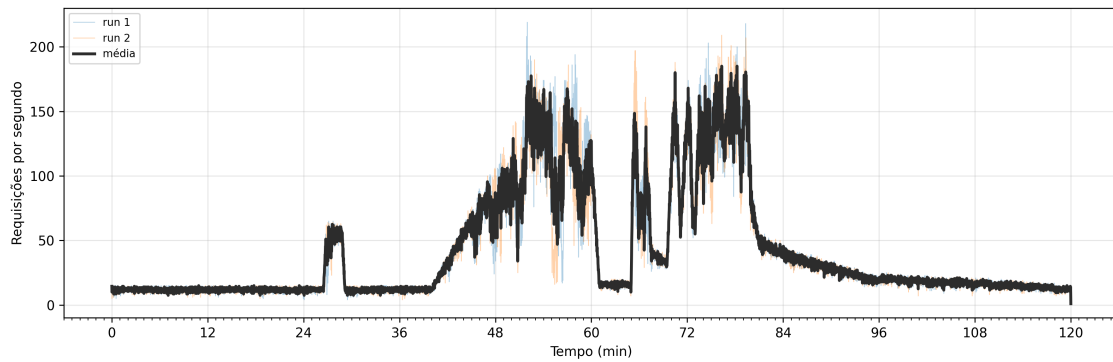
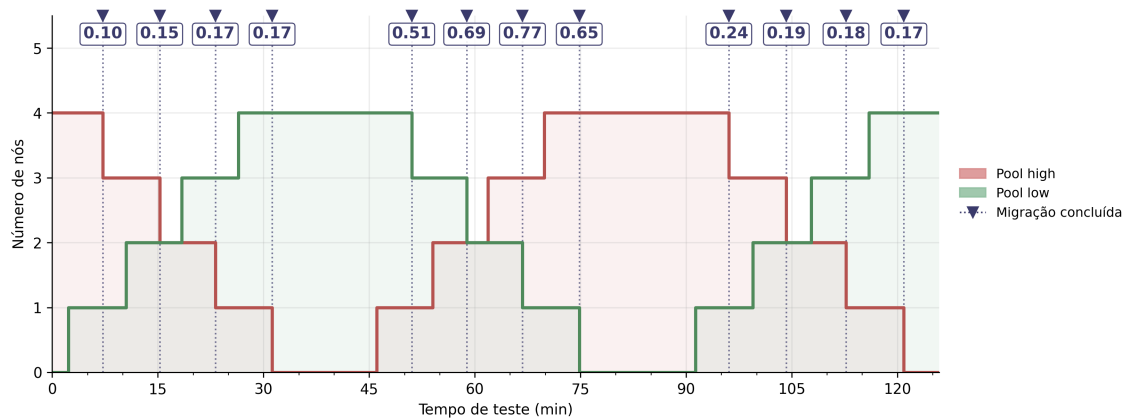


Figura 6.18: Composição dos nós do cluster ao longo do tempo no *profile* de dia útil com o CANM ativado



A Figura 6.18 apresenta as migrações realizadas pelo mecanismo ao longo do tempo, evidenciando tanto o *score* que motivou o início de cada migração (retângulos superiores) quanto a demarcação do seu término (linhas verticais pontilhadas).

Nela, observa-se que entre o minuto 0 e o minuto 30, o CANM identificou a ociosidade na utilização dos recursos do *high pool* – coerente com o baixo volume de requisições apresentado na Figura 6.17. Durante esse período, o mecanismo realizou um total de 4 migrações, com duração aproximada de 7 minutos cada. Verifica-se que, ao detectar a subutilização, o CANM provisiona uma nova máquina no *pool* mais econômico (linha verde) e, assim que essa instância se estabiliza, remove a máquina correspondente do *high pool*, reduzindo o custo associado à manutenção de uma instância de maior desempenho subutilizada.

Em contrapartida, a partir do minuto 45, o CANM detectou o aumento gradual do tráfego referente à simulação do padrão de início de horário comercial. Nesse contexto, dos 45 minutos até aproximadamente os 72 minutos, inicia-se o processo de migração das máquinas do *pool* mais econômico para instâncias de melhor desempenho, visando, desse modo, responder ao aumento do tráfego com máquinas de maior desempenho por núcleo.

Ainda, nota-se por essa figura que, entre os minutos 61 e 65 – intervalo correspondente à simulação do vale do horário de almoço – o CANM realizou uma migração do *low pool* para o *high pool*, com *score* de 0,77. No entanto, em um cenário real, esse vale seria muito mais espaçado, seguindo o padrão de carga apresentado na Figura 5.1, no qual o período de almoço corresponde a aproximadamente uma hora de ociosidade. Nesse contexto, o comportamento esperado seria uma migração no sentido inverso, do *high pool* para o *low pool*, com o objetivo de reduzir custos durante a queda de demanda. Contudo, esse comportamento não foi observado nos testes, uma vez que a compressão do experimento de 24 horas para 2 horas reduziu o vale a apenas 4 a 5 minutos – duração insuficiente para que o CANM acionasse a decisão de migração no sentido inverso.

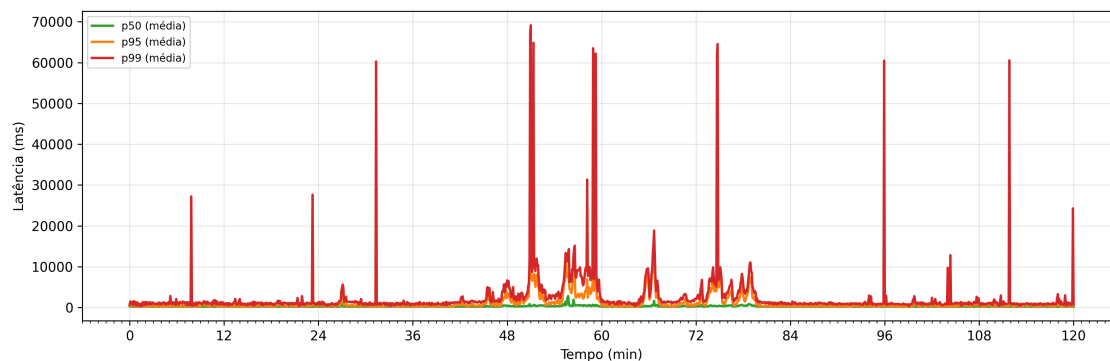
Por fim, a partir do minuto 90, instante em que a carga sintética simula o encerramento do horário comercial e o início do período noturno, o CANM detectou novamente a redução da demanda e iniciou a migração das máquinas de volta para o *pool* mais econômico (*low pool*), retornando o cluster a uma configuração de menor custo durante o período de baixa utilização simulado.

Entretanto, é interessante notar que, embora a redução da demanda tenha tido início no minuto 80, a migração só foi efetivamente iniciada 10 minutos depois. Esse comportamento evidencia a influência do parâmetro `HIGH_POOL_TIME_WINDOW_EVAL`, discutido na Subseção 5.5.2, segundo o qual é necessário observar uma utilização média baixa de recursos ao longo de 10 minutos antes de decidir migrar um nó do *high pool* para o *low pool*. Esse intervalo de observação tem como objetivo evitar migrações precipitadas, desencadeadas por quedas momentâneas de demanda, que poderiam levar a oscilações desnecessárias na composição do cluster e, conseqüentemente, a um custo adicional associado à criação e remoção frequentes de máquinas.

Diante disso, observa-se que o CANM foi capaz de identificar as variações de demanda por recursos ao longo dos diferentes momentos da experimentação, ajustando a composição do cluster de acordo com a carga. Contudo, cabe destacar que o processo de migração teve impactos evidentes na latência (Figura 6.19) e na taxa de erros (Figura 6.20).

Em um primeiro momento, são perceptíveis picos de latência exatamente nos instantes em que o processo de migração estava em curso. Esse comportamento está diretamente relacionado às etapas de migração discutidas na Subseção 4.3.1, em especial à etapa de drenagem, na qual o CANM esvazia o nó de origem para transferir a carga de trabalho a um novo nó. Durante essa etapa, o nó de origem é primeiro marcado como não-agendável (*cordon*) e, em seguida, seus *Pods* são despejados (*eviction*) para

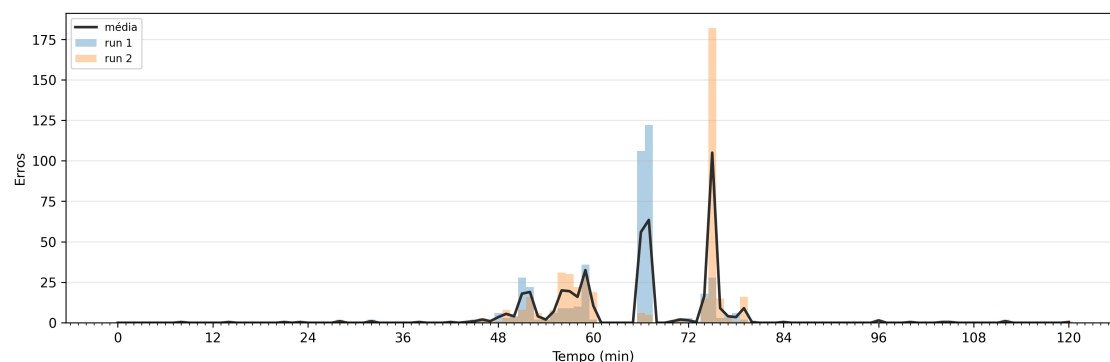
Figura 6.19: Latência de requisições do *profile* de dia útil com o CANM ativado



que sejam reagendados no nó de destino recém-criado (ou em qualquer outro nó com capacidade de aloca-los). Assim, durante esse período o cluster perde momentaneamente parte de sua frota de *Pods*, haja vista que as réplicas despejadas tornam-se temporariamente indisponíveis até que sejam reagendadas, iniciadas e consideradas prontas para receber tráfego no novo nó. Com a redução transitória da capacidade de processamento, o mesmo volume de requisições passa a ser atendido por um número menor de réplicas, o que eleva o tempo de resposta e provoca os picos de latência observados. Uma vez concluída a migração e restabelecida a frota completa de *Pods*, a latência retorna aos patamares anteriores.

Esse cenário agrava-se nos períodos de demanda mais elevada, entre os minutos 40 e 80. Nesses intervalos, ao realizar a migração do *low pool* para o *high pool* com o objetivo de responder ao aumento da carga, o CANM drena um nó que opera sob alta utilização de recursos. Como consequência, os *Pods* nele hospedados são despejados e, enquanto não são reagendados e considerados prontos no nó de destino, a mesma demanda passa a ser atendida por um conjunto reduzido de réplicas. Os nós remanescentes absorvem, portanto, uma carga proporcionalmente maior, o que intensifica a degradação do tempo de resposta e explica os picos de latência mais acentuados observados na Figura 6.19.

Figura 6.20: Erros em requisições do *profile* de dia útil com o CANM ativado



Em relação à taxa de erros, apresentada na Figura 6.20, observa-se que as falhas ocorreram essencialmente pela mesma razão que ocasionou o aumento de latência – perda momentânea de parte da frota de *Pods* durante a drenagem dos nós. Entretanto, é importante distinguir o comportamento conforme o sentido da migração. Nas migrações do *high pool* para o *low pool*, a quantidade de erros manteve-se muito baixa, quase nula. Isso decorre do fato de que, nesses períodos, a demanda encontra-se reduzida, de modo que o conjunto remanescente de réplicas é suficiente para atender às requisições mesmo com a indisponibilidade temporária dos *Pods* em reagendamento. Portanto, conclui-se que ainda que a latência se eleve durante esses intervalos de migração, esse impacto não se propaga em forma de erros. Por outro lado, o impacto é significativamente maior nas migrações do *low pool* para o *high pool*, realizadas justamente para responder ao crescimento da demanda. Nesse cenário, conforme já explicitado, a carga aumenta ao mesmo tempo em que parte da frota de *Pods* é temporariamente perdida durante a drenagem, de forma que o conjunto reduzido de réplicas não consegue absorver o volume de requisições, resultando em falhas no atendimento.

Os valores médios das execuções coletados nesse cenário são apresentados na Tabela 6.3.

Tabela 6.3: Comparativo das métricas médias do *profile* de dia útil entre os cenários de referência e o CANM

Métrica	<i>Low pool</i>	<i>High pool</i>	CANM
Requisições	328.616	298.460	296.332
Erros (média total)	108 (0,0 %)	296 (0,1 %)	448 (0,2 %)
Vazão	45,6 req/s	41,4 req/s	41,2 req/s
Latência média	438,3 ms	583,4 ms	595,6 ms
Latência p50	241,0 ms	218,7 ms	239,4 ms
Latência p95	1287,6 ms	1792,5 ms	1892,8 ms
Latência p99	3404,8 ms	7409,1 ms	6413,2 ms

Comparando a Tabela 6.3 com a dos cenários de referência (Tabela 6.1), nota-se que a vazão obtida com o CANM (296.332 requisições) foi praticamente idêntica à do *high pool* (298.460) e cerca de 10% inferior à do *low pool* (328.616). Esse resultado indica que, ainda que o CANM tenha identificado corretamente o aumento da demanda e migrado a carga para máquinas de maior desempenho, a limitação de E/S discutida anteriormente continuou a restringir a vazão, do mesmo modo observado no *high pool*. A quantidade de erros, por sua vez, foi superior à dos dois cenários de referência (448 contra 108 no *low pool* e 296 no *high pool*), o que decorre diretamente do processo de migração de um

nó do *low pool* sob carga para um nó do *high pool*, que reduz temporariamente a frota de *Pods*, conforme já discutido. Em contrapartida, a latência mediana (p50) manteve-se muito próxima da observada nos demais cenários (239,4 ms, ante 241,0 ms no *low pool* e 218,7 ms no *high pool*), de modo que, na mediana, o CANM apresentou uma latência adequada.

6.2.2 Profile fim de semana

No *profile* de fim de semana, a análise é consideravelmente mais simples, uma vez que o cenário é trivial. Nesse sentido, a carga caracteriza-se por uma utilização baixa e constante, interrompida apenas por picos esporádicos, conforme o padrão definido na Subseção 5.4.1. A Figura 6.21 confirma esse comportamento, com um total de apenas 92.772 requisições processadas ao longo de toda a experimentação.

Figura 6.21: Vazão média de requisições do *profile* de fim de semana com o CANM ativado

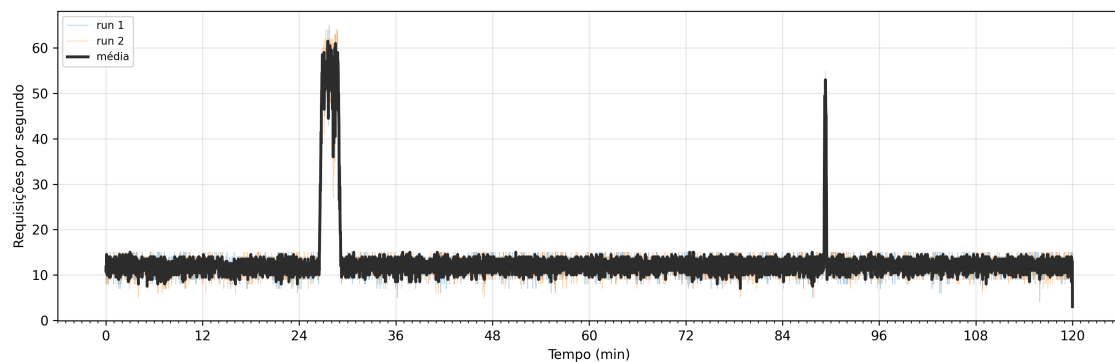
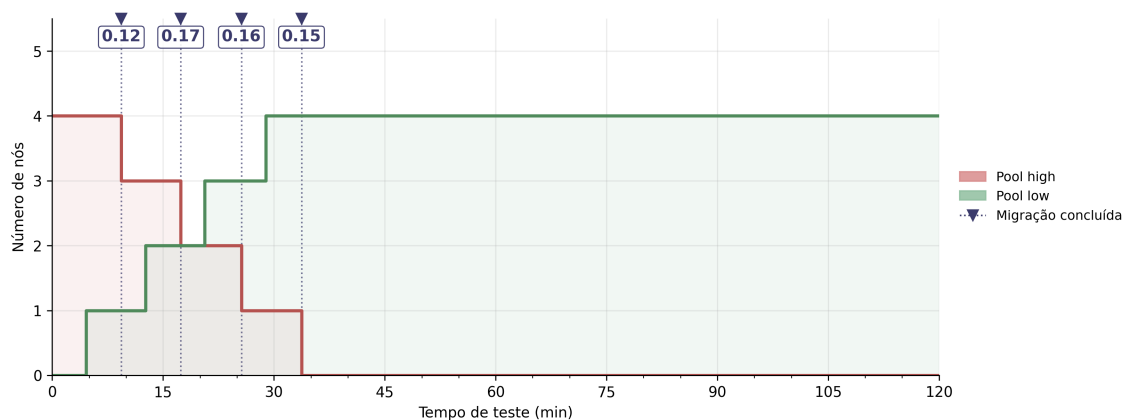


Figura 6.22: Composição dos nós do cluster ao longo do tempo no *profile* de fim de semana com o CANM ativado



Como as máquinas foram provisionadas inicialmente no *high pool*, o CANM, ao detectar a ociosidade característica do fim de semana, realizou a migração para o *low pool*, reduzindo o custo de operação (Figura 6.22). Cabe ressaltar que esse ponto de

partida decorre do isolamento do experimento, uma vez que em um cenário contínuo e realista – encadeado, por exemplo, ao *profile* de uma sexta-feira –, essas instâncias muito provavelmente já estariam no *low pool*, visto que o CANM teria detectado a ociosidade durante a noite de sexta e migrado a frota antecipadamente.

Figura 6.23: Erros em requisições do *profile* de fim de semana com o CANM ativado

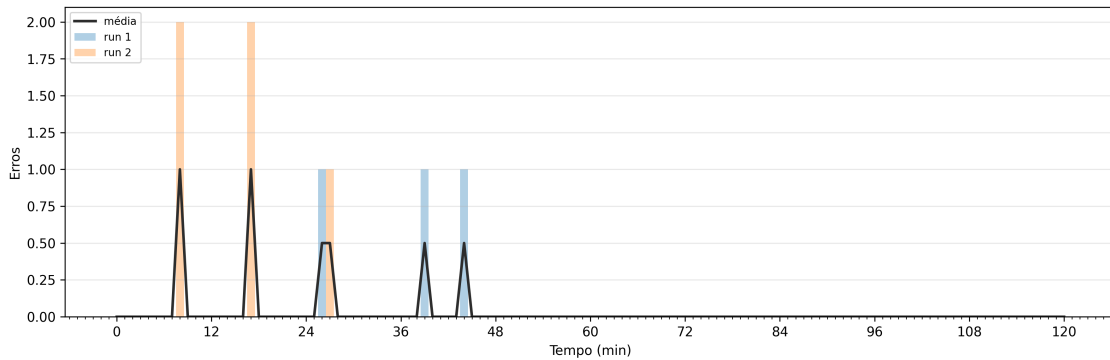
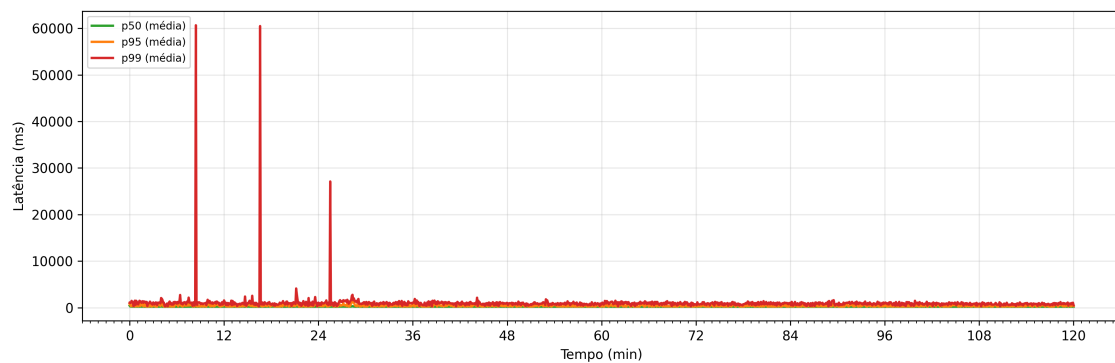


Figura 6.24: Latência de requisições do *profile* de fim de semana com o CANM ativado



Quanto à latência (Figura 6.24) e à taxa de erros (Figura 6.23), ambas mantiveram-se baixas durante toda a execução – foram registradas, em média, apenas 4 falhas no total. Conforme já discutido no *profile* de dia útil, na ausência de demanda expressiva o cluster praticamente não é afetado pelas migrações. Assim, o conjunto remanescente de réplicas é mais do que suficiente para atender ao baixo volume de requisições, de modo que a perda momentânea de *Pods* durante a drenagem não se traduz em degradação perceptível do serviço. As métricas médias desse cenário, em comparação com os experimentos de referência, são apresentadas na Tabela 6.4, na qual se observa que os três cenários apresentaram resultados muito próximos entre si.

Nesse cenário, o CANM apresentou um resultado excelente, praticamente equiparando-se aos dois cenários de referência em todas as métricas avaliadas. Esse comportamento comprova que as migrações de nós do *high pool* para o *low pool* – predominantes em períodos de baixa demanda – impõem uma degradação praticamente nula ao sistema,

Tabela 6.4: Comparativo das métricas médias do *profile* de fim de semana entre os cenários de referência e o CANM

Métrica	<i>Low pool</i>	<i>High pool</i>	CANM
Requisições	93.572	93.510	92.772
Erros (média total)	6 (0,0 %)	2 (0,0 %)	4 (0,0 %)
Vazão	13,0 req/s	13,0 req/s	12,9 req/s
Latência média	243,6 ms	244,6 ms	254,4 ms
Latência p50	208,6 ms	204,6 ms	209,5 ms
Latência p95	312,5 ms	325,3 ms	418,8 ms
Latência p99	1119,0 ms	1282,0 ms	1245,3 ms

justamente por ocorrerem quando o conjunto remanescente de réplicas é suficiente para absorver a carga.

6.3 Economia de custo

O custo de provisionamento das máquinas ao longo dos experimentos define os limites entre os quais se avalia a economia proporcionada pelo CANM. Em cada cenário de experimentação no *low pool*, o custo de provisionamento de 4 máquinas *e2-custom-2-16384* localizadas na região *us-central1* executando ao longo de um período de duas horas corresponde a ~US\$ 0,72. Esse valor considera apenas o preço sob demanda de cada instância – totalizando US\$ 0,09039966 por hora¹ – multiplicado pelas 4 máquinas e pelas 2 horas de execução. O custo de armazenamento em SSD foi desconsiderado, por se tratar de um valor fixo e idêntico em todos os testes.

Cabe destacar que esse valor representa o custo mínimo de provisionamento da infraestrutura proposta, uma vez que corresponde à operação contínua das 4 instâncias no *pool* mais econômico durante todo o período de experimentação. Dessa forma, não se espera que as experimentações com operação do CANM apresentem custo inferior a esse, servindo tal valor como limite inferior de referência para a avaliação de economia obtida pela ferramenta.

Já em cada cenário de experimentação no *high pool*, o custo de provisionamento de 4 máquinas *n2-custom-2-16384* localizadas na região *us-central1* executando ao longo de um período de duas horas corresponde a ~US\$ 1,05. Esse valor considera apenas o

¹Preço sob demanda da instância *e2-highmem-2* (família *general-purpose*), de especificação equivalente à *e2-custom-2-16384*, na região *us-central1* (Iowa). Disponível em: <<https://cloud.google.com/products/compute/pricing/general-purpose#e2-machine-types>>. Acesso em: 14 jun. 2026.

preço sob demanda de cada instância – totalizando US\$ 0,131014 por hora² – multiplicado pelas 4 máquinas e pelas 2 horas de execução. Esse cenário, correspondente a um cluster homogêneo composto integralmente por máquinas de maior desempenho, constitui o teto de custo da avaliação.

Para avaliar o impacto financeiro da atuação do CANM, comparou-se o custo de computação de cada *profile* com e sem o mecanismo ativo. O cenário *sem CANM* corresponde a manter as 4 máquinas no *high pool* durante todo o experimento, ao passo que o cenário *com CANM* corresponde ao custo real integrado a partir da composição de nós observada ao longo do tempo. O custo de cada cenário é calculado somando, ao longo do tempo, o preço por hora de cada máquina ativa em seu respectivo *pool*. Considera-se também o breve intervalo em que o cluster opera com 5 máquinas, já que o CANM cria o nó de destino antes de remover o nó de origem. A Tabela 6.5 demonstra os resultados.

Tabela 6.5: Comparativo de custo de computação com e sem a atuação do CANM

<i>Profile</i>	Horizonte	Sem CANM (US\$)	Com CANM (US\$)	Economia
Dia útil	2 h	1,0481	0,9977	4,8 %
Fim de semana	2 h	1,0481	0,8105	22,7 %
Dia útil	24 h	12,5773	10,3028	18,1 %
Fim de semana	24 h	12,5773	8,8404	29,7 %

Observa-se que, mesmo no horizonte de 2 horas, o CANM já reduz o custo em todos os cenários, com economia mais expressiva no *profile* de fim de semana (22,7 %), em que a baixa demanda permite manter a frota predominantemente no *pool* mais econômico. No *profile* de dia útil, a economia em 2 horas é modesta (4,8 %), pois o cluster permanece boa parte do tempo no *pool* caro para atender aos picos e ainda arca com o sobrecusto das migrações de ida e volta.

É importante destacar, contudo, que a economia proporcionada pelo CANM tende a crescer à medida que o horizonte de operação aumenta. Isso decorre do fato de que o sobrecusto das migrações – em especial o intervalo de coexistência de 5 máquinas – é determinado por tempo de relógio real e permanece praticamente constante (cerca de 58 minutos no dia útil e 19 minutos no fim de semana), ao passo que os períodos de ociosidade, nos quais a frota permanece no *pool* barato, se prolongam proporcionalmente ao tempo total. Desse modo, o custo fixo das migrações é diluído ao longo de janelas de economia cada vez maiores.

²Preço sob demanda da instância *n2-highmem-2* (família *general-purpose*), de especificação equivalente à *n2-custom-2-16384*, na região *us-central1* (Iowa). Disponível em: <<https://cloud.google.com/products/compute/pricing/general-purpose#n2-machine-types>>. Acesso em: 14 jun. 2026.

Para evidenciar esse efeito, projetou-se o custo para um horizonte de 24 horas mantendo exatamente as mesmas decisões de migração tomadas pelo CANM durante o experimento de 2 horas. Como a carga sintética utilizada nos testes comprime um dia inteiro em 2 horas (Subseção 5.4.1), essa projeção corresponde a desfazer a compressão temporal. As Figuras 6.25 e 6.26 apresentam a composição projetada do cluster ao longo de 24 horas para os *profiles* de dia útil e de fim de semana, respectivamente. O resultado confirma a tendência esperada, com a economia saltando de 4,8 % para 18,1 % no dia útil e de 22,7 % para 29,7 % no fim de semana, o que demonstra que o benefício do mecanismo é tão maior quanto mais longo for o período de operação do cluster.

Vale destacar, por fim, a existência de um **limite teórico** de economia. Esse limite corresponde ao piso de custo da infraestrutura, alcançado quando todas as 4 máquinas permanecem integralmente no *low pool* ($4 \times e2$), o que resulta em US\$ 0,7232 no horizonte de 2 horas e US\$ 8,6784 no de 24 horas. Esse cenário estabelece o teto de economia possível, de 31,0 % em relação ao cluster homogêneo no *high pool* – valor que equivale exatamente à razão entre os preços das duas famílias de máquinas ($1 - 0,09039966/0,131014$) e que, por isso, independe do horizonte de operação. Contudo, esse limite é inatingível na prática sempre que houver demanda significativa, uma vez que o CANM necessita manter parte da frota no *high pool* para atender à carga sem violar os requisitos de desempenho. Nesse sentido, a economia obtida no *profile* de fim de semana em 24 horas (29,7 %) aproxima-se bastante desse teto, ao passo que a do dia útil (18,1 %) permanece mais distante, refletindo o tempo necessário no *pool* de maior desempenho durante os picos de demanda.

Figura 6.25: Composição projetada do cluster ao longo de 24 horas no *profile* de dia útil com o CANM ativado

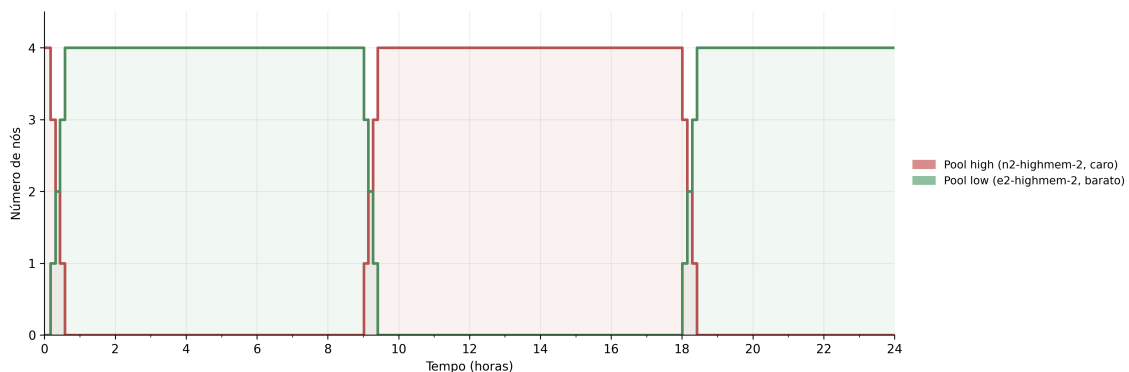
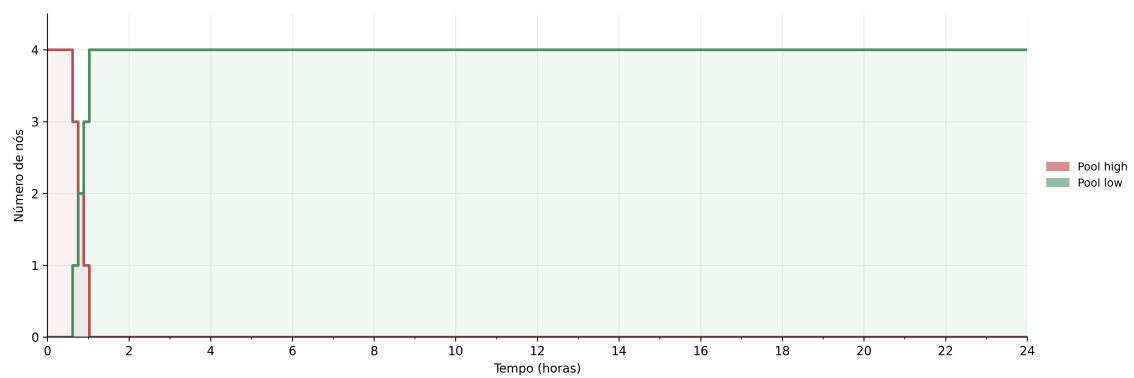


Figura 6.26: Composição projetada do cluster ao longo de 24 horas no *profile* de fim de semana com o CANM ativado



7 CONCLUSÃO E TRABALHOS FUTUROS

Com base nos resultados apresentados no Capítulo 6, é perceptível que o CANM cumpriu o objetivo a que se propôs, realizando a migração de cargas de trabalho entre *pools* de diferentes capacidades a fim de reduzir o custo de operação do cluster. Conforme observado nos experimentos, essa redução de custo foi acompanhada de impactos transitórios de latência e de erros durante as migrações, especialmente no sentido *low pool* para *high pool*, realizado sob alta demanda. Em contrapartida, nas migrações do *high pool* para o *low pool*, predominantes em períodos de baixa utilização, a degradação mostrou-se praticamente nula. Os resultados evidenciaram ainda que o desempenho do mecanismo é fortemente dependente tanto da arquitetura do software em execução – em especial de sua sensibilidade a operações de E/S, identificada como o real fator limitante dos serviços avaliados – quanto das características do próprio cluster, como a imagem do sistema operacional dos nós e a quantidade de *Pods* a serem reagendados, fatores que influenciam diretamente o tempo de cada migração. Como trabalhos futuros, destacam-se a experimentação do CANM em outros cenários e arquiteturas de aplicação, a investigação de uma migração preditiva capaz de antecipar os picos de demanda e drenar os nós antes da chegada da carga e a definição de políticas de migração cientes do *downstream* e do compromisso entre custo e orçamento de erros.

REFERÊNCIAS

- AHMAD, H. et al. Towards resource-efficient reactive and proactive auto-scaling for microservice architectures. **Journal of Systems and Software**, v. 225, p. 112390, 2025.
- Amazon Web Services. **Spot Instances**. 2026. Acesso em: 30 mar. 2026. Available from Internet: <<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-spot-instances.html>>.
- ARMBRUST, M. et al. A view of cloud computing. **Communications of the ACM**, Association for Computing Machinery, v. 53, n. 4, p. 50–58, 2010. Available from Internet: <<https://doi.org/10.1145/1721654.1721672>>.
- BAARZI, A. F.; KESIDIS, G. Showar: Right-sizing and efficient scheduling of microservices. In: **Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)**. [S.l.: s.n.], 2021. p. 427–441.
- BURNS, B. et al. Borg, omega, and kubernetes. **Communications of the ACM**, Association for Computing Machinery, v. 59, n. 5, p. 50–57, 2016. Available from Internet: <<https://doi.org/10.1145/2890784>>.
- CHUNG, A.; PARK, J. W.; GANGER, G. R. Stratus: Cost-aware container scheduling in the public cloud. In: **Proceedings of the ACM Symposium on Cloud Computing (SoCC '18)**. [S.l.: s.n.], 2018.
- CORTEZ, E. et al. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In: **Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)**. New York, NY, USA: Association for Computing Machinery, 2017. p. 153–167. Available from Internet: <<https://doi.org/10.1145/3132747.3132772>>.
- Docker. **What is Docker?** 2026. Acesso em: 18 jun. 2026. Available from Internet: <<https://docs.docker.com/get-started/docker-overview/>>.
- Google Cloud. **Tipos de máquinas**. 2026. Acesso em: 30 mar. 2026. Available from Internet: <<https://docs.cloud.google.com/compute/docs/machine-resource?hl=pt-br>>.
- Heroku. **Heroku: The Cloud Application Platform**. 2026. Acesso em: 18 jun. 2026. Available from Internet: <<https://www.heroku.com/>>.
- Kubernetes. **Pods**. 2026. Acesso em: 30 mar. 2026. Available from Internet: <<https://kubernetes.io/docs/concepts/workloads/Pods/>>.
- KURANAGE, M. P. J. et al. Ai-assisted proactive scaling solution for cnfs deployed in kubernetes. In: **2023 IEEE 12th International Conference on Cloud Networking (CloudNet)**. [S.l.: s.n.], 2023.
- LAKSHAN, S.; HUSSAIN, S. A review of ai-driven techniques for cost optimization in kubernetes environments. In: **2025 International Research Conference on Smart Computing and Systems Engineering (SCSE)**. [S.l.: s.n.], 2025. p. 1–5.

MELL, P.; GRANCE, T. **The NIST Definition of Cloud Computing**. Gaithersburg, MD, 2011. Acesso em: 18 jun. 2026. Available from Internet: <<https://doi.org/10.6028/NIST.SP.800-145>>.

Microsoft. **What is a virtual machine (VM)?** 2026. Acesso em: 29 mar. 2026. Available from Internet: <<https://azure.microsoft.com/>>.

REISS, C. et al. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In: **Proceedings of the Third ACM Symposium on Cloud Computing (SoCC '12)**. New York, NY, USA: Association for Computing Machinery, 2012. p. 7. Available from Internet: <<https://doi.org/10.1145/2391229.2391236>>.

STRAESSER, M. et al. Autoscaler evaluation and configuration: A practitioner's guideline. In: **Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE '23)**. New York, NY, USA: Association for Computing Machinery, 2023. p. 31–41. ISBN 979-8-4007-0068-2. Available from Internet: <<https://doi.org/10.1145/3578244.3583721>>.

TAMIRU, M. A. et al. An experimental evaluation of the kubernetes cluster autoscaler in the cloud. In: **2020 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)**. [S.l.: s.n.], 2020. p. 17–24.

TRAN, M.-N.; VU, D.-D.; KIM, Y. A survey of autoscaling in kubernetes. In: **2022 Thirteenth International Conference on Ubiquitous and Future Networks (ICUFN)**. [S.l.: s.n.], 2022. p. 263–265.

WANG, M.; ZHANG, D.; WU, B. A cluster autoscaler based on multiple node types in kubernetes. In: **2020 IEEE 4th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC)**. [S.l.: s.n.], 2020. v. 1.

ZHONG, Z.; BUYYA, R. A cost-efficient container orchestration strategy in kubernetes-based cloud computing infrastructures with heterogeneous resources. **ACM Transactions on Internet Technology**, 2020.